



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

MÉDIA- ÉS OKTATÁSI INFORMATIKA TANSZÉK

Interaktív Tic-Tac-Toe játék döntési
algoritmusokkal és működésük
szemléltetésével

Témavezető:

Menyhárt László Gábor
egyetemi adjunktus

Szerző:

Gerzsényi Levente Márton
programtervező informatikus BSc

Budapest, 2026

SAKDOLOGOZAT TÉMABEJELENTŐ

Hallgató adatai:

Név: Gerzsényi Levente Márton

Neptun kód: GIWFE2

Képzési adatok:

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Menyhárt László Gábor

munkahelyének neve, tanszéke: ELTE IK, Média- és Oktatásinformatika Tanszék

munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.

beosztás és iskolai végzettsége: egyetemi adjunktus

A szakdolgozat címe: Interaktív Tic-Tac-Toe játék döntési algoritmusokkal és működésük szemléltetésével

A szakdolgozat témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

A szakdolgozat célja egy olyan alkalmazás megvalósítása, amelyben a felhasználó különböző döntési algoritmusokkal vagy neurális hálókkal szemben játszhat Tic-Tac-Toe játékot. Az ellenfelek között megtalálható a Minimax algoritmus, Donald Michie MENACE modellje, egy visszaterjesztéssel tanított neurális hálózat, egy genetikusan optimalizált neuronháló, valamint egy véletlenszerű döntéseket hozó játékos is. A felhasználó kiválaszthatja az ellenfelet, a játszmák számát, illetve beállíthatja a lépések sebességét is. A rendszer lehetőséget biztosít arra is, hogy az algoritmusok egymás ellen mérkőzzenek meg.

A program törekszik az egyes algoritmusok működésének bemutatására is, amelyet a szöveges magyarázatokon felül rajzokkal és egyszerű animációkkal támogat. Például a MENACE modell esetében egy adott játékalánál megjelenik a megfelelő doboz és az abban található színes üveggolyók, majd ezek arányának megfelelően szeletekre osztott szerencsekerék megpördül, és ennek eredménye alapján kerül kiválasztásra a következő lépés.

A felhasználónak lehetősége van saját neurális háló létrehozására, ahol megadhatja a rétegszámot, a neuronok számát, a kezdeti súlyokat és eltolásokat, valamint beállíthatja a tanítási iterációk számát. A tanítás eredményeként frissült hálózatot a felhasználó elmentheti, és ellenfélként is használhatja a játékban.

Budapest, 2025. 05. 24.

Tartalomjegyzék

1. Bevezetés	3
1.1. Előszó	3
1.2. A Tic-Tac-Toe játék	3
1.2.1. Ismertetés	3
1.2.2. Játékszabályok	3
1.3. Mesterséges intelligencia	4
1.4. Célközönség	4
2. Felhasználói dokumentáció	5
2.1. Rövid leírás	5
2.2. Rendszerkövetelmények és használat	5
2.2.1. Rendszerkövetelmények	5
2.2.2. Használat	6
2.3. Navigáció az alkalmazásban	7
2.3.1. Navigációs sor	7
2.3.2. Egyéb navigációs elemek	7
2.3.3. Lapdiagram	8
2.4. Lapok és funkciók	9
2.4.1. Kezdőlap	10
2.4.2. Tanulás	16
2.4.3. Regisztráció	29
2.4.4. Bejelentkezés	30
2.4.5. Kezdőlap - bejelentkezett felhasználónak	32
2.4.6. Barkácsolás - csak bejelentkezett felhasználónak	33
2.5. Bemenetek és hibaüzenetek	38
2.5.1. Bemenetek	38
2.5.2. Hibaüzenetek	40

3. Fejlesztői dokumentáció	43
3.1. Követelményleírás	43
3.1.1. Funkcionális követelmények	43
3.1.2. Nem funkcionális követelmények	50
3.2. Megvalósítás	52
3.2.1. Prezentációs réteg	53
3.2.2. Üzletilogika-réteg	64
3.2.3. Adatelérési réteg	94
3.3. Tesztelés	99
3.3.1. Automatikus	99
3.3.2. Manuális	104
3.4. Futtatás lokális környezetben	104
3.4.1. Forráskód letöltése	104
3.4.2. Alkalmazás futtatása	104
4. Összegzés	108
4.1. További fejlesztési lehetőségek	108
Irodalomjegyzék	110
Ábrajegyzék	114
Táblázatjegyzék	117
Algoritmusjegyzék	118

1. fejezet

Bevezetés

1.1. Előszó

A mesterséges intelligencia (MI) egyre nagyobb szerepet tölt be napjainkban. A szakdolgozatom célja egy webalkalmazás fejlesztése, amely lehetővé teszi a felhasználók számára, hogy különböző MI-megközelítéseket ismerjenek meg és teszteljenek egy ismert játék, a Tic-Tac-Toe (amőba) keretében.

1.2. A Tic-Tac-Toe játék

1.2.1. Ismertetés

A Tic-Tac-Toe egy egyszerű kétszemélyes játék, amelyet egy 3x3-as táblán játszanak, első változatai az ókori egyiptomból származnak. A játék, amennyiben a játékosok optimálisan játszanak, mindig döntetlennel végződik. Zéró összegű és véges jellege miatt ideális algoritmusok és mesterségesintelligencia-megközelítések tesztelésére.

1.2.2. Játékszabályok

Az egyik játékos az X -et, a másik pedig az O -t használja. A játékot az X játékos kezdi, és felváltva helyezik el jelüket a táblán. Az a játékos nyer, aki először helyez el három azonos jelet vízszintesen, függőlegesen vagy átlósan. A játék döntetlennel végződik, ha a tábla megtelik anélkül, hogy bármelyik játékos nyerne.

1.3. Mesterséges intelligencia

A mesterséges intelligencia (MI) célja olyan rendszerek létrehozása, amelyek képesek emberi intelligenciát igénylő feladatok elvégzésére.

A korai MI-kutatásokban a problémamegoldást főként keresési algoritmusokkal valósították meg. Ilyen algoritmus a Minimax, amely a játékokban optimális lépéseket keres.

A MENACE (Matchbox Educable Noughts and Crosses Engine) már a gépi tanulás egy kezdetleges formáját képviselte, mivel tapasztalatból "tanulta meg" az optimális lépéseket.

A modern MI-rendszerek alapelemei a neurális hálózatok, amelyek tanításához többek között a visszaterjesztés és genetikusan algoritmus is alkalmazhatók.

1. Definíció. A dolgozatban a **visszaterjesztés** kifejezés a neurális hálózatok tanítására szolgáló, láncszabályon alapuló eljárást jelöli, amelynek során meghatározzuk a veszteségfüggvény gradiensét a hálózat paramétereire nézve, majd ezt felhasználva gradienacsökkenéssel frissítjük a paramétereket.

1.4. Célközönség

Az alkalmazás azoknak készült, akik érdeklődnek a mesterséges intelligencia iránt, és szeretnének megismerkedni néhány megközelítés alapjával egy interaktív környezetben. Bizonyos részek megértéséhez az olvasónak olyan matematikai előismeretekkel kell rendelkeznie, amelyeket a dolgozat nem tárgyal részletesen.

2. fejezet

Felhasználói dokumentáció

2.1. Rövid leírás

A weboldalon a felhasználónak lehetősége van Tic-Tac-Toe játékot játszani öt alapértelmezett ellenfél ellen. A játék beállításait testreszabhatja. Az ellenfélként használt algoritmusok működését szöveges magyarázatok, ábrák és interaktív elemek segítségével ismerheti meg. Regisztráció után saját neurális hálózatokat is létrehozhat, taníthat és használhat a játék során.

2.2. Rendszerkövetelmények és használat

2.2.1. Rendszerkövetelmények

Hardverkövetelmények

- **Processzor:** Bármely modern processzor.
- **Memória:** Minimum 2 GB RAM.
- **Böngészőablak mérete:** Minimum 360 pixel széles¹ (ajánlott legalább 1024 pixel széles).
- **Hálózat:** Aktív internetkapcsolat.

Szoftverkövetelmények

Az alkalmazás platformfüggetlen, bármely modern operációs rendszeren futtatható az alábbi böngészők használatával:

¹Az alkalmazás ellenőrzi a böngészőablak méretét betöltéskor. Ha az ablak szélessége kisebb, mint 360 pixel, egy figyelmeztető üzenet jelenik meg.

- **Támogatott böngészők:**

- Google Chrome 80+
- Mozilla Firefox 80+
- Microsoft Edge 80+
- Safari 14.1+
- Opera 67+

- **Egyéb feltételek:** A böngészőben engedélyezni kell a *JavaScript* futtatását és a *sütik* fogadását.

2.2.2. Használat

Az alkalmazás használatához telepítésre nincs szükség, internetkapcsolattal melllett böngészőből érhető el.

Lépések:

1. Böngésző megnyitása.
2. *Az alkalmazás webes felülete*[1] oldal megnyitása.
3. Az alkalmazás azonnal használható, és első indításkor vendég módban indul.

2.3. Navigáció az alkalmazásban

2.3.1. Navigációs sor

A képernyő tetején található navigációs sáv mindig elérhető, és a következő menüpontokat tartalmazza:

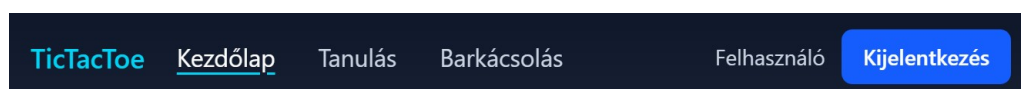
Megnevezés	Elérhetőség
<i>Kezdőlap</i>	Mindenki számára elérhető.
<i>Tanulás</i>	Mindenki számára elérhető.
<i>Barkácsolás</i>	Csak bejelentkezett felhasználóknak elérhető.
<i>Bejelentkezés</i>	Csak nem bejelentkezett felhasználóknak elérhető.
<i>Kijelentkezés</i>	Csak bejelentkezett felhasználóknak elérhető.

2.1. táblázat. Navigációs menü áttekintése

Megjegyzés. A *Bejelentkezés* és *Kijelentkezés* menüpontok nem láthatóak egyszerre.



2.1. ábra. Navigációs menü vendég felhasználó számára



2.2. ábra. Navigációs menü bejelentkezett felhasználó számára

Kisebbs képernyőméret esetén a navigációs elemek egy hamburger menüben érhetők el.

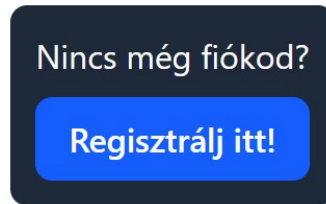


2.3. ábra. Hamburger menü

2.3.2. Egyéb navigációs elemek

A navigációs sávon kívüli navigációs elemek:

- **Regisztráció:** A bejelentkezés lapon található gomb, a regisztrációs lapra vezet.



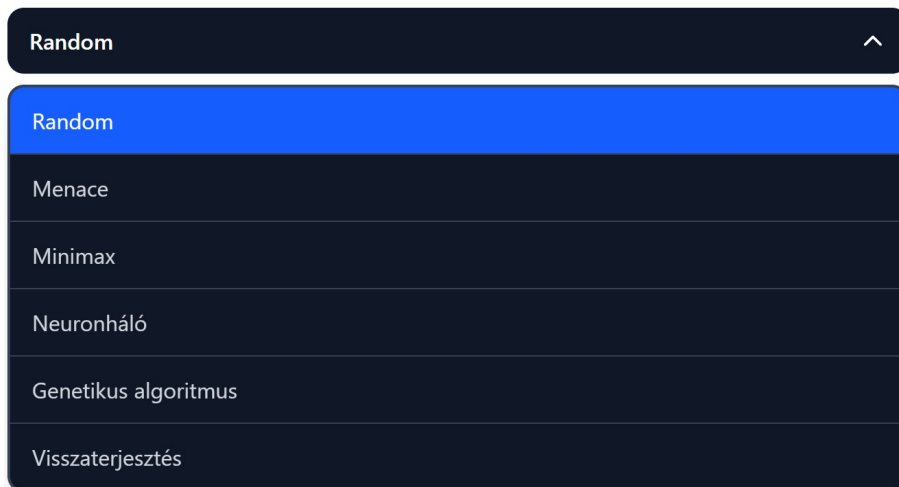
2.4. ábra. Navigációs gomb regisztrációhoz

- **Témaválasztás:** A *Tanulás* lapon található legördülő menü segítségével lehet váltani a témák között.



2.5. ábra. Legördülő menü

A menüre kattintva megjelenik az összes téma:

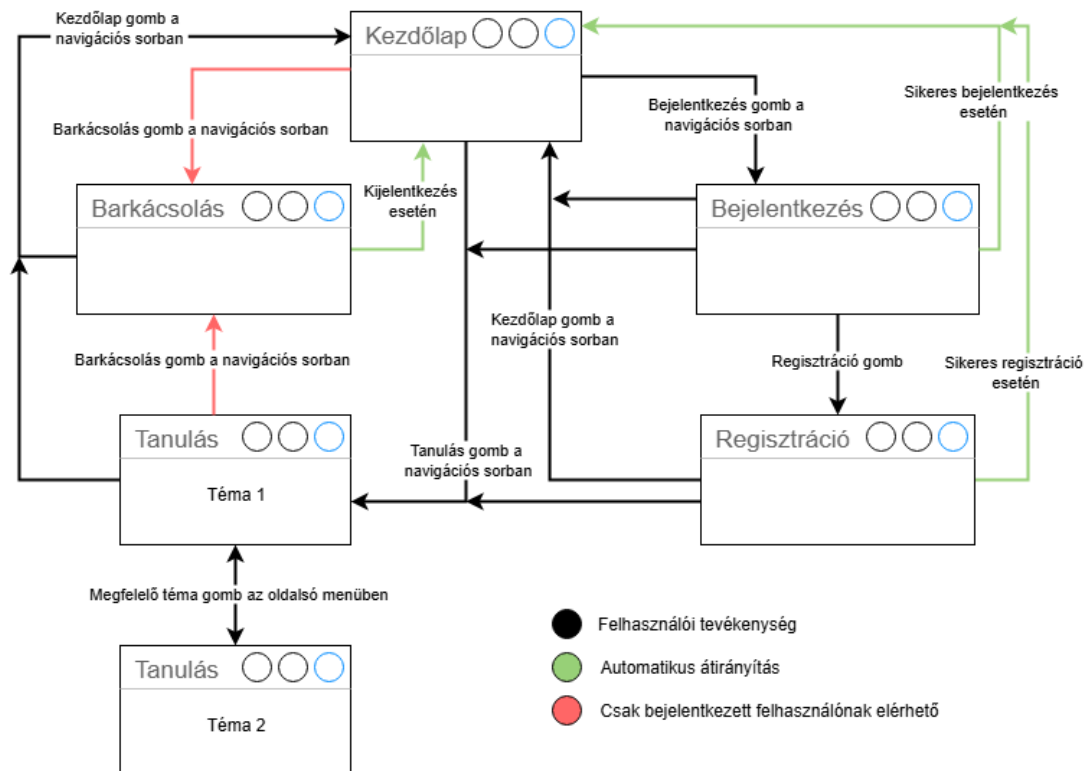


2.6. ábra. Legördülő menü kinyitva

Az egyes témákra kattintva megjelenik a megfelelő tartalom a *Tanulás* lapon.

2.3.3. Lapdiagram

Az alkalmazásban elérhető lapokat és a navigáció irányait az alábbi diagram foglalja össze:

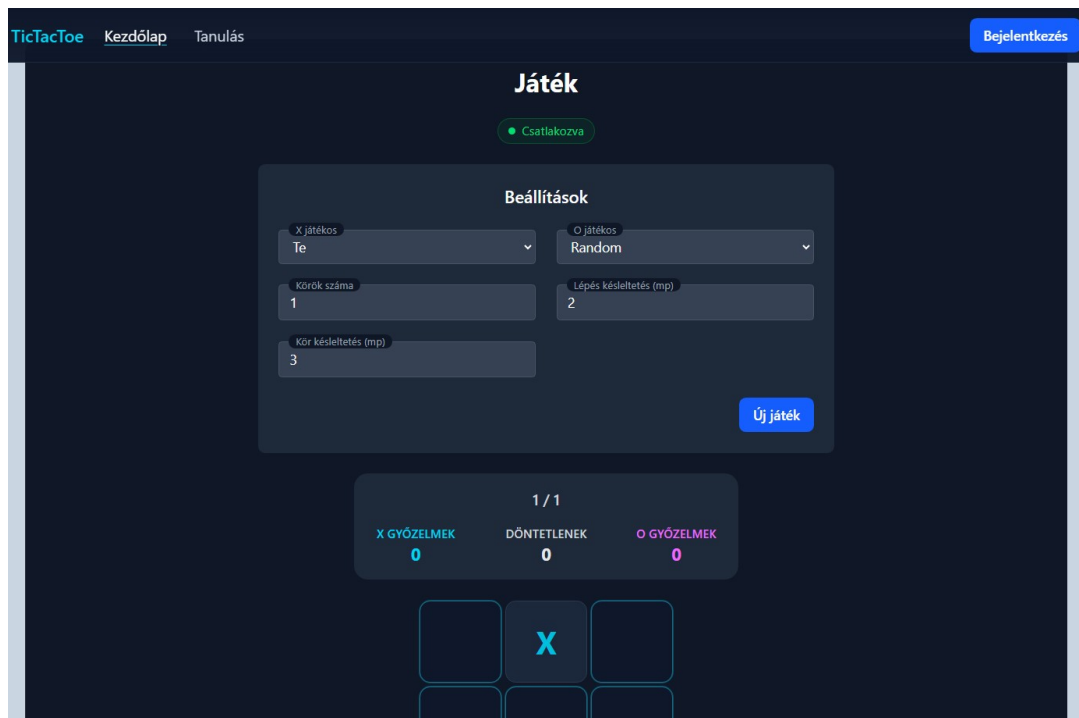


2.7. ábra. Az alkalmazás lapjai és navigációs útvonalai

2.4. Lapok és funkciók

Az alkalmazást vendég, illetve bejelentkezett felhasználói módban is lehet használni. A lapok és funkciók ebben a fejezetben kerülnek bemutatásra. Először **vendég módban** mutatom be a lapokat, végül a **bejelentkezett felhasználói mód** extra funkciói kerülnek ismertetésre.

2.4.1. Kezdőlap



2.8. ábra. Az alkalmazás kezdőlapja

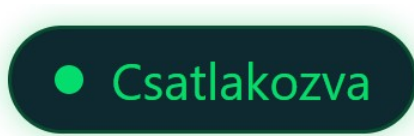
A *Kezdőlap*on érhető el a Tic-Tac-Toe játék. Itt a felhasználó megadhatja a játék beállításait, és elindíthatja a játékot.

Csatlakozási státusz

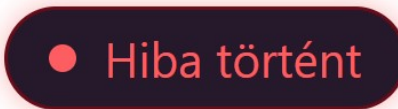
A játék indításához WebSocket kapcsolat szükséges. A lap betöltésekor a kapcsolat automatikusan kezdeményeződik. A kapcsolat státusza a képernyő tetején látható. A státusz a következők egyike lehet:

Státusz	Leírás
<i>Csatlakozás</i>	A kapcsolat létrehozása folyamatban van.
<i>Csatlakozva</i>	A kapcsolat létrejött, a játék elindítható.
<i>Hiba történt</i>	A kapcsolat létrehozása közben hiba lépett fel.
<i>Nincs kapcsolat</i>	A kapcsolat megszakadt. Az ikon kattintásával újrapróbálkozik a rendszer.

2.2. táblázat. Csatlakozási státuszok



2.9. ábra. „Csatlakozva” státusz ikon



2.10. ábra. „Hiba történt” státusz ikon

Játékbeállítások

A játék indítása előtt a következő beállítások adhatók meg:

- **Első játékos:** Választható a felhasználó vagy az 5 ellenfél közül.
- **Második játékos:** Választható a felhasználó vagy az 5 ellenfél közül.
- **Játék köreinek száma:** Megadható, hogy hány körből álljon a játék.
- **Lépések közötti szünet:** Beállítható, hogy mennyi idő teljen el a lépések között.
- **Körök közötti szünet:** Beállítható, hogy mennyi idő teljen el a körök között.

Az öt ellenfél a következő:

- **Random**
- **MENACE**
- **Minimax**
- **Genetikus** (genetikus algoritmussal tanított neurális hálózat)
- **Visszaterjesztés** (visszaterjesztéssel tanított neurális hálózat)

Megszorítások:

Megnevezés	Elérhetőség
<i>Játékosok</i>	A felhasználó nem választható mindkét játékosnak.
<i>Körök száma</i>	1 és 100 között választható.
<i>Lépések közötti szünet</i>	0,02 és 10 másodperc között választható.
<i>Körök közötti szünet</i>	0,02 és 10 másodperc között választható.

2.3. táblázat. Játékbeállítások megszorításai

The screenshot shows a dark-themed settings panel titled 'Beállítások'. It contains five input fields arranged in two columns. The first column has 'X játékos' (Te) and 'Körök száma' (1). The second column has 'O játékos' (Random) and 'Lépés késleltetés (mp)' (2). Below these, on the left, is 'Kör késleltetés (mp)' (3). All fields have a dark blue background and white text.

2.11. ábra. A játékbeállítások panelje

Játék indítása és beállítások módosítása

A beállítások panel alján található **Játék létrehozása** gombra kattintva a játék a megadott beállításokkal indul el.

This screenshot is identical to the previous one, showing the 'Beállítások' panel. The only difference is that a blue button labeled 'Játék létrehozása' is now visible in the bottom right corner of the panel.

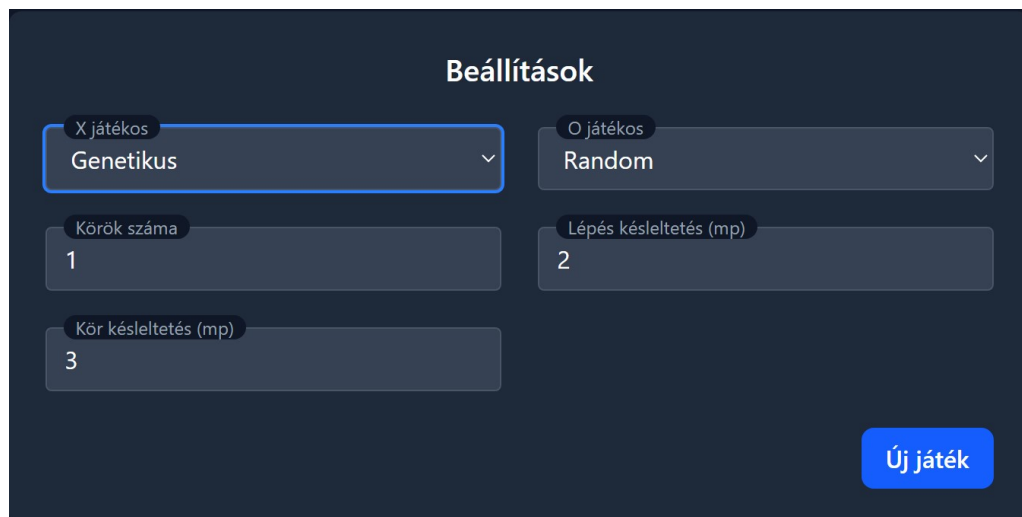
2.12. ábra. „Játék létrehozása” gomb

A játék elindulása után a beállítások panelen az **Új játék** gomb jelenik meg, amellyel egy új játék indul az aktuális beállításokkal.

Játék közben a beállítások módosíthatóak a helyzetnek megfelelő gombbal. Ezek az alábbiak lehetnek:

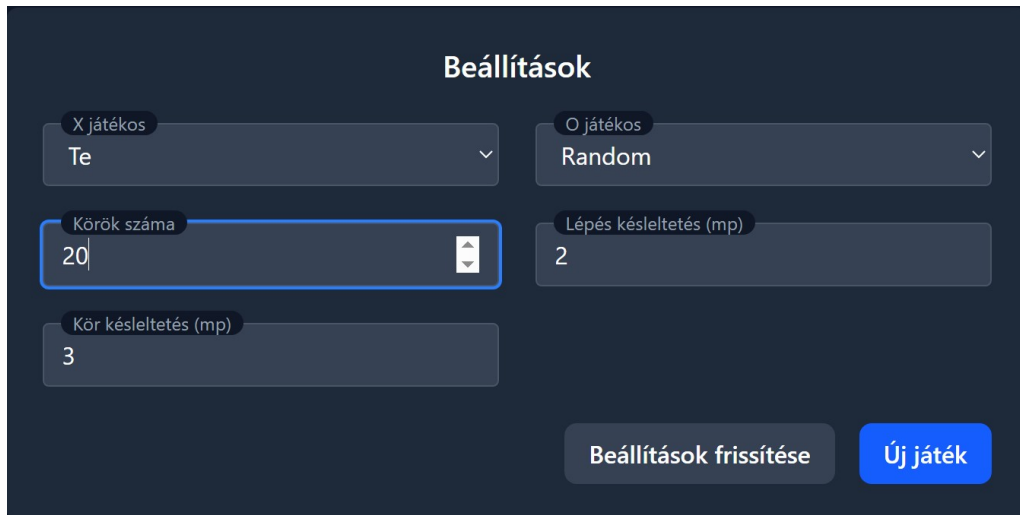
Beállítás	Gomb	Gomb, ha a játékosok is módosulnak
<i>Játékosok</i>	Új játék	Új játék
<i>Körök száma</i> ²	Beállítások frissítése	Új játék
<i>Lépések közötti szünet</i>	Beállítások frissítése	Új játék
<i>Körök közötti szünet</i>	Beállítások frissítése	Új játék

2.4. táblázat. Játékbeállítások módosítása játék közben



2.13. ábra. „Új játék” gomb

²A körök száma csak az aktuális kör számánál nagyobb lehet.



2.14. ábra. „Beállítások frissítése” gomb

Játékállás

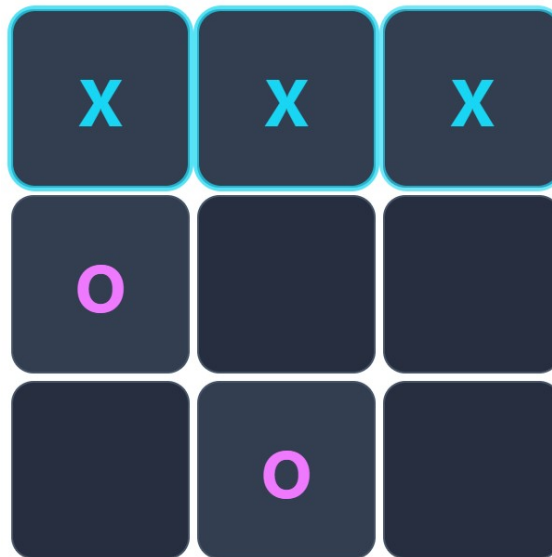
A játékálláspanel mutatja a játék aktuális állását, tehát a lejátszott és az összes kör számát, illetve a lejátszott körök eredményét. **Új játék** vagy **Játék létrehozása** gombra kattintás után az aktuális kör száma 1, és az eredmények nullázódnak.



2.15. ábra. Játékálláspanel

Játéktábla

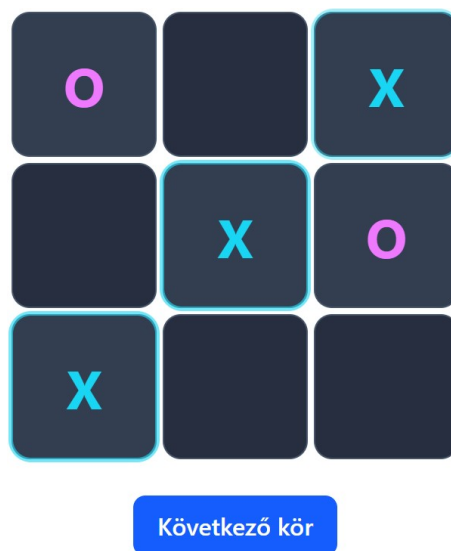
A játéktábla panel 9 mezőből áll. A mezőkre kattintva a felhasználó megteheti lépését, ha ő a soron következő játékos. Az *X* játékos színe kék, az *O* játékosé pink. A játéktábla üres mezői a soron következő játékos színével vannak kiemelve. A körök végén a nyertes mezők (amennyiben vannak) a győztes játékos színével vannak kiemelve.



2.16. ábra. Játéktábla

Új kör indítása

Két algoritmus közötti játék indítása esetén a körök automatikusan követik egymást, a megadott *Körök közötti szünettel*. Amennyiben a felhasználó is részt vesz a játékban, a játéktábla alatt megjelenő **Következő kör** gombra kattintva indítható el a következő kör.

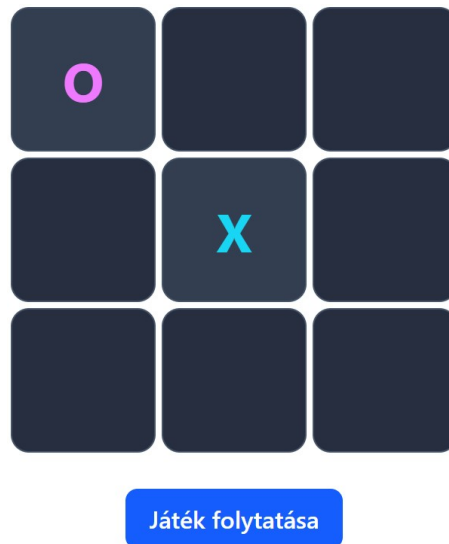


2.17. ábra. „Következő kör” gomb

Korábbi játék folytatása

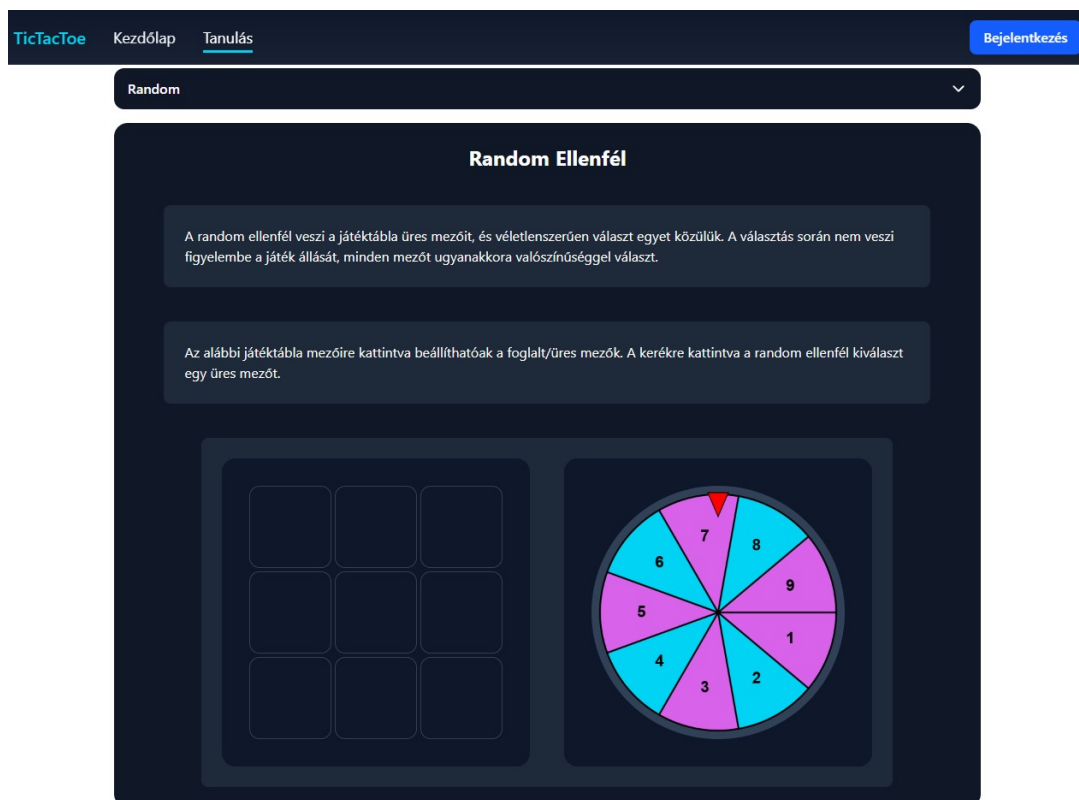
A felhasználó a *Kezdőlap* elhagyása után is folytathatja a játékot, amennyiben rövid időn belül visszatér a *Kezdőlapra*. Ilyenkor a játékbeállítások, a játékállás és

a játéktábla is visszaáll a legutóbbi állapotra. A játék folytatásához a **Játék folytatása** gombra kell kattintani. Az oldal bezárása vagy frissítése esetén a játék nem folytatható.



2.18. ábra. „Játék folytatása” gomb

2.4.2. Tanulás

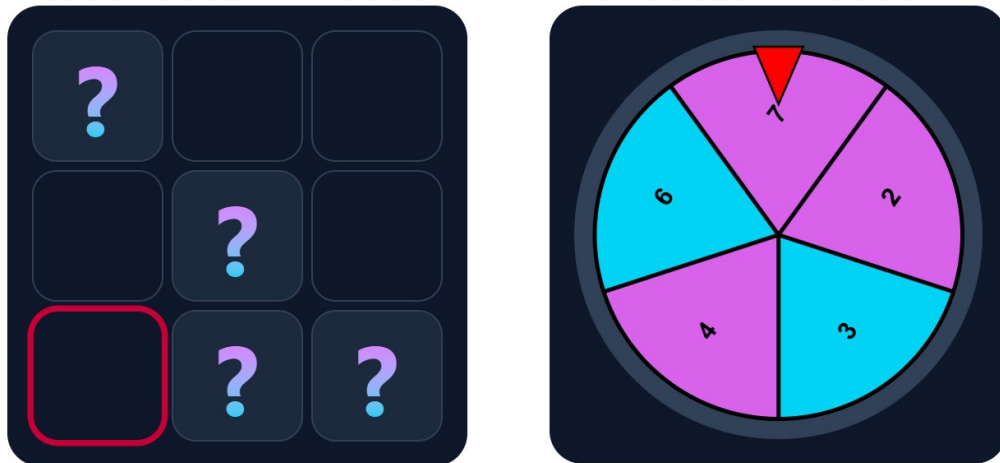


2.19. ábra. Tanulás lap, Random témakör

A *Tanulás* lapon a felhasználó megismerheti a Tic-Tac-Toe játékot játszó algoritmusokat. A készítéshez felhasznált források: [2, MENACE.], [3, minimax.], [4, Neurális hálózat.], [5, genetikusan algoritmus.], [6, visszaterjesztés - példa.], [7, visszaterjesztés - matek.].

Random

A *Random* algoritmust bemutató oldalon a szöveges leírás mellett egy interaktív játéktábla és egy rulettkerék segíti a megértést. A játéktábla foglalt mezőit **kérdőjelek** elhelyezésével lehet meghatározni. A keréken azonos méretű körcikkeken a szabad mezők indexei (1-től indexelve) szerepelnek. Kattintáskor a kerék megpördül, szimulálva a véletlenszerű választást. A választott mező kiemelésre kerül a játéktáblán.



2.20. ábra. Random algoritmus animáció

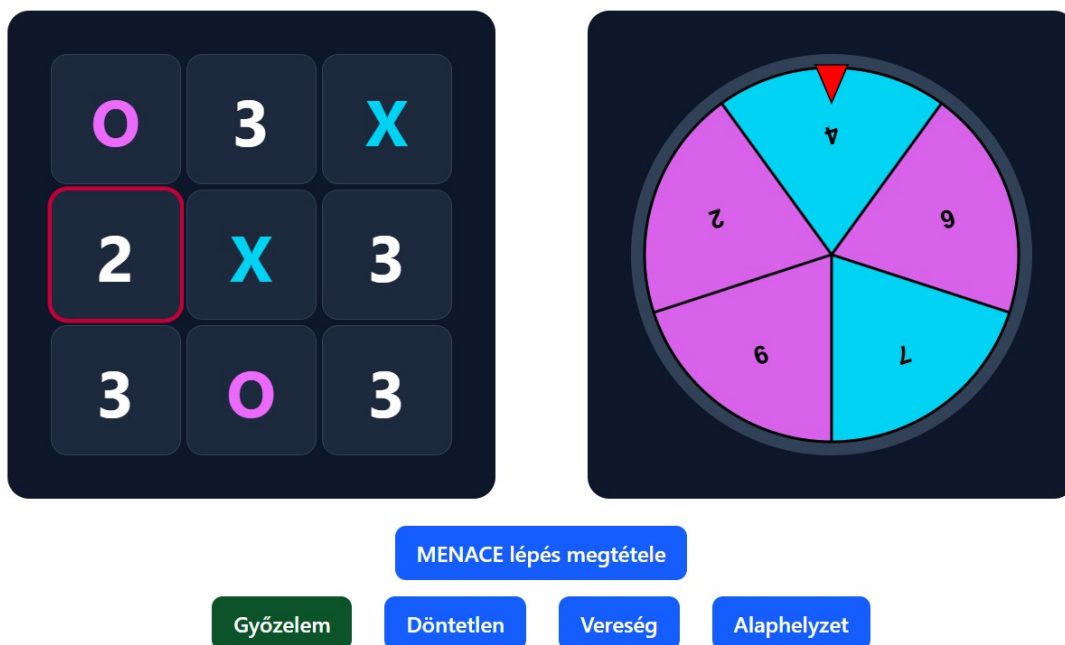
MENACE

A *MENACE*-t bemutató oldalon a szöveges leírás mellett egy a **tanulási folyamatot** bemutató interaktív animáció, és egy **játékszimuláció** található.

A **tanulási folyamatot** bemutató animáció azt mutatja be, hogyan játszik le egy kört a MENACE algoritmus, majd hogyan tanul annak eredményéből. A három lehetséges eredményhez vezető játékhöz, illetve a rákövetkező tanuláshoz tartozó bemutatót, a megfelelő (*Győzelem*, *Döntetlen*, *Vereség*) gombra kattintva lehet elindítani. Az animációs panelt az *Alaphelyzet* gomb a kezdőállapothoz állítja. Az animáció lépésekből áll, amelyeket a megfelelő gombra kattintva lehet végrehajtani:

Gomb	Leírás
<i>Gyufásdoboz kiválasztása</i>	A játéktábla üres mezőibe számok kerülnek. A mezőbe kerülő szám a gyufásdobozban lévő, az adott sorszámmal jelölt gyöngyök számát mutatja.
<i>Gyöngy kihúzása</i>	A számok arányában szeletekre osztott rulettkerék megpördül, és kiválaszt egy mezőt. A kiválasztott mező piros körvonalat kap. A kiválasztott mező száma 1-el csökken.
<i>MENACE lépés megtétele</i>	A megjelölt mezőre a MENACE szimbóluma, "X" kerül.
<i>Ellenfél lépése</i>	Az egyik üres mezőre az "O" szimbólum kerül.
<i>Tanítás</i>	Elindítja a tanulási lépéseket.
<i>MENACE lépés törlése</i>	Törli a MENACE ("X") legutóbbi lépését.
<i>Jutalom gyöngyök</i>	A MENACE által legutóbb választott mezőt kiemeli és a számához hozzáadja a jutalom gyöngyök számát.
<i>Gyufásdoboz visszahelyezése</i>	A számokat tartalmazó mezőket üres mezőkké alakítja.
<i>Ellenfél lépés törlése</i>	Törli az ellenfél ("O") legutóbbi lépését.

2.5. táblázat. MENACE animáció lépései



2.21. ábra. MENACE tanulási folyamat animációs panel

A **játékszimuláció** -panelen lehetőség van játékállásokat beállítani. Amennyiben az állás folyamatban lévő játékot ábrázol és lehetséges³, a tábla melletti

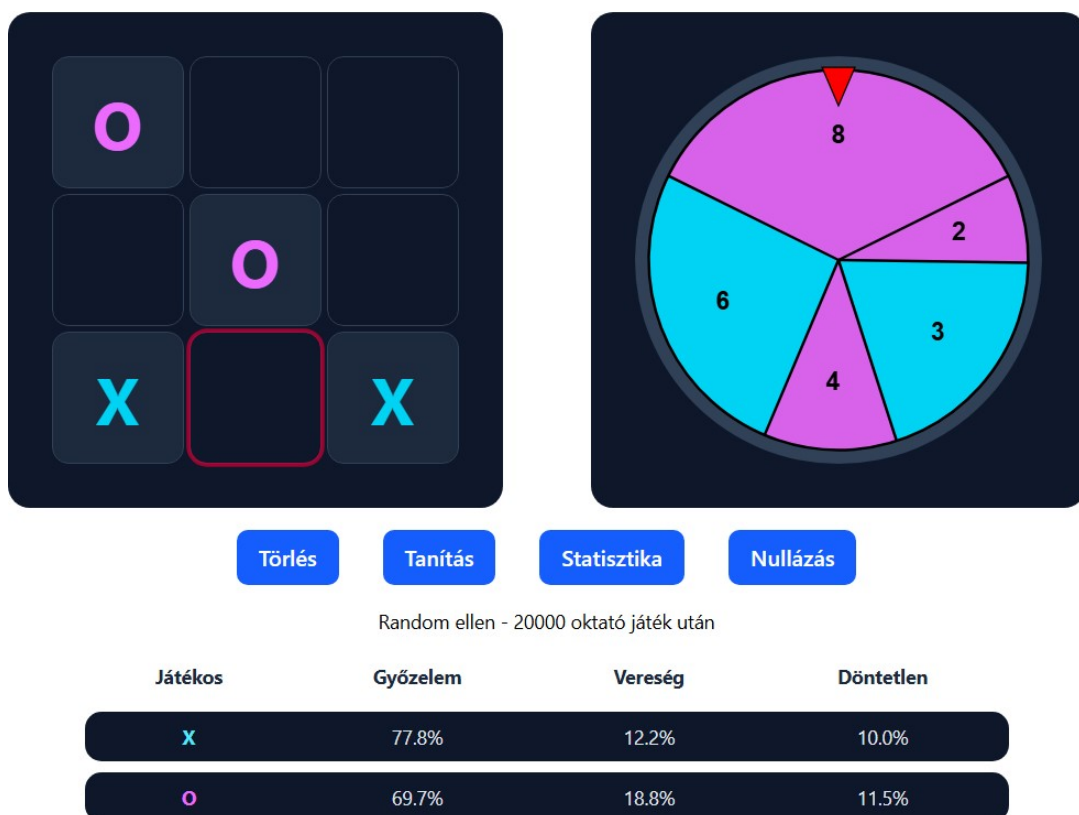
³Egy állás akkor lehetséges, ha az X-ek száma nem kisebb az O-k számánál.

rulettkerék szeletein megjelennek az üres mezők sorszámai. A szeletek a MENACE által megtanult gyöngyszámokkal arányosak.

Az elérhető műveletek:

Művelet	Hatás
<i>Tábla mezőjének kattintása</i>	Az adott mező tartalma változik. Üresből "X", "X"-ből "O", "O"-ból üres lesz.
<i>Rulettkerék kattintása</i>	A kerék megpördül, a kiválasztott mező kiemelésre kerül.
<i>Törlés gomb</i>	A játéktábla minden mezője üres lesz.
<i>Tanítás gomb</i>	A MENACE 20 000 kört játszik egy random ellenfél ellen.
<i>Statisztika gomb</i>	A MENACE 100-100 kört játszik X és O játékosként is random ellenfél ellen. A játékok eredménye megjelenik a panel alján.
<i>Nullázás gomb</i>	A játéktábla minden mezője üres lesz. A MENACE tudása nullázódik.

2.6. táblázat. MENACE szimuláció műveletei



2.22. ábra. MENACE játékszimuláció-panel

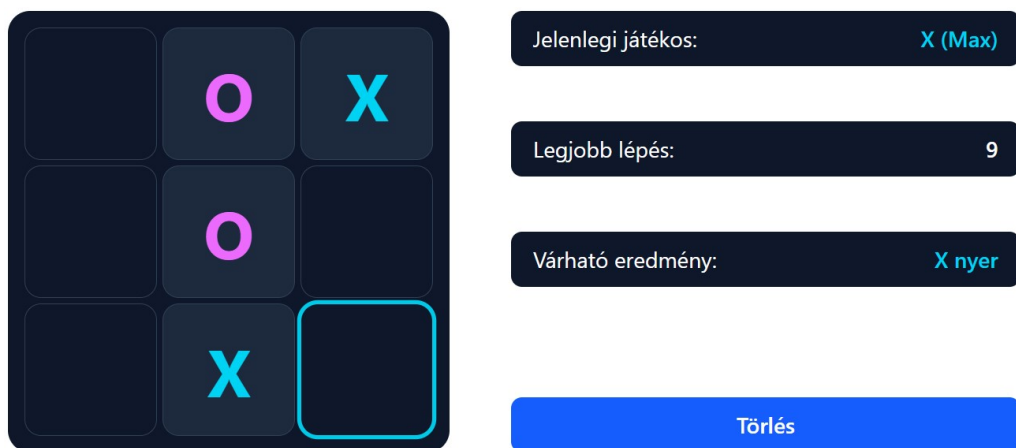
Minimax

A *Minimax* algoritmust bemutató oldalon a szöveges leírás mellett egy interaktív játéktábla és a táblához tartozó játékfa található.

A játéktáblán egy előre meghatározott játékállás alakítható tovább. A tábla mezőjére kattintva a **Jelenlegi játékos** szimbóluma kerül a mezőre. A **Törlés** gombbal a tábla visszaáll a kezdőállapotra.

A táblától jobbra található elemek és azok tartalma a következő:

- **Jelenlegi játékos:** Az aktuális játékos $X(Max)$ vagy $O(Min)$.
- **Legjobb lépés:** Az aktuális álláshoz tartozó legjobb lépés. Ez a lépés a táblán az aktuális játékos színével ki van emelve.
- **Várható eredmény / Eredmény:** A játék várható kimenetele, ha mindkét játékos tökéletesen játszik. / A játék végeredménye.



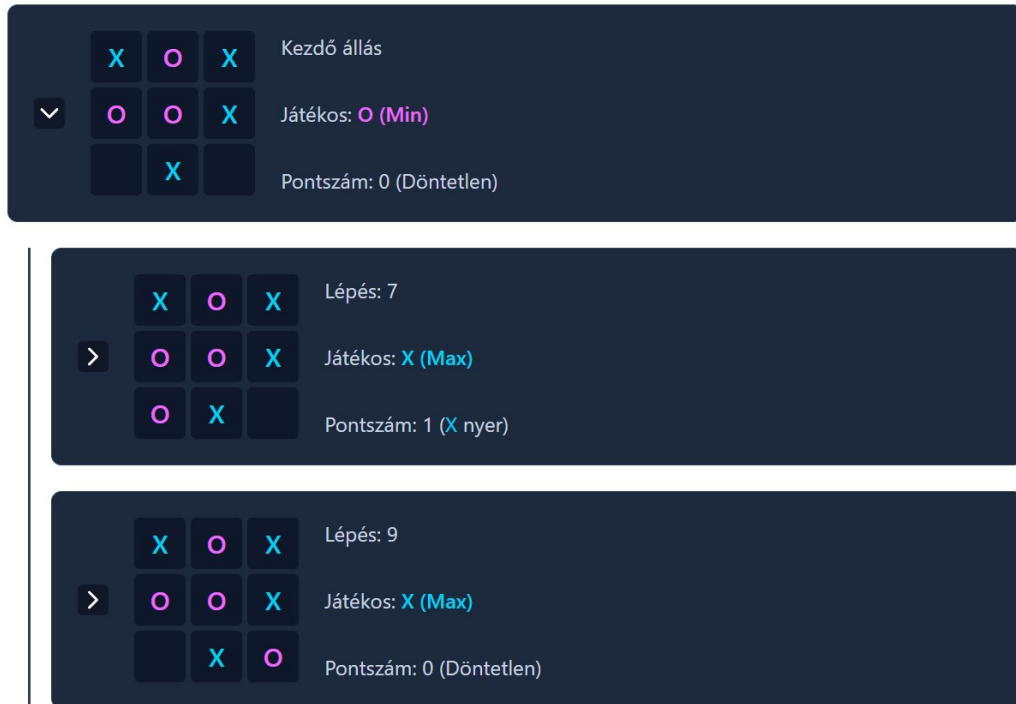
2.23. ábra. Minimax játéktábla, kezdőállapotban

A játékfa a játéktábla alatt található. A fa csúcsai a lehetséges játékállásokat, élei pedig a lépéseket reprezentálják. A fa gyökere a játéktábla aktuális állapotát ábrázolja.

Egy csúcsot reprezentáló elem a következőket tartalmazza:

- **Nyitó / záró nyíl:** A csúcs gyerekeinek mutatása / elrejtése.
- **Játéktábla:** A csúcsnak megfelelő játéktábla.
- **Lépés:** Az előző csúcsból a jelenlegi csúcsba vezető lépés. Gyökér esetén nincs ilyen.

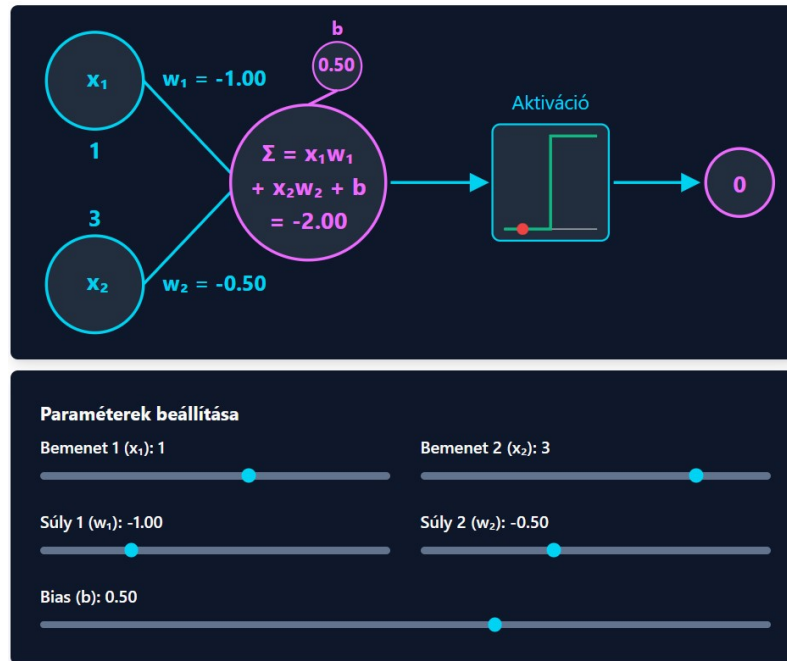
- **Játékos:** Az aktuális játékos, aki a csúcs játéktábláján lépni fog.
- **Pontszám:** A játék kimenetele (tökéletes játékot feltételezve) számszerűsítve: 1 az "X" győzelem, 0 a döntetlen, -1 az "O" győzelem.



2.24. ábra. Minimax játékfa gyökérrel és a gyökér két gyerekével

Neuronháló

A *Neuronháló*-t bemutató lapon a neuron és a neuronháló felépítése kerül ismertetésre. A szöveges leírásokat egy interaktív neuronvizualizáció és egy interaktív neuronhálózat-vizualizáció egészíti ki. A neuronvizualizáció a neuron felépítését mutatja be. Az ábra alatti csúszkák segítségével lehetőség van a bemeneti értékek, súlyok és torzítás módosítására.



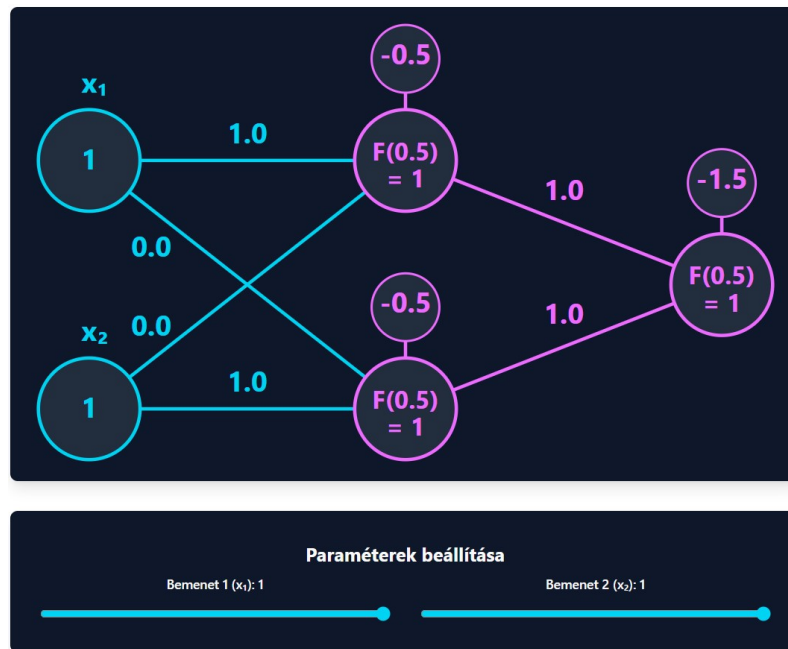
2.25. ábra. Neuronvizualizáció és paraméterek beállításai

A *Neuronháló* fejezetben három előre definiált neuronháló kerül bemutatásra, melyek logikai kapukat valósítanak meg (ÉS, VAGY, KIZÁRÓ VAGY). A felhasználó a gombok segítségével válthat a különböző hálók között.



2.26. ábra. Neuronháló kiválasztó gombok

A hálók bemeneti értéke lehet 0 vagy 1, a kimeneti érték pedig szintén 0 vagy 1 lesz (0 a hamis, 1 az igaz érték).

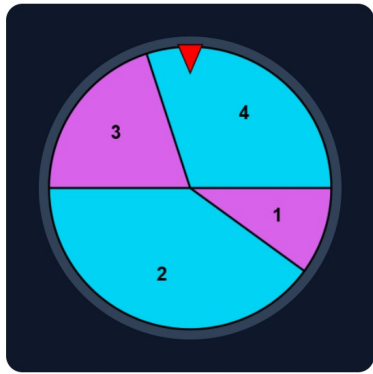


2.27. ábra. Neuronháló-vizualizáció és paraméterek beállításai

Genetikus algoritmus

A *Genetikus algoritmus*-t bemutató lapon a genetikus algoritmus működése kerül ismertetésre. A lap elején az algoritmus működésének megértéséhez szükséges alapok találhatók. Egy táblázatban a fontos fogalmak kerülnek összefoglalásra, ezt követően egy neuron és egy neuronháló lehetséges kódolása látható, illetve a fitness függvény definíciója és ábrás szemléltetése is szerepel. A genetikus algoritmus operátorai a felsorolt ismeretekre építve interaktív ábrákon kerülnek bemutatásra.

A kiválasztás operátorok közül a Kerékfordatásos kiválasztás és a Versenykiválasztás kerül bemutatásra. A Kerékfordatásos kiválasztás esetén egy az egyedek fitness értékeivel arányosan felosztott kerék pördül meg. A versenykiválasztásánál egy adott méretű, véletlenszerűen kiválasztott csoportból kerül kiválasztásra a legjobb fitness értékű egyed.



(a) Kerékforgatásos



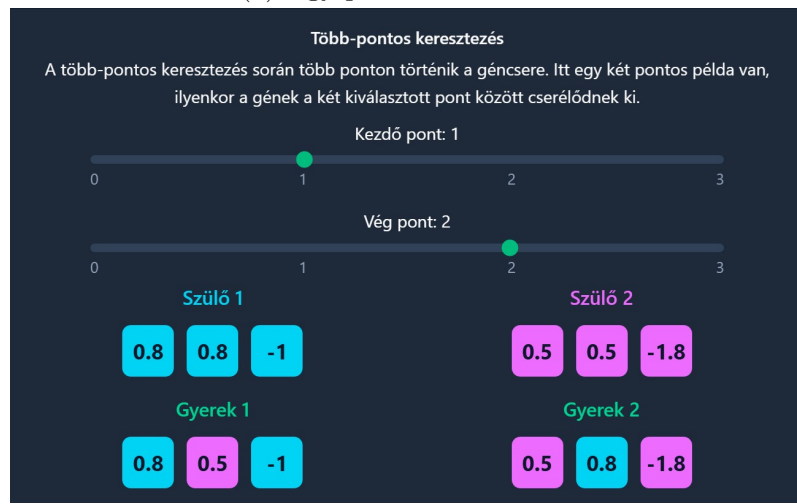
(b) Verseny

2.28. ábra. Genetikus algoritmus, kiválasztás operátorok

A keresztezés operátorok közül az Egy-pontos keresztezés, Két-pontos keresztezés és Egységes keresztezés kerül bemutatásra. Az Egy-pontos keresztezésnél a csúszkával megadott pontnál történik a szülő gének felcserélése. A Két-pontos keresztezésnél a két csúszkával megadott pontok között történik a szülő gének felcserélése. Az Egységes keresztezésnél egyenlő eséllyel kerül felcserélésre minden egyes gén a szülők között.



(a) Egy-pontos keresztezés



(b) Két-pontos keresztezés



(c) Egységes keresztezés

2.29. ábra. Genetikus algoritmus, keresztezés operátorok

Az utolsó bemutatott operátor a Mutáció. Két csúszka segítségével lehet megadni a mutációs rátát (milyen eséllyel változzon) és a mutációs erősséget (mekkora lehet a változás maximális mértéke).

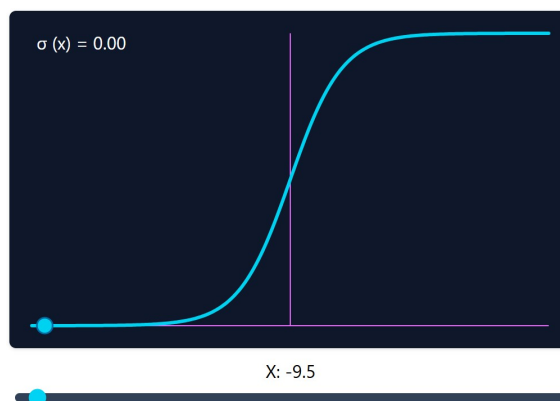


2.30. ábra. Genetikus algoritmus, mutáció operátor

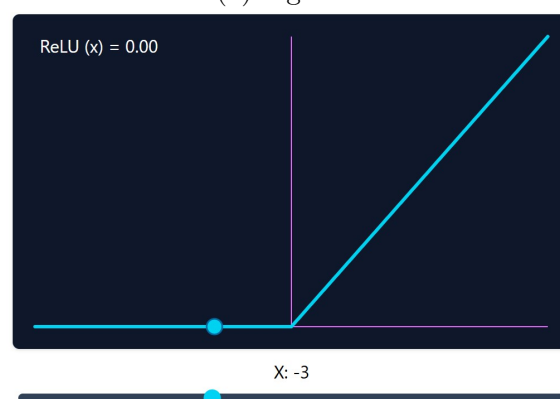
Visszaterjesztés

A *Visszaterjesztés*-t bemutató lapon a visszaterjesztési tanulás működése kerül ismertetésre. A lap fejezetei:

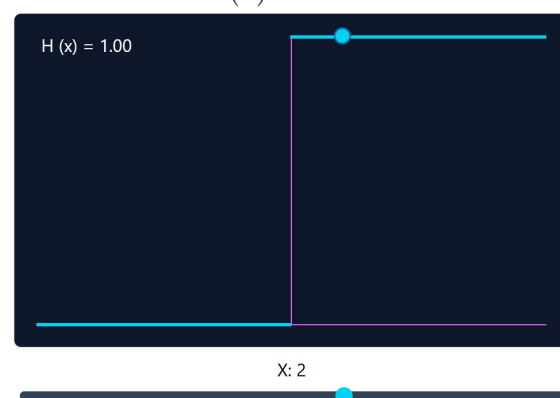
- **Bemenetből kimenet:** A bemeneti értékek előrecsatolásának menete, súlyozott összeg és aktivációs érték képleteivel.
- **Aktivációs függvények:** Népszerű aktivációs függvények bemutatása képletekkel és grafikonjaikkal.
- **Veszteség:** A veszteség fogalma, két gyakran használt veszteségfüggvény bemutatása képletekkel és ábrákkal.
- **Visszaterjesztés lépései:** A visszaterjesztési tanítás folyamata lépésről lépésre, a hozzá tartozó matematikai képletekkel és ábrákkal.



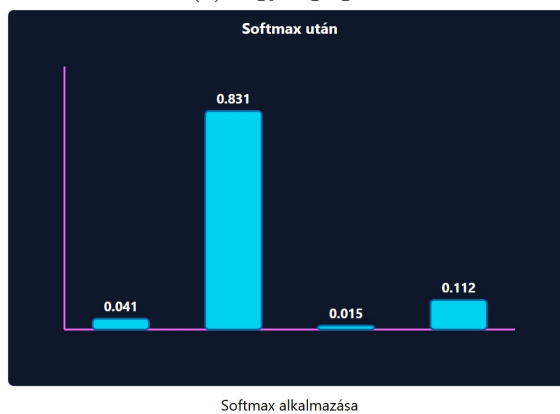
(a) Sigmoid



(b) ReLU

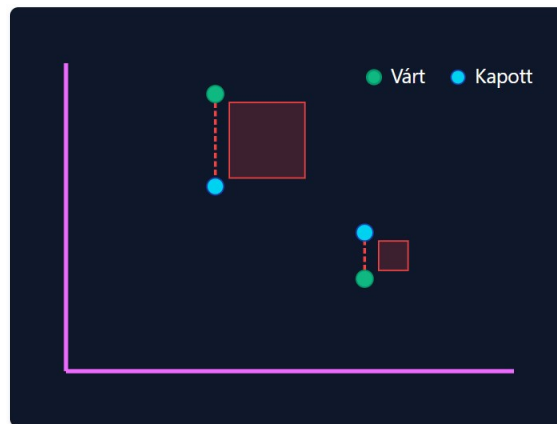


(c) Egységugrás

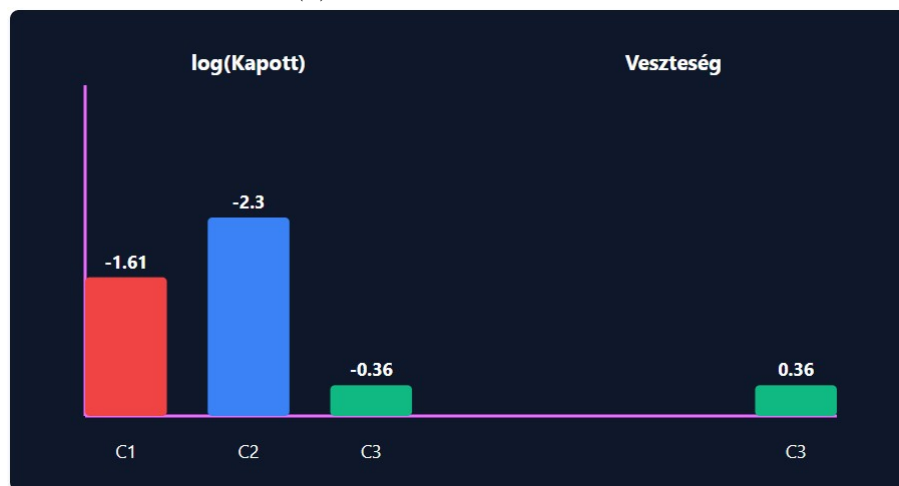


(d) Softmax

2.31. ábra. Aktivációs függvények



(a) Átlagos négyzetes hiba



Kapott eloszlás



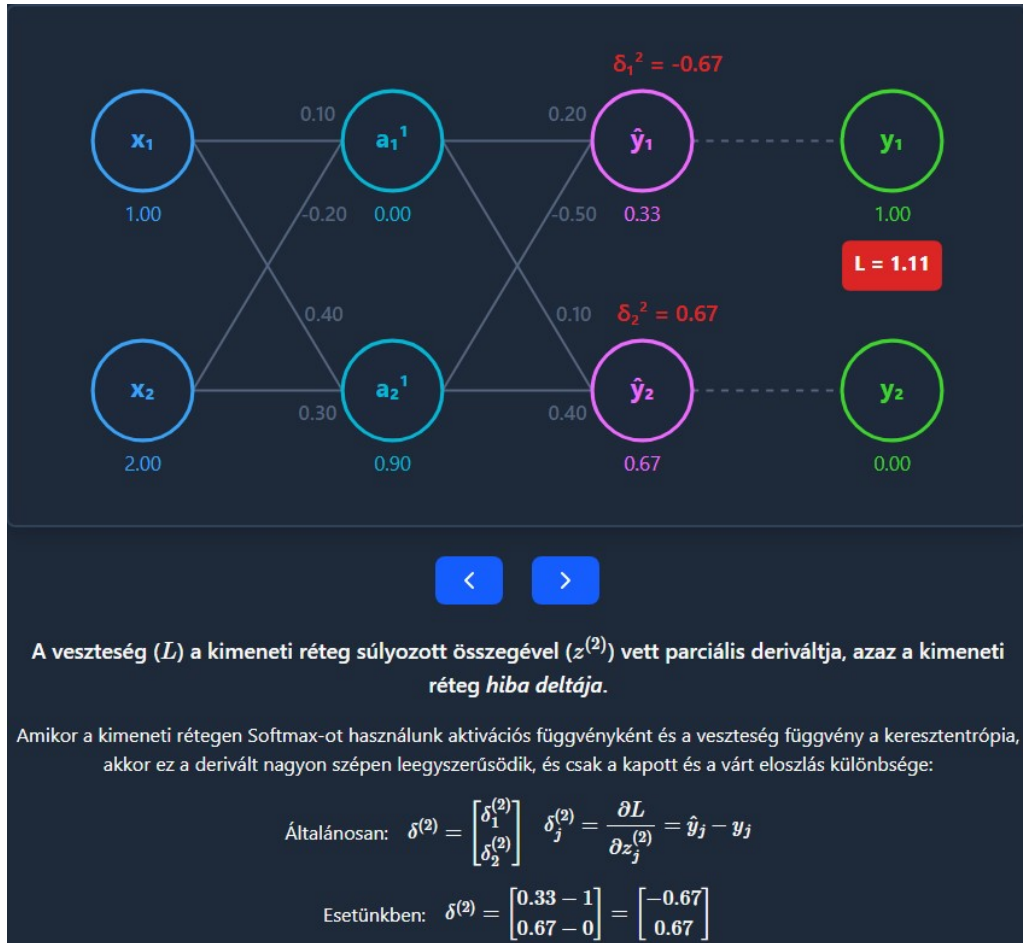
Határozd meg a várt eloszlást!



(b) Keresztentropia

2.32. ábra. Veszteségfüggvények

A *Visszaterjesztés lépései* fejezetben nyilakkal lehet haladni a visszaterjesztés tanítás egyes lépései között. A nyilak felett egy neuronháló ábra található, amely mindig kiemeli a lépéshez tartozó adatokat. A nyilak alatt pedig a lépéshez tartozó képletek olvashatók.



2.33. ábra. Visszaterjesztés egy lépése

2.4.3. Regisztráció

A *Regisztráció* lapon a felhasználó létrehozhat egy fiókot az alkalmazásban. A regisztrációhoz a következő adatok megadása szükséges:

- Felhasználónév
- Jelszó
- Jelszó megerősítése

Megszorítások:

Mező	Megszorítás
<i>Felhasználónév</i>	Minimum 3 és maximum 32 karakter, csak betűket és számokat tartalmaz, valamint egyedi.
<i>Jelszó</i>	Minimum 8 karakter, tartalmaz nagybetűt, kisbetűt, számot és 72 bájtól nem hosszabb.
<i>Jelszó megerősítése</i>	Megegyezik a jelszó mezővel.

2.7. táblázat. Regisztrációs mezők megszorításai

Sikeres regisztráció után a felhasználó automatikusan bejelentkezik az alkalmazásba, és a *Kezdőlapra* kerül átirányításra.

The image shows a dark-themed registration form titled "Regisztráció". It contains three input fields: "Név" (Name) with placeholder text "Add meg a felhasználóneved", "Jelszó" (Password) with placeholder text "Add meg a jelszavad" and a visibility toggle icon, and "Jelszó megerősítése" (Password confirmation) with placeholder text "Jelszó ismét" and a visibility toggle icon. A blue "Regisztráció" button is located at the bottom of the form.

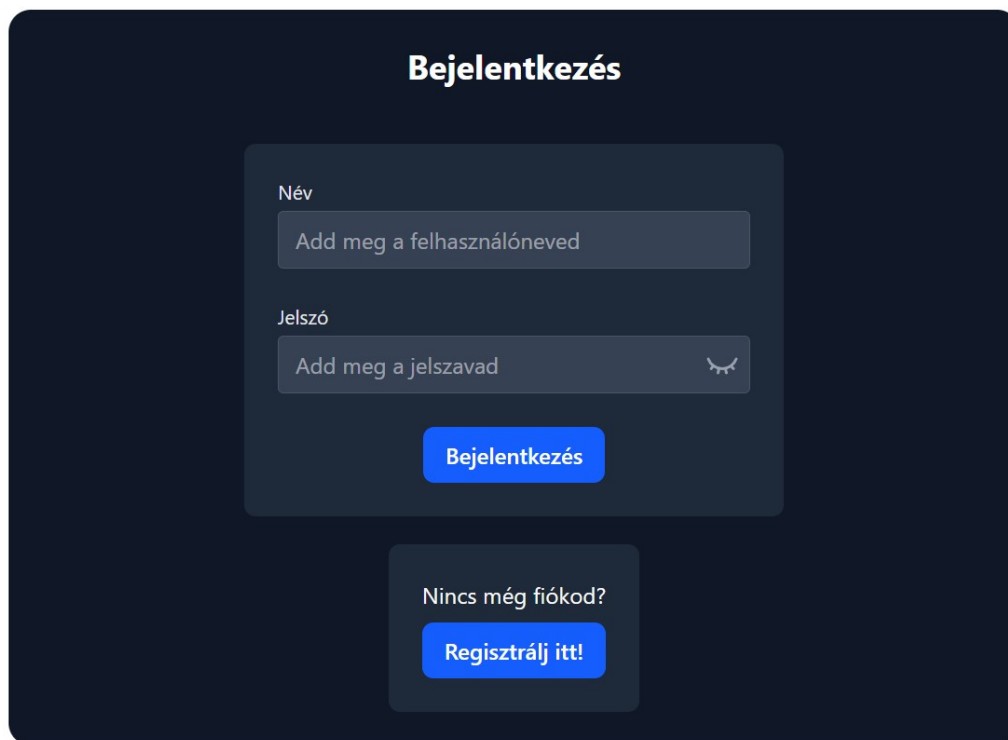
2.34. ábra. Regisztrációs lap

2.4.4. Bejelentkezés

A *bejelentkezési* lapon a felhasználó bejelentkezhets az alkalmazásba. A bejelentkezéshez a következő adatok megadása szükséges:

- Felhasználónév
- Jelszó

Amennyiben a megadott adatokhoz tartozik felhasználói fiók, a bejelentkezés sikeres, és a felhasználó a *Kezdőlapra* kerül átirányításra.



The image shows a login form titled "Bejelentkezés" (Login) on a dark blue background. The form is centered and contains two input fields: "Név" (Name) with the placeholder text "Add meg a felhasználóneved" and "Jelszó" (Password) with the placeholder text "Add meg a jelszavad" and a toggle icon. Below the input fields is a blue button labeled "Bejelentkezés". Below the button is a link "Nincs még fiókod?" (Don't have an account yet?) with a blue button labeled "Regisztrálj itt!" (Register here!).

2.35. ábra. Bejelentkezési lap

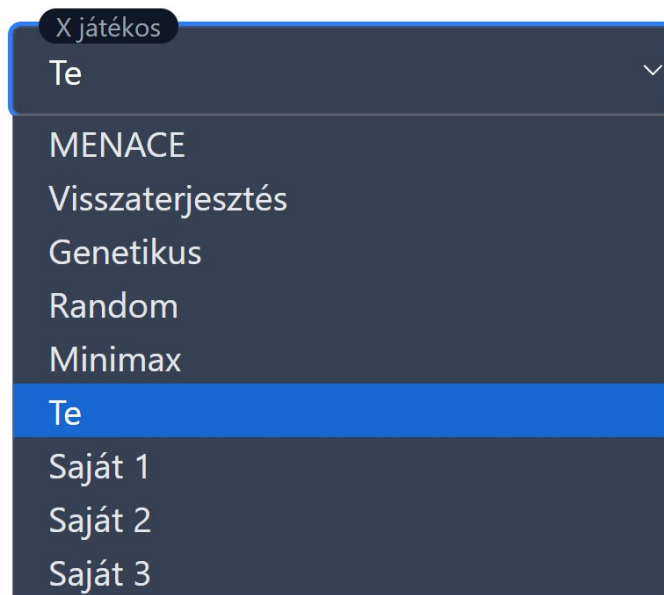
Bejelentkezés után a felhasználó elhagyja a vendég módot, és új funkciók, valamint egy új oldal válik elérhetővé.

2.4.5. Kezdőlap - bejelentkezett felhasználónak

A *Kezdőlap* bejelentkezett felhasználóként két szempontból változik.

Játékbeállítások bővítése

A játékbeállítások panelen a felhasználó most már saját neurális hálózatait is választhatja játékosként.



2.36. ábra. Játékosválasztás bejelentkezett felhasználó számára

Korábbi játék folytatása

Bejelentkezett felhasználóként a játék folytatására akkor is van lehetőség, ha az alkalmazást bezárták vagy frissítették. Az időkorlát ebben az esetben is érvényes.

2.4.6. Barkácsolás - csak bejelentkezett felhasználónak

2.37. ábra. Barkácsolás lap

Bejelentkezett felhasználóként a *Barkácsolás* lap is elérhető. Itt a felhasználó saját neurális hálózatokat hozhat létre, módosíthat, illetve visszaterjesztéssel taníthat.

Neurális hálózat létrehozása

A lap tetején található űrlap segítségével új neurális hálózatot lehet létrehozni.

2.38. ábra. Neurális hálózat létrehozása űrlap

Megszorítások:

Mező	Megszorítás
<i>Hálózat neve</i>	1 és 15 karakter hosszú.
<i>Rétegek</i>	Számok vesszővel elválasztva, első szám 18, utolsó 9.

2.8. táblázat. Neurális hálózat létrehozásának megszorításai

Neurális hálózat táblázat

A létrehozott neurális hálózatok egy táblázatban jelennek meg. A táblázatban a hálózatra vonatkozó információk és az elvégezhető műveletek találhatók.

NÉV	RÉTEGEK	STATISZTIKÁK	SZERKESZTÉS	TANÍTÁS	TÖRLÉS
Gyenge	18, 9	Iterációk: 10, Pontosság: 59.33%, Veszteség: 1.28	Szerkesztés	Tanítás	Törlés
Erős	18, 9	Iterációk: 1000, Pontosság: 70.65%, Veszteség: 0.75	Szerkesztés	Tanítás	Törlés
Saját 1	18, 9	Nem tanított	Szerkesztés	Tanítás	Törlés

2.39. ábra. Neurális hálózat táblázat

Neurális hálózat szerkesztése⁴

A 2.39 ábra táblázatában található **Szerkesztés** gombra kattintva a hálózat megnyílik szerkesztésre a lap alján. A gomb újbóli kattintásra bezárja a szerkesztőpanelt és elveti a nem mentett változtatásokat. A változtatásokat a szerkesztőpanel **Mentés** gombjára kattintva lehet elmenteni. A módosítható értékek:

⁴A funkció csak 768 px széles ablakmérettől érhető el

Érték	Módosítás módja	Megszorítás
<i>Hálózat néve</i>	Szerkesztőpanel <i>Név</i> mezőjének módosításával.	1 és 15 karakter hosszú.
<i>Rétegek</i>	Hozzáadás: a réteg indexének megadásával, az <i>Új réteg</i> gombbal.	A megadott index a bemeneti és kimeneti réteg közé kell eszen.
	Eltávolítás: a réteg összes neuronjának törlésével.	Csak rejtett réteg esetén lehetséges.
<i>Neuron</i>	Hozzáadás: a réteg indexének megadásával, a <i>Neuron hozzáadás réteghez</i> gombbal.	A megadott index a bemeneti és kimeneti réteg közé kell eszen.
	Eltávolítás: a neuron kijelölésével, a <i>Kiválasztott neuron törlése</i> gombbal.	Csak rejtett réteghez tartozó neuron esetén lehetséges.
<i>Torzítás</i>	A Megjelenítőpanelen a neuron kiválasztásával, a sarokban megjelenő <i>Torzítás</i> mező értékének módosításával.	-10 000 és 10 000 közötti érték adható meg.
<i>Súly</i>	A Megjelenítőpanelen a neuron, majd a súly kiválasztásával, a sarokban megjelenő <i>Súly</i> mező értékének módosításával.	-10 000 és 10 000 közötti érték adható meg.

2.9. táblázat. Neurális hálózat szerkesztése és megszorításai

Szerkesztő

Az első, és az utolsó réteg neuronzáma nem módosítható. A bemeneti réteg a 0-ás indexű.

Név

Gyenge

Mentés

Kiválasztott neuron törlése

Neuron hozzáadás réteghez

Új réteg

Törlés

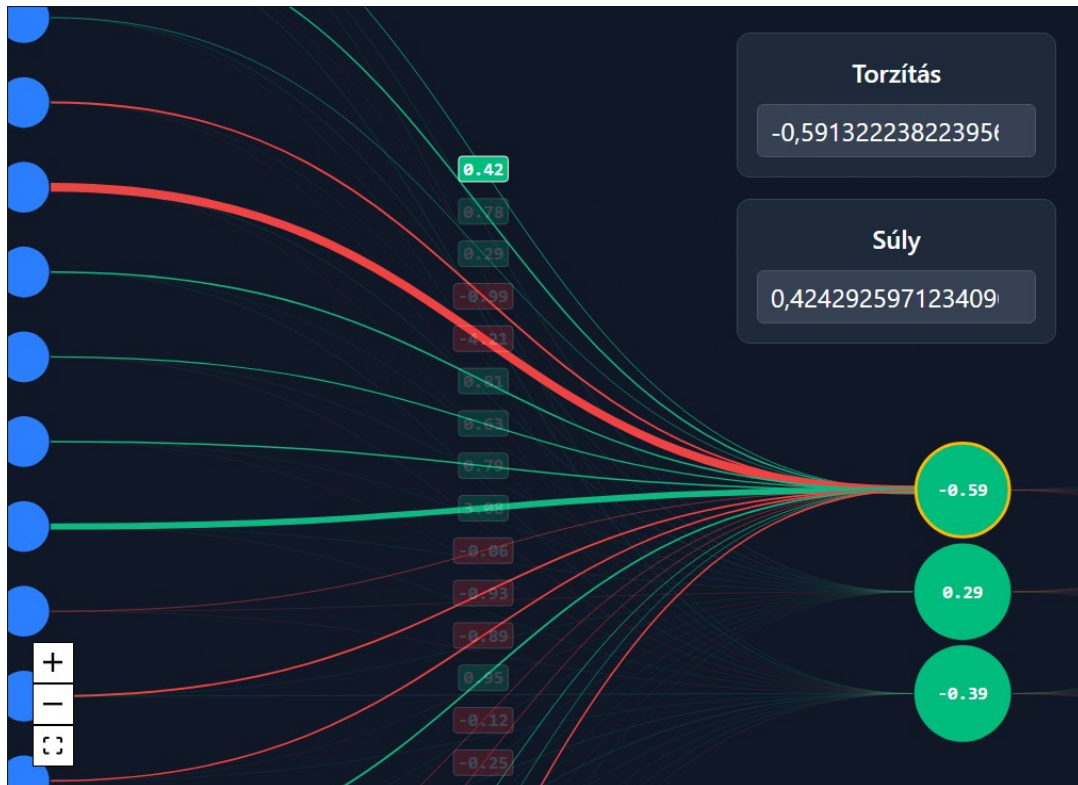
1

Hozzáadás

1

Hozzáadás

2.40. ábra. Neurális hálózat szerkesztő főmenü



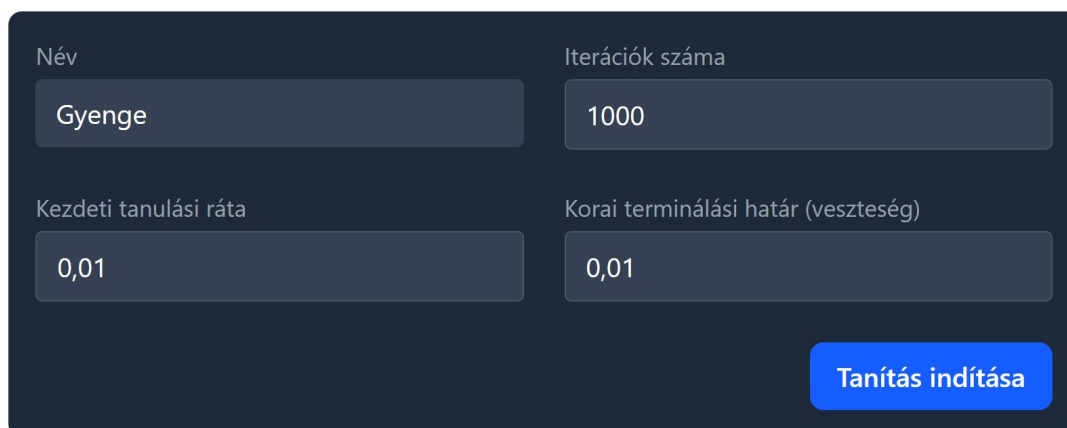
2.41. ábra. Neurális hálózat megjelenítőpanel

Neurális hálózat tanítása

A 2.39 ábra táblázatában található **Tanítás** gombra kattintva megnyílik a tanítópanel. A gomb újbóli kattintásra bezárja a tanítópanel. A tanítópanelen a következő beállítások adhatók meg:

Mező	Megszorítás
<i>Iterációk száma</i>	10 és 10 000 közötti szám.
<i>Kezdeti tanulási ráta</i>	0,000001 és 30 közötti szám.
<i>Korai terminálási határ</i>	0 és 25 közötti szám.
<i>Név</i>	Nem módosítható.

2.10. táblázat. Neurális hálózat tanítópanel értékei és megszorításai

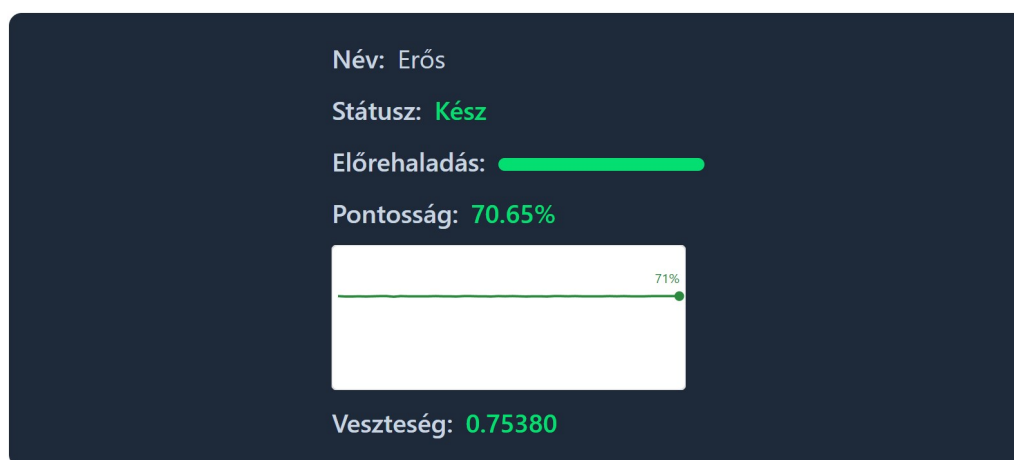


2.42. ábra. Neurális hálózat tanítópanel

A **Tanítás indítása** gombra kattintva elindul a tanítás a megadott beállításokkal. A folyamatban lévő tanítás állapotáról egy panelen kap visszajelzést a felhasználó.

A panelen a következő információk jelennek meg:

- Név - a tanított hálózat neve
- Státusz - a tanítás aktuális státusza (Várakozás, Folyamatban, Sikeres, Hiba)
- Előrehaladás - a tanítás előrehaladása százalékban
- Pontosság - az aktuális érték és a korábbi értékek grafikonja
- Veszteség - az aktuális érték



2.43. ábra. Neurális hálózat tanításának státusza

Neurális hálózat törlése

A 2.39 ábra táblázatában található **Törlés** gombra kattintva a hálózat törlésre kerül.

2.5. Bemenetek és hibaüzenetek

2.5.1. Bemenetek

Az alkalmazás minden beviteli mezőjére vonatkoznak megszorítások. A beviteli mezők aktiválásakor megjelenik egy tipp, amely a vonatkozó megszorításokat tartalmazza. A Lapok és funkciók fejezetben az egyes lapoknál már szerepeltek a mezők és a hozzájuk tartozó megszorítások. Az alábbi táblázat összefoglalja ezeket:

Mezőnév	Mezőtípus	Megszorítás
Kezdőlap		
<i>Játékosok</i>	Legördülő lista	Nem a felhasználó (<i>Te</i> opció) van választva mindkét játékosnak.
<i>Körök száma</i>	Szám	1 és 100 között.
<i>Lépések közötti szünet</i>	Szám	0,02 és 10 között.
<i>Körök közötti szünet</i>	Szám	0,02 és 10 között.
Regisztráció		
<i>Felhasználónév</i>	Szöveg	Minimum 3 és maximum 32 karakter, csak betűket és számokat tartalmaz, valamint egyedi.
<i>Jelszó</i>	Szöveg	Minimum 8 karakter, tartalmaz nagybetűt, kisbetűt, számot és 72 bájtól nem hosszabb.
<i>Jelszó megerősítése</i>	Szöveg	Megegyezik a jelszó mezővel.
Bejelentkezés		
<i>Felhasználónév</i>	Szöveg	Egy felhasználói fiókhoz tartozó név.
<i>Jelszó</i>	Szöveg	A felhasználónévhez tartozó jelszó.
Barkácsolás - csak bejelentkezett felhasználónak		
<i>Hálózat neve</i>	Szöveg	1 és 15 karakter közötti hosszúságú.
<i>Rétegek</i>	Szöveg	Számok vesszővel elválasztva, első szám 18, utolsó 9, összegük maximum 120.
<i>Új réteg</i>	Szám	A bemeneti és kimeneti réteg sorszáma között.
<i>Neuron hozzáadása</i>	Szám	A bemeneti és kimeneti réteg sorszáma között.
<i>Torzítás</i>	Szám	-10 000 és 10 000 között.
<i>Súly</i>	Szám	-10 000 és 10 000 között.
<i>Iterációk száma</i>	Szám	10 és 10 000 között.
<i>Kezdeti tanulási ráta</i>	Szám	0,000001 és 30 között.
<i>Korai terminálási határ</i>	Szám	0 és 25 között.

2.11. táblázat. Bemeneti mezők megszorításai

2.5.2. Hibaüzenetek

A hibákról vagy nem megfelelő adatokról az alkalmazás visszajelzést ad a felhasználónak. Ez egy fentről beúszó értesítés formájában jelenik meg.



Adj meg egy érvényes réteg konfigurációt. (pl. 18,9)

2.44. ábra. Az értesítés megjelenése

Az alkalmazás lehetséges értesítései a következők:

- Érvénytelen felhasználónév.
- A felhasználónév foglalt.
- A jelszavak nem egyeznek meg.
- A jelszónak tartalmaznia kell legalább egy nagybetűt.
- A jelszónak tartalmaznia kell legalább egy kisbetűt.
- A jelszónak tartalmaznia kell legalább egy számot.
- A jelszónak legalább 8 karakter hosszúnak kell lennie.
- A jelszó túl hosszú.
- Érvénytelen felhasználónév vagy jelszó.
- A név 1-15 karakter hosszú lehet.
- Adjon meg egy érvényes rétegkonfigurációt. (pl. 18,9)
- Érvénytelen rétegek: legalább két elemet tartalmazó listának kell lennie, a bemeneti rétegnek 18 neuronnal, a kimeneti rétegnek pedig 9 neuronnal kell rendelkeznie.
- Érvénytelen formátum: a neuronok teljes száma nem haladhatja meg a 120-t.
- Elérted a neuronháló maximális számát. Kérlek, töröld egy meglévő neuronhálót, mielőtt újat hozol létre.
- Csak a bemeneti és kimeneti réteg közé lehet réteget hozzáadni.

- Csak rejtett réteghez lehet neuront hozzáadni.
- Nincs ilyen réteg / neuronháló.
- Neuronháló nem található.
- A neuronháló nem található vagy nem a sajátod.
- A tanítás nem található.
- Nem hozzád tartozó tanítás.
- A tanítás már befejeződött.
- Már fut egy tanításod.
- A szerver elfoglalt, próbáld újra később.
- Nem te következel!
- Hiba a játékosok betöltése közben.
- A munkamenet lejárt, kérlek jelentkezz be újra.
- Érvénytelen token.
- Hiányzó token.
- Hiányzó token nem vendég WebSocket-kapcsolatához.
- A játék nem található.
- Nem sikerült kapcsolódni a szerverhez.
- Nem sikerült friss tokent szerezni.
- Túl sok kérés érkezett. Próbáld újra később.
- Túl sok kérés érkezett. Maximum x kérés engedélyezett y időegység időtartamon belül. Próbáld újra később.
- Szerver hiba.
- Az adatbázis elfoglalt, kérlek próbáld újra.
- Az erőforrás már létezik.

- Az adatbázis nem elérhető.
- *[Egyéb, nem várt hibaüzenet a szervertől]*

Megjegyzés. A felsorolt üzenetek egy része a bemeneti mezőkbe beépített megszorítások miatt nem fog előfordulni, hiszen a felhasználó nem tud érvénytelen adatot megadni.

3. fejezet

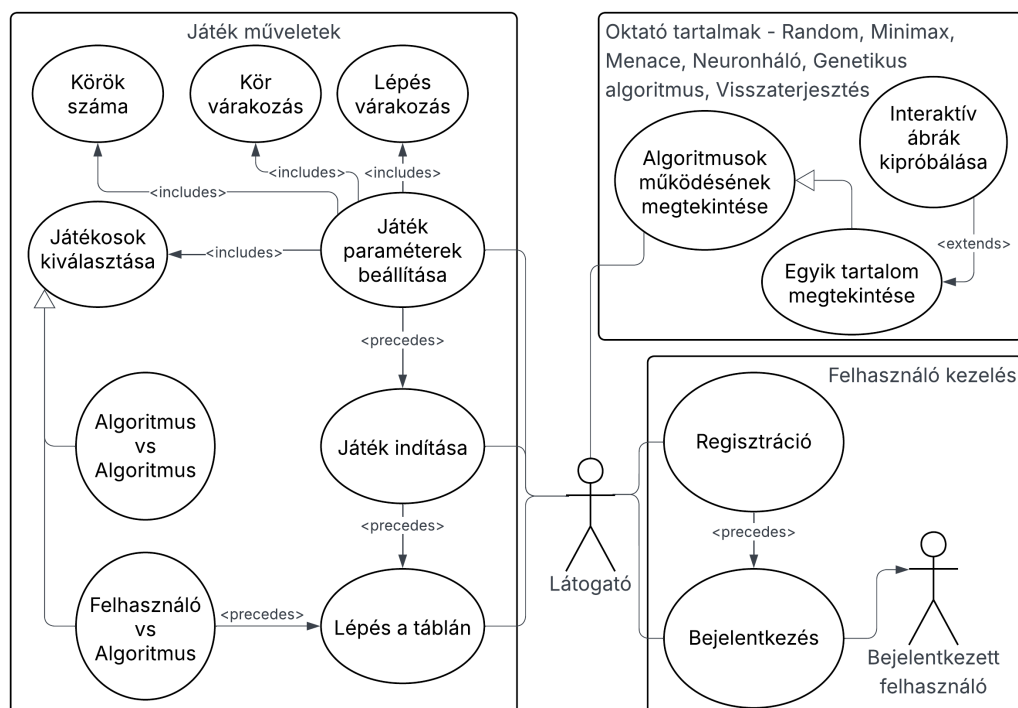
Fejlesztői dokumentáció

3.1. Követelményleírás

3.1.1. Funkcionális követelmények

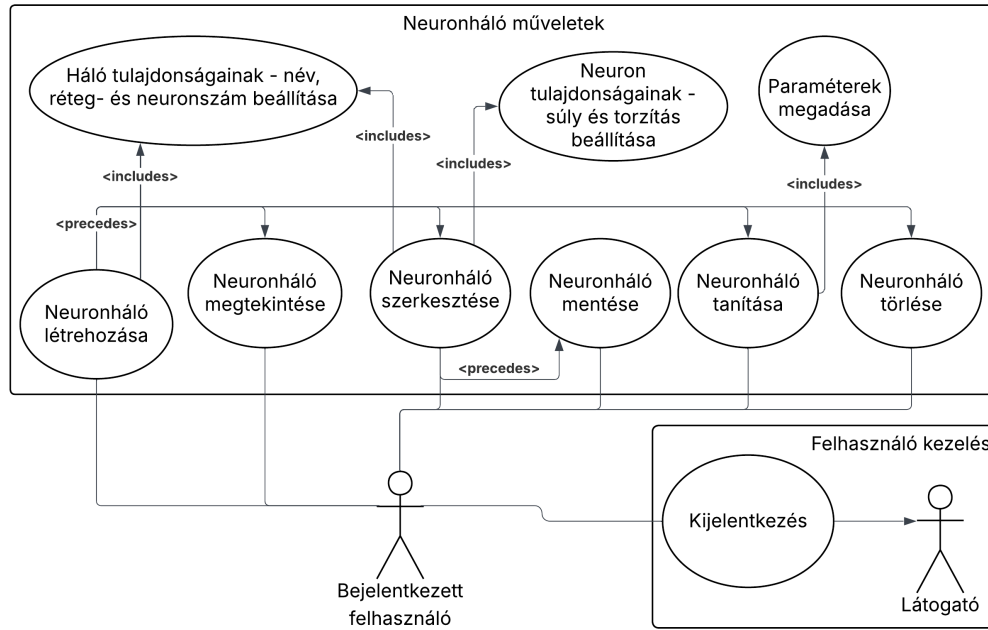
Használatieset-diagram

Látogató



3.1. ábra. A látogató használatieset-diagramja

Bejelentkezett felhasználó



3.2. ábra. A bejelentkezett felhasználó használatieset-diagramja

Megjegyzés. A bejelentkezett felhasználóra minden, a Látogatóra vonatkozó use-case is érvényes.

Felhasználói történetek

ID	Eset		Leírás
1.1	Regisztráció helyesen	GIVEN	A felhasználó a regisztrációs felületen tartózkodik.
		WHEN	A felhasználó érvényes és egyedi felhasználónevet, valamint kétszer érvényes jelszót ad meg.
		THEN	A rendszer létrehozza a fiókot és bejelentkezeti a felhasználót.
1.2	Regisztráció helytelenül	GIVEN	A felhasználó a regisztrációs felületen tartózkodik.
		WHEN	Hibás vagy hiányos adatokat ad meg.
		THEN	A rendszer nem hoz létre fiókot és hibaüzenetet jelenít meg.
2.1	Bejelentkezés helyesen	GIVEN	A felhasználó regisztrált és a bejelentkezési felületen van.

ID	Eset		Leírás
2.2	Bejelentkezés helytelenül	WHEN	A felhasználó a fiókjához tartozó felhasználónevet és jelszót helyesen megadja.
		THEN	A rendszer bejelentkezteti a felhasználót.
		GIVEN	A felhasználó a bejelentkezési oldalon tartózkodik.
3	Kijelentkezés	WHEN	A felhasználó olyan felhasználónevet és jelszót ad meg, melyhez nem tartozik felhasználói fiók.
		THEN	A rendszer nem engedi bejelentkezni és hibaüzenetet jelenít meg.
		GIVEN	A felhasználó be van jelentkezve.
4.1.1	Ellenfelek kiválasztása	WHEN	A felhasználó kijelentkezik.
		THEN	A rendszer kijelentkezteti a felhasználót. Ha védett felületen van, átirányítja a bejelentkezés felületre.
		GIVEN	A felhasználó a játékindító felületen tartózkodik.
4.1.2	Lépési várakozás megadása	WHEN	A felhasználó kiválasztja X és O játékost.
		THEN	A rendszer ellenőrzi, hogy legfeljebb az egyik játékos felhasználó, és beállítja a két játékost. Ha nem megfelelőek a játékosok, értesíti a felhasználót.
		GIVEN	A felhasználó a játékindító felületen tartózkodik.
4.1.3	Körök számának megadása	WHEN	A felhasználó megadja a várakozás mértékét.
		THEN	A rendszer ellenőrzi a megadott számot, majd beállítja a várakozást.
		GIVEN	A felhasználó a játékindító felületen tartózkodik.
		WHEN	A felhasználó megadja a körök számát.
		THEN	A rendszer ellenőrzi a megadott számot, majd beállítja a körök számát.
		GIVEN	A felhasználó a játékindító felületen tartózkodik.

ID	Eset		Leírás
4.1.4	Körök közötti várakozás megadása	GIVEN	A felhasználó a játékindító felületen tartózkodik.
		WHEN	A felhasználó megadja a körök közötti várakozás mértékét.
		THEN	A rendszer ellenőrzi a megadott számot, majd beállítja a várakozást.
4.2	Játék indítása	GIVEN	A felhasználó a játékindító felületen tartózkodik és beállította a játék paramétereit.
		WHEN	A felhasználó kezdeményezi a játék indítását.
		THEN	A rendszer elindítja a játékot a megadott beállításokkal.
4.3.1	Lépés a játékban, helyesen	GIVEN	A rendszer elindított egy algoritmus-felhasználó játékot, a felhasználó a játék felületen van és ő következik.
		WHEN	A felhasználó a játéktábla üres mezőjére kattint.
		THEN	A rendszer lehelyezi a megjelölt mezőre a játékos szimbólumát, majd megnézi, vége van-e a körnek, illetve a játéknak.
4.3.2	Lépés a játékban, helytelenül	GIVEN	A rendszer elindított egy algoritmus-felhasználó játékot, a felhasználó a játék felületen.
		WHEN	A felhasználó a játéktábla egy nem üres mezőjére kattint vagy olyankor kattint, amikor nem ő következik.
		THEN	A rendszer értesíti a felhasználót a lépés helytelenségéről és (ha a felhasználó következik) lehetőséget ad új lépésre.
4.3.3	Lépés a játékban, kör nyérése	GIVEN	A rendszer lehelyezte egy mezőre a felhasználó szimbólumát, és nem az utolsó kör zajlik.

ID	Eset		Leírás
		WHEN	A lehelyezett szimbólum sorában, oszlopában vagy átlójában háromszor szerepel a szimbólum.
		THEN	A felhasználó megnyerte a kört, új kört tud indítani a felhasználó.
4.3.4	Lépés a játékban, nincs vége a menetnek	GIVEN	A rendszer lehelyezte egy mezőre a felhasználó szimbólumát, és van még üres mező.
		WHEN	A lehelyezett szimbólum sorában, oszlopában vagy átlójában nem szerepel háromszor a szimbólum.
		THEN	Az ellenfél következik.
4.3.5	Lépés a játékban, döntetlen menet	GIVEN	A rendszer lehelyezte egy mezőre a felhasználó szimbólumát, nincs már üres mező.
		WHEN	A lehelyezett szimbólum sorában, oszlopában vagy átlójában nem szerepel háromszor a szimbólum.
		THEN	A menetnek vége, döntetlennel zárul, ha van még menet, a felhasználó új menetet tud indítani.
4.3.6	Játék vége	GIVEN	A játék összes menete lejátszásra került.
		WHEN	A felhasználó új menetet indítana.
		THEN	A felhasználó nem tud új menetet indítani. Új játékot tud indítani.
5.1	Algoritmus működésének megtekintése	GIVEN	A felhasználó a tanítás felületen tartózkodik és az tanítás navigációs menü meg van nyitva.
		WHEN	A felhasználó rákattint a megtekinteni kívánt algoritmus nevére.
		THEN	A felhasználó a kiválasztott algoritmus felületére kerül.
5.2	Algoritmus animáció megtekintése	GIVEN	A felhasználó egy algoritmus felületén van.

ID	Eset		Leírás
		WHEN	A felhasználó interakcióba lép az animált komponenssel.
		THEN	A rendszer az interakciónak megfelelően változtatja a komponenst.
6.1	Saját neuronháló létrehozása	GIVEN	A felhasználó be van jelentkezve és a neuronháló-kezelő felületen tartózkodik.
		WHEN	A felhasználó rétegszámot és nevet ad meg.
		THEN	A rendszer ellenőrzi az adatokat és létrehoz egy új neuronhálót. Ha az adatok nem megfelelőek, értesíti a felhasználót.
6.2.1	Saját neuronháló szerkesztése	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, és van mentett neuronhálójája.
		WHEN	A felhasználó kezdeményezi neuronhálójának szerkesztését.
		THEN	A rendszer a kiválasztott neuronhálójával megjeleníti a szerkesztő felületet.
6.2.2	Saját neuronháló szerkesztése, réteg beállítás	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, egy neuronháló van megnyitva a szerkesztő módban.
		WHEN	A felhasználó beállítja a rétegek számát / rétegekben szereplő neuronok számát.
		THEN	A rendszer validálja az adatokat, ha megfelelőek, menti a változtatásokat, ha nem, akkor értesíti a felhasználót.
6.2.3	Saját neuronháló szerkesztése, neuron paraméterek	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, egy neuronháló van megnyitva a szerkesztő módban és egy neuron ki van jelölve.
		WHEN	A felhasználó beállítja a súlyokat / a torzítást.

ID	Eset		Leírás
		THEN	A rendszer validálja az adatokat, ha megfelelőek, menti a változtatásokat, ha nem, akkor értesíti a felhasználót.
6.2.4	Saját neuronháló szerkesztése, név állítása	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, és egy neuronháló van megnyitva a szerkesztő módban.
		WHEN	A felhasználó megadja a neuronháló nevét.
		THEN	A rendszer ellenőrzi a megadott nevet, és menti a változtatást. Ha a megadott adat nem megfelelő, értesíti a felhasználót.
6.3	Saját neuronháló mentése	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, szerkesztő módban van, és történt változás a megnyitott neuronhálóban.
		WHEN	A felhasználó kezdeményezi neuronhálójának mentését.
		THEN	A rendszer ellenőrzi és elmenti a megnyitott neuronháló módosításait. Ha a módosítások érvénytelenek, értesíti a felhasználót.
6.4	Saját neuronháló megtekintése	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, és van mentett neuronhálójja.
		WHEN	A felhasználó kezdeményezi neuronhálójának megtekintését.
		THEN	A rendszer a kiválasztott neuronhálójával megjeleníti a szerkesztő felületet.
6.5	Saját neuronháló törlése	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, és van mentett neuronhálójja.
		WHEN	A felhasználó kezdeményezi neuronhálójának törlését.

ID	Eset		Leírás
		THEN	A rendszer törli a neuronhálót.
6.6.1	Saját neuronháló tanítása, paraméterek megadása	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, van mentett neuronhálójája, és a tanítás felület meg van nyitva.
		WHEN	A felhasználó megadja a tanítási paramétereket.
		THEN	A rendszer ellenőrzi a paramétereket, és rögzíti azokat. Ha az adatok nem megfelelőek, értesíti a felhasználót.
6.6.2	Saját neuronháló tanítása, tanítás indítása	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, van mentett neuronhálójája, és a tanítás felület meg van nyitva.
		WHEN	A felhasználó kezdeményezi neuronhálójának tanítását.
		THEN	A rendszer megkezdi a neuronháló tanítását a rögzített paraméterekkel.
6.6.3	Saját neuronháló tanításakor progresszió megjelenítése	GIVEN	A felhasználó be van jelentkezve, a neuronháló-kezelő felületen tartózkodik, és elindította egy neuronhálójának a tanítását.
		WHEN	A tanítási folyamat követve van.
		THEN	A rendszer valós idejű visszajelzést ad a tanulás előrehaladásáról.

3.1. táblázat. Felhasználói történetek

3.1.2. Nem funkcionális követelmények

Hatékonyaság

- Gyors válaszidő a felhasználói interakciókra.

- Legalább 2 egyidejű tanítási folyamat támogatása a szerveren.
- Legalább 30 egyidejű felhasználó kiszolgálása.
- A frontend alkalmazás betöltési ideje átlagosan kevesebb, mint 3 másodperc.

Megbízhatóság

- Szabványos használat esetén nem fordul elő hibajelenség.
- Minden bevitt adat validálásra kerül.
- Hibás vagy hiányos bemenet esetén olvasható hibaüzenetet kap a felhasználó, és újrapróbálási lehetőséget.
- A szerver minden API hívása HTTP státuszkóddal jelzi a sikerességet vagy hibát.

Biztonság

- A felhasználói neuronhálók csak hitelesítés után hozzáférhetőek.
- A jelszavaknak erősnek kell lenniük (minimum 8 karakter, tartalmazzon nagybetűt, kisbetűt és számot).
- A jelszavak sosem kerülnek a kliens oldalra.
- A jelszavak hashelve vannak tárolva.
- Az adatkommunikáció frontend és backend között WSS-en és HTTPS-en történik.

Hordozhatóság

- Az alkalmazás internetelés mellett, a modern böngészőkben elérhető.

Felhasználhatóság

- A funkciók nagy részének használata egyértelmű, nincs szükség segédletre, a saját neuronháló műveletekhez elérhető útmutató.
- Hiba esetén a felhasználó vizuális visszajelzést kap.

Működési

- Általában rövid futási idő, maximum 1-2 óra.

Környezeti

- Folyamatos internet elérés szükséges a rendszer működéséhez.
- A rendszernek szüksége van egy perzisztens adatbázis-szolgáltatásra.
- A szoftvernek nem kell más külső szoftverrel együttműködnie.

Fejlesztési

- A frontend fejlesztése React és TypeScript használatával történik.
- A stílusok kialakításához Tailwind CSS kerül alkalmazásra.
- A backend fejlesztése Python nyelven, FastAPI keretrendszer segítségével valósul meg.
- A projektben használt külső könyvtárak a *package.json* és *requirements.txt* fájlokban kerülnek felsorolásra.

3.2. Megvalósítás

Az alkalmazás három-rétegű architektúrát követ, melyek a következők:

- Prezentációs réteg
- Üzleti logika réteg
- Adatelérés réteg

A projekt fejlesztése során a következő eszközöket használtam:

- Visual Studio Code (verzió 1.105.1) [8] a kód írásához és szerkesztéséhez
- pgAdmin 4 (verzió 9.6) [9] az adatbázis kezeléséhez
- Git (verzió 2.44.0) [10] verziókezeléshez
- Docker Desktop (verzió 4.43.1) [11] (Docker Engine (verzió 28.3) és Docker Compose (verzió 2.38.1)) konténerek létrehozásához és futtatásához

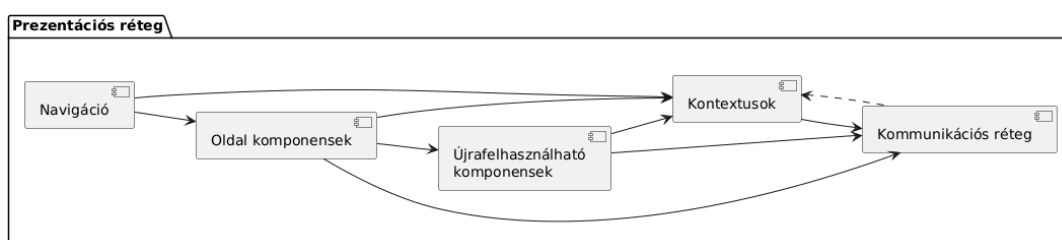
3.2.1. Prezentációs réteg

A prezentációs réteg kialakításához a React [12] könyvtár lett alkalmazva TypeScript [13] nyelven. Ez lehetővé teszi a komponens-alapú fejlesztést, így a felhasználói felület elemei újrahasznosíthatóak és könnyen karbantarthatóak. A stílusok kialakításához a Tailwind CSS [14] keretrendszer került alkalmazásra, utility-osztályaival egyszerűen formázhatóak a felhasználói felület elemei.

Architektúra áttekintés

A prezentációs réteg modern React architektúrát követ, a következő fő elemekből áll:

- **Navigáció:** Útvonalak kezelése, védett oldalak biztosítása
- **Kontextusok:** Megosztott állapotkezelés (autentikáció, értesítések, tanítás)
- **Kommunikációs réteg:** API hívások és valós idejű kapcsolatok (WebSocket, SSE)
- **Oldal komponensek:** Az alkalmazás főbb nézeteit reprezentálják
- **Újrafelhasználható komponensek:** Kisebb, önálló UI elemek



3.3. ábra. A prezentációs réteg fő komponensei és kapcsolataik

Navigáció

Az alkalmazás React Router [15] könyvtárat használ a navigációhoz. A fő útvonalak:

- `/`: Játékoldal avagy Kezdőlap (PlayPage)
- `/register`: Regisztrációs oldal (RegisterPage)
- `/login`: Bejelentkezési oldal (LoginPage)

- `/learn`: Tanulási tartalmak (`LearnPage`)
- `/playground`: Neuronháló szerkesztő / tanító oldal - védett (`PlaygroundPage`)

Az útvonalak védelme a `PrivateRoutes` burkoló komponens segítségével történik. A komponens az `AuthContext` használatával ellenőrzi, hogy a felhasználó be van-e jelentkezve. Ha nincs, átirányítja a `/login` oldalra.

A navigációs menü a `Navbar` komponensben valósul meg, amely dinamikusan jeleníti meg a menüpontokat a felhasználó hitelesítési állapotának függvényében. Ez a komponens az alkalmazás minden állapotában elérhető. A `LearnPage` oldalon belül a megjelenített tartalmak között a `LearnMenu` komponens segítségével lehet navigálni.

Kontextusok

Az alkalmazás négy kontextust használ a különböző állapotok kezelésére. Az `AuthContext` és az `ErrorContext` globális, több komponens között megosztott állapotokat tárol, míg a `JobContext` a tanulási adatok megőrzését biztosítja a `PlaygroundPage` komponens életciklusán túl. A `WindowSizeContext` a böngésző-ablak méretének követésére szolgál, és a komponensek reszponzivitását segíti elő. Az egyszerű használat érdekében egyedi hook-ok kerültek létrehozásra mind a négy kontextushoz.

AuthContext Az `AuthContext` felelős a felhasználói hitelesítésért és a token kezelésért, valamint ezen állapot megosztásáért az alkalmazás komponensei között.

- **Állapot:**
 - `user`: Aktuális, azonosított felhasználó adatai (ID, felhasználónév).
 - `accessToken`: JWT [16] access token.
 - `isReady`: A kontextus állapota, kezdeti bejelentkezési kísérlet lezárult-e.
 - `guestID`: Aktuális, azonosítatlan felhasználó ideiglenes azonosítója.
- **Metódusok:**
 - `login()`: Bejelentkezés felhasználónév és jelszó alapján.
 - `register()`: Új felhasználó regisztrációja.

- `logout()`: Kijelentkezés és token-ek törlése.
- `refreshAuth()`: Friss token-ek beszerése a refresh token segítségével.
- `getFreshToken()`: Garantáltan aktív access token lekérdezése, szükség esetén frissítéssel.
- `me()`: Aktuális felhasználó adatainak lekérdezése (ID, felhasználónév).

- **Hatások:**

- Az oldal betöltésekor, kezdeményezi a bejelentkezést. Amennyiben a kísérlet sikertelen, létrehoz egy vendég felhasználó azonosítót.
- A globálisan használt API példányhoz egy olyan interceptort ad hozzá, amely minden kéréshez hozzáfűzi az aktuális access token.
- A globálisan használt API példányhoz egy olyan interceptort ad hozzá, amely a 401-es válaszok esetén megpróbálja frissíteni a token, és újraküldi az eredeti kérést. A mechanizmus nem történik meg, ha a kérés már újrapróbálásra került vagy a hiba a refresh endpoint-ról érkezett.

A `getFreshToken()` különösen hasznos olyan esetekben, mint a WebSocket vagy SSE kapcsolatok, ahol a hitelesítési adatot a kapcsolat létrehozásakor kell megadni, és az interceptor mechanizmusa nem alkalmazható.

Az access token memóriában, a refresh token HttpOnly sütiben kerül tárolásra.

ErrorContext Az `ErrorContext` központosított hibaüzenet-kezelést biztosít az alkalmazásban.

- **Állapot:**

- `errors`: Üzenetek listája. Minden üzenet tartalmaz egy szöveget és egy egyedi azonosítót.

- **Metódusok:**

- `addError()`: Értesítés hozzáadása az `errors` listához. Az üzenet adott idő után automatikus eltávolításra kerül a listából.

- **Hatások:**

- Az értesítéseket a kontextusban szereplő `Notification` komponens jeleníti meg, toast stílusban.

JobContext A `JobContext` az éppen futó tanítási folyamatot követi nyomon.

- **Állapot:**

- `jobID`: Követett tanítási folyamat azonosítója.
- `jobNetworkName`: Követett tanítási folyamathoz tartozó neuronháló neve.

- **Metódusok:**

- `setJobID()`: A `jobID` beállítása.
- `setJobNetworkName()`: A `jobNetworkName` beállítása.
- `clearJob()`: Az állapot törlése.

- **Hatások:**

- A tanítást kezdeményező és követő `PlaygroundPage` komponens újratöltéskor, ha létezik `jobID`, automatikusan újracsatlakozik a tanításhoz. A felhasználó így válthat a lapok között anélkül, hogy elvesztené a tanítási folyamat követését. Az oldal újratöltése esetén a kontextusok állapota is törlődik, így a tanítási folyamat nem követhető tovább.

WindowSizeContext A `WindowSizeContext` az aktuális ablakméretet figyeli, és ezt töréspont változókon keresztül osztja meg. Lehetővé teszi például, hogy mobil eszközökön oda illő menü jelenjen meg.

- **Állapot:**

- `isCritical`: Kritikus méretű ablak.
- `isMobile`: Mobil eszközön méretű ablak.
- `isAboveSm`: Kisebb mint a *sm* töréspont.
- `isAboveMd`: Kisebb mint a *md* töréspont.
- `isAboveNormal`: Kisebb mint a *normal* töréspont.
- `isAboveLg`: Nagyobb mint a *lg* töréspont.

- **Hatások:**

- A komponensek a kontextus állapotát használva alkalmazkodnak a képernyőmérethez.

Kommunikációs réteg

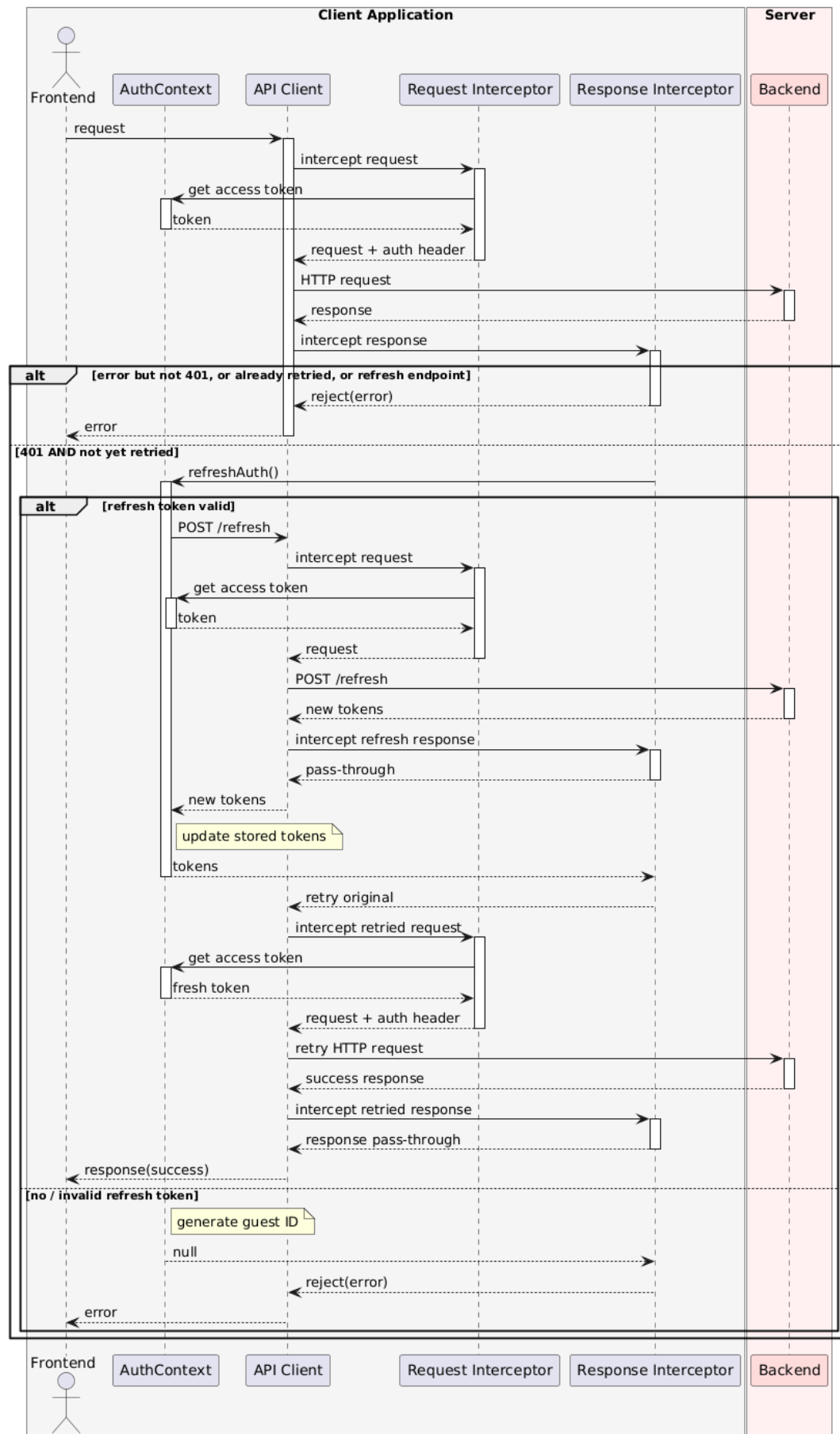
API kommunikáció Az `api` modul központosítja a REST API [17] hívásokat, és az Axios [18] könyvtár segítségével valósítja meg a HTTP kommunikációt.

- **Tartalom:**

- `api`: Konfigurált Axios példány, amelyet a komponensek API kliensként használnak.

- **Hatások:**

- A példány a backend alap URL-jével van konfigurálva, így a komponensekben csak a relatív végpontokat kell megadni.
- A példány alapértelmezetten JSON formátumot használ a kérések és a válaszok esetén is.
- A példány X-Requested-With fejlécet ad hozzá minden kéréshez.
- Az `AuthContext` erre az `api` példányra regisztrálja a hitelesítési folyamatokhoz használt interceptorokat.



3.4. ábra. Az AuthContext által az api példányra regisztrált Axios interceptor-ok működése

WebSocket kommunikáció A `websocket` modul segítségével lehet WebSocket [19] kapcsolatot létesíteni a backenddel. A modul egy egyedi React hook-ot (`useWebSocket()`) exportál, amely a komponensekben használható.

- **Állapot:**

- `ws`: Aktuális WebSocket kapcsolat példány.
- `status`: A kapcsolat állapota.
- `messageHandlers`: Regisztrált üzenetkezelők.

- **Metódusok:**

- `sendMessage()`: Üzenet küldése a WebSocket kapcsolaton keresztül.
- `addMessageHandler()`: Üzenetkezelő regisztrálása, leiratkozó függvény visszaadásával.

- **Hatások:**

- A kapcsolat felépítését a `shouldConnect` és `connectSignal` opciók vezérlik, így a felhasználó komponens explicit módon tud újracsatlakozást indítani.
- A beérkező üzeneteket JSON-ként feldolgozza, majd átadja a regisztrált üzenetkezelőknek
- A `useWebSocket()` a komponens életciklusához igazodva kezeli a kapcsolatot, és eltávolításkor automatikusan lezárja azt

SSE kommunikáció A harmadik kommunikációs mód a Server-Sent Events [20] használata. Különálló modulként nincs implementálva, a `PlaygroundPage` komponens `subscribeToTraining()` metódusában valósul meg.

Oldalak

Kezdőlap - Játéklap A `PlayPage` komponens feladata WebSocket kapcsolaton keresztül lehetővé tenni a játékot.

- **Állapot:**

- `wsUrl`: WebSocket szerver URL-je

- `gameState`: Játék állapota
- `settings`: Játék beállításai
- `guestID`, `user`: `AuthContext`ből származó hitelesítési adatok.
- `status`: `useWebSocket()` hookból származó kapcsolat állapota.

- **Metódusok:**

- `toggleConnection()`: WebSocket kapcsolat felépítése és bontása.
- `handleSendMessage()`: Üzenetek küldése a WebSocket kapcsolaton keresztül.
- Játékvezérlő üzenetkezelők: Különböző vezérlő típusú üzeneteket küldő függvények.

- **Alkomponensek:**

- `GameControls`: Beállítások megadása, játék indítása / frissítése.
- `GameBoard`: Játéktábla megjelenítése.
- `GameStats`: Játékállás megjelenítése.
- `ConnectionStatus`: WebSocket kapcsolat állapota.

- **Hatások:**

- A komponens betöltéskor létrehozza a WebSocket kapcsolatot a szerverrel.
- Sikeres kapcsolatfelépítés után *resume* üzenetet küld a szervernek, amely visszaküldi az utolsó játékállapotot, ha tartozik folyamatban lévő játék a felhasználó azonosítójához.
- A komponens figyeli az `AuthContext`ben lévő `user` és `guestID` változásokat, és ezek alapján automatikusan kiépíti és bontja a WebSocket kapcsolatot.
- Kijelentkezéskor automatikusan bontja a kapcsolatot és törli az állapotot

Hitelesítési lapok A `LoginPage` és `RegisterPage` hitelesítési lapok, amelyek az `AuthContext` szolgáltatásait használják. A regisztrációs oldal sikeres regisztráció után automatikusan bejelentkezteti a felhasználót. A bejelentkezett felhasználót mindkét oldal a kezdőlapra irányítja át.

Tanulási lap A LearnPage komponens a oktató tartalmakat jeleníti meg.

- **Állapot:**
 - `activeTopic`: A kiválasztott tanulási téma azonosítója.
 - `isMenuOpen`: Menü állapota.
- **Metódusok:**
 - `handleTopicChange()`: Beállítja az aktív témát és bezárja a menüt.
- **Alkomponensek:**
 - **navigáció**: LearnMenu komponens
 - **tartalom**:
 - * **Random**: RandomContent komponens.
 - * **Minimax**: MinimaxContent komponens.
 - * **MENACE**: MenaceContent komponens.
 - * **Neuronháló**: NeuralNetworkContent komponens.
 - * **Genetikus algoritmus**: GeneticContent komponens.
 - * **Visszaterjesztés**: BackpropagationContent komponens.
- **Hatások:**
 - A kiválasztott téma alapján dinamikusan jeleníti meg a megfelelő tartalom komponensét.
 - Témaváltáskor az oldal tetejére görget.

Szerkesztő- és tanítólap A PlaygroundPage komponens a neuronhálók szerkesztésére és tanítására szolgáló felületeteket foglalja magába.

- **Állapot:**
 - `networks`: A bejelentkezett felhasználó mentett neuronhálóinak listája.
 - *segédváltozók*: A létrehozási, szerkesztési és tanítási folyamatokhoz szükséges változók.
 - `jobStatus`: A tanítási folyamat állapota.

- **sseRef**: EventSource példány referenciája.
- **pollerRef**: Az időzített lekérdezés (polling) referenciája.
- **heartbeatRef**: Az SSE kapcsolatot figyelő időzítő referenciája.
- **lastMessageRef**: Az SSE kapcsolaton érkező utolsó üzenet időbélyegének referenciája.
- **pollingTimeoutRef**: Az időzített lekérdezésnél használt időzítő referenciája.

- **Metódusok:**

- **listNetworks()**: Mentett neuronhálók lekérdezése.
- **createNetwork()**: Új neuronháló létrehozása.
- **saveNetwork()**: Neuronháló mentése.
- **deleteNetwork()**: Neuronháló törlése.
- **trainNetwork()**: Neuronháló tanításának indítása.
- **subscribeToTraining()**: SSE kapcsolat létrehozása a tanítási állapot követésére.
- **startPolling()**: Időzített lekérdezés indítása a tanítási állapot követésére.

- **Alkomponensek:**

- **NetworkCreateForm**: Új hálózat létrehozása, paraméterek megadása.
- **NetworkTable**: Mentett hálózatok listája műveleti gombokkal.
- **NetworkEditor**: Neuronháló szerkesztése.
- **NetworkTrainingForm**: Tanítás indítása, paraméterek megadása.
- **NetworkTrainingStatus**: Tanítási folyamat állapotának követése.

- **Hatások:**

- Komponens betöltésekor lekéri a bejelentkezett felhasználó mentett neuronhálóit, ehhez a **listNetworks()** metódust használja.
- Tanítás indításakor létrehozza az SSE kapcsolatot, és szükség esetén időzített lekérdezésre vált, ehhez a **subscribeToTraining()** és **startPolling()** metódusokat használja.

- Komponens újracsatolásakor ellenőrzi a `JobContext`-ben lévő `jobID`-t, és ha létezik, újracsatlakozik a tanításhoz, ehhez a `subscribeToTraining()` metódust használja.

Tanítási folyamat követése Elsődleges módja a valós idejű követés SSE használatával. A `subscribeToTraining()` metódus létrehozza az `EventSource` objektumot, amely a backend SSE végpontjához csatlakozik. Az objektumhoz üzenetkezelőt rendel, és referenciáját a `sseRef` változóban tárolja. Az SSE kapcsolat állapotának figyelésére egy heartbeat mechanizmust hoz létre, amely időközönként ellenőrzi a legutóbbi üzenet érkezése (`lastMessageRef`) óta eltelt időt. Ha az eltelt idő egy határon túl van, a kapcsolatot rossznak tekinti, bezárja azt, majd időzített lekérdezésre vált. Az időzített lekérdezés a `startPolling()` metódusban valósul meg, amely periodikusan lekéri a tanítási folyamat állapotát a backend megfelelő végpontjától.

Részkomponensek

Matematikai megjelenítők A `MathLatex` és `MathWhere` komponensek a matematikai kifejezések megjelenítésére szolgálnak. A renderelést a `MathJax` [21] végzi, amely a `better-react-mathjax` [22] könyvtáron keresztül kerül integrálásra.

Rulettkerék A `SpinningWheel` komponens egy pörgethető kereket nyújt. A rajzolás a `Canvas API` [23]-val történik.

Ábrák A függvények és neuronhálók ábrázolása *svg*-ként történik.

Példák:

- `ReluVisual`: ReLu függvény grafikonja.
- `SoftmaxVisual`: A Softmax függvény grafikonja.
- `NeuralNetworkVisual`: Neuronháló ábra.
- `SingleNeuronVisual`: Neuron ábra.

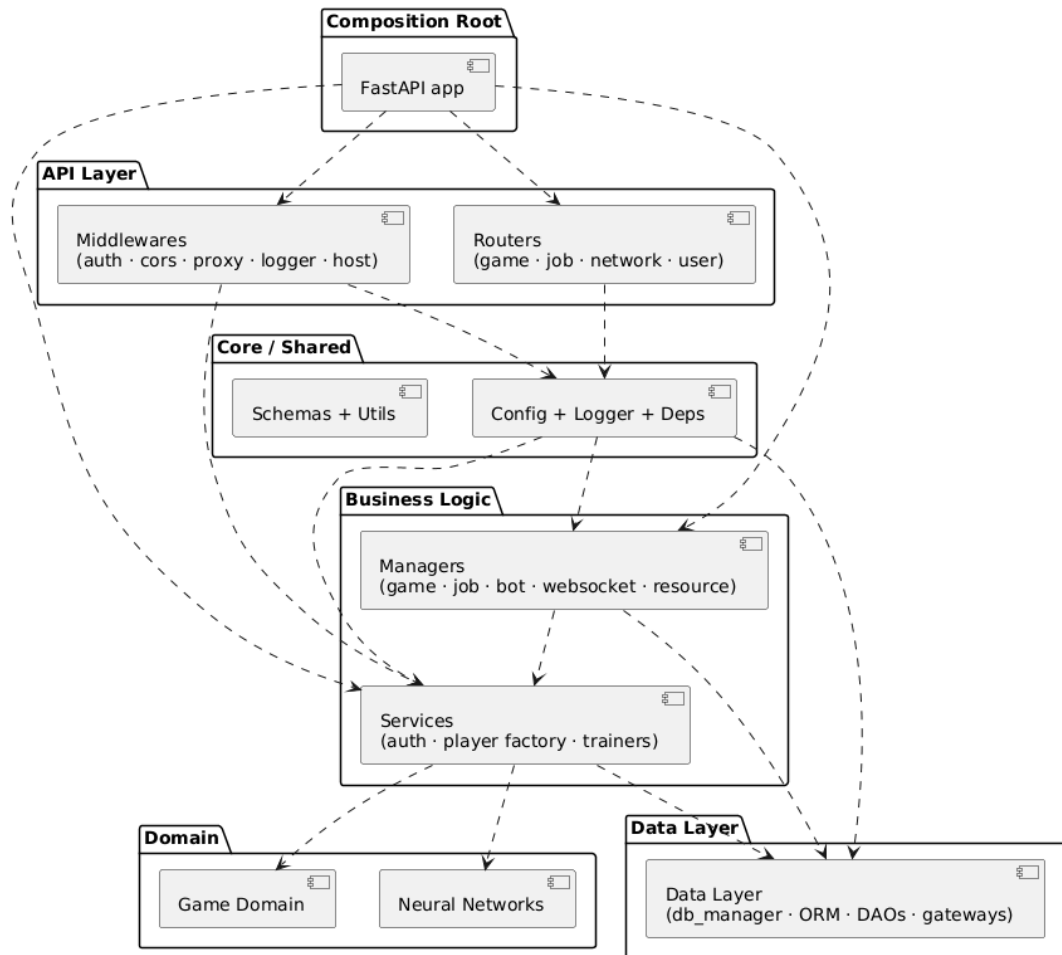
Értesítő A `Notification` komponens üzenetek megjelenítéséért felelős. A komponens a toast stílus eléréséhez, az üzenetek animálásához a `Motion` [24] könyvtárat használja.

Egyéb komponensek Az alkalmazásban számos további komponens kerül felhasználásra, megismerésük a forráskódban javasolt.

3.2.2. Üzletlogika-réteg

Az üzletlogika-réteg Python nyelven, a FastAPI [25] keretrendszerrel került megvalósításra, az alkalmazást az Uvicorn [26] ASGI-szerver kezeli. Ez a réteg biztosítja az alkalmazás alapvető funkcionalitását, többek között a hitelesítést, a játéklógikát, a neuronháló kezelését és tanítását, valamint ezen funkciók elérhetővé tételét a prezentációs réteg számára.

Architektúra áttekintés



3.5. ábra. Az üzletlogika-réteg komponens diagramja¹

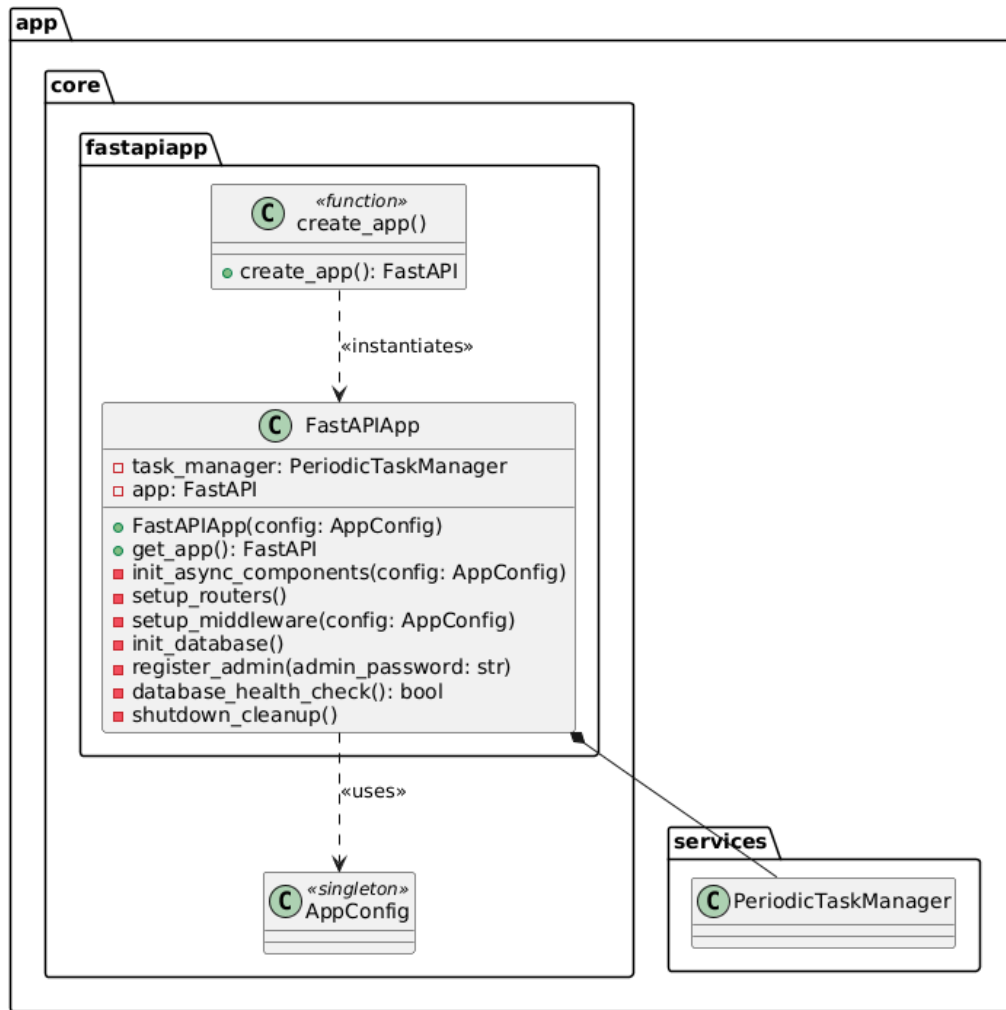
A rendszer fontos komponenseit a következők alapján csoportosítva tárgyalom:

¹A *Core / Shared* komponenseket szinte az összes többi felhasználja, ezen kapcsolatok az átláthatóság érdekében nem kerültek ábrázolásra.

- **FastAPI alkalmazás:** Az alkalmazás magja, konfigurációja, és életciklusa.
- **API végpontok:** REST API [17], WebSocket [27] és Server-Sent Events [28] végpontok.
- **Szolgáltatások (és menedzserek):** A logika megvalósítása.
- **Játéklogika:** Játékszabályok és játékos implementációk.
- **Algoritmusok:** Játékosként használt algoritmusok.

FastAPI alkalmazás

A rendszerben a `FastAPIApp` osztály látja el a központi inicializációs és konfigurációs feladatokat. Feladata, hogy létrehozza a FastAPI [25] példányt, amelyhez ezután hozzáadja a különböző útvonalakat és a szükséges middleware-rétegeket, például a CORS-kezelést. Az osztály kezeli az alkalmazás teljes életciklusát. Az indulási folyamat során ellenőrzi és inicializálja az adatbázist, létrehozza az alapértelmezett adminisztrátori felhasználót, valamint elindítja a periodikus takarító háttérfeladatokat. Az osztály a leállítási fázis során leállítja a még futó aszinkron feladatokat, folyamatokat és végrehajtja a szükséges takarító műveleteket.

3.6. ábra. A FastAPIApp osztály²

Inicializálás és konfiguráció A `create_app()` függvény hozza létre a FastAPI példányt, amely a `FastAPIApp` osztályba van burkolva.

A konfiguráció a `AppConfig` singleton osztályban van központosítva, amely környezeti változókból tölti be a beállításokat.

Middleware regisztráció Az alkalmazás öt middlewaret használ, melyeket az `AppConfig` adataival konfigurál.

- `TrustedHostMiddleware` [29]
- `AuthMiddleware`: Hitelesítés előkészítése, JWT token (sütiből vagy paraméterből) dekódolása és a felhasználói adatok hozzáadása a kérés kontextusához.

²Számos használt osztály hiányzik a diagramról

- ProxyMiddleware [30]
- CORSMiddleware [31]
- LoggerMiddleware: HTTP kérések naplózása.

Router regisztráció Öt router modul kerül regisztrálásra. Minden router egy-egy logikai egységet képvisel, és a hozzájuk tartozó végpontokat csoportosítja.

Életciklus kezelés A FastAPI lifespan kontextuskezelő segítségével az alkalmazás indulásakor és leállásakor speciális műveletek hajtódnak végre.

Indulás:

1. **Erőforrások:** Szemafor és processpool létrehozása.
2. **Adatbázis:** Kapcsolat ellenőrzése az adatbázissal, majd táblák és indexek létrehozása.
3. **Admin regisztráció:** Alapértelmezett admin felhasználó létrehozása.
4. **Alapértelmezett ellenfelek tanítása:** MENACE és neuronhálók betanítása.
5. **Periodikus feladatok indítása:** Takarító háttérfeladatok elindítása.

Leállítás:

1. **Periodikus feladatok leállítása**
2. **Folyamatok leállítása:** A tanításhoz kapcsolódó folyamatok és feladatok felszámolása.
3. **WebSocket kapcsolatok lezárása**
4. **Erőforrások:** Szemafor és processpool felszámolása.
5. **Adatbázis:** Adatbázis kapcsolat lezárása.

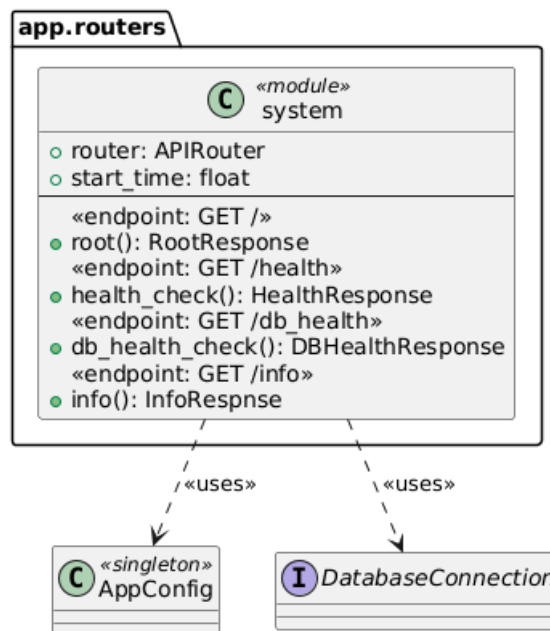
API végpontok

A router modulok `APIRouter` [32] példányt használnak.

Az alkalmazás három kommunikációs protokollt használ:

- **REST API** [17]: Hagyományos HTTP kérés-válasz
- **WebSocket** [27]: Kétirányú, valós idejű kommunikáció
- **Server-Sent Events** [33]: Egyirányú szerver-kliens streaming

System Router (/) A rendszer állapotához kapcsolódó végpontok gyűjteménye.

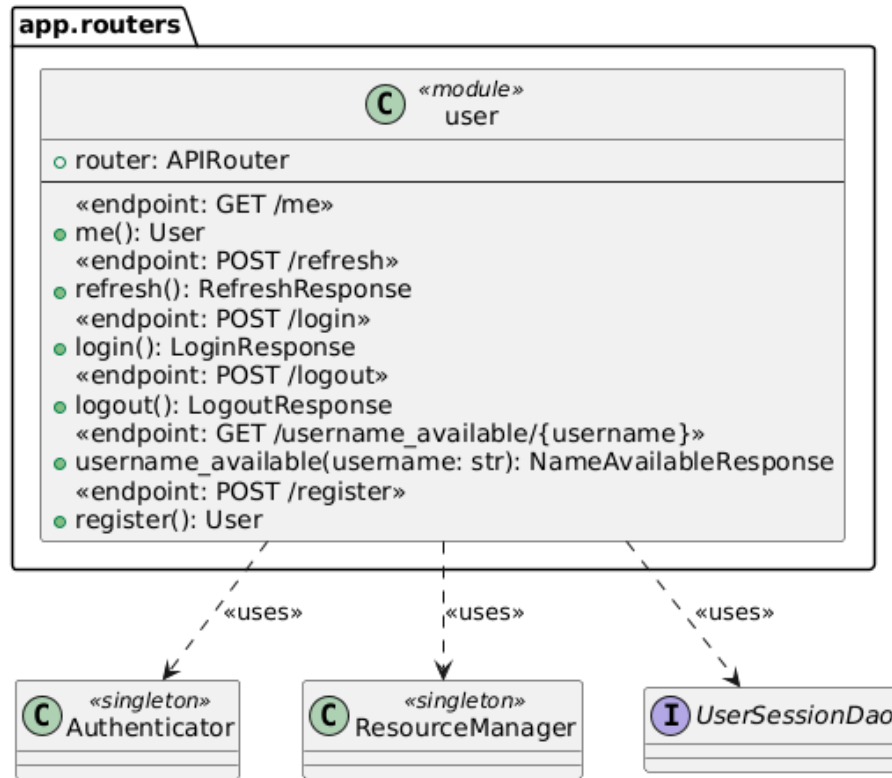


3.7. ábra. A System Router

Végpontok:

- `GET /health`: Egyszerű health check.
- `GET /db_health`: Adatbázis kapcsolat ellenőrzése.
- `GET /`: Üdvözlő üzenet.
- `GET /info`: Alapvető információk.

User Router (/users) Felhasználó hitelesítés és munkamenet kezelés.

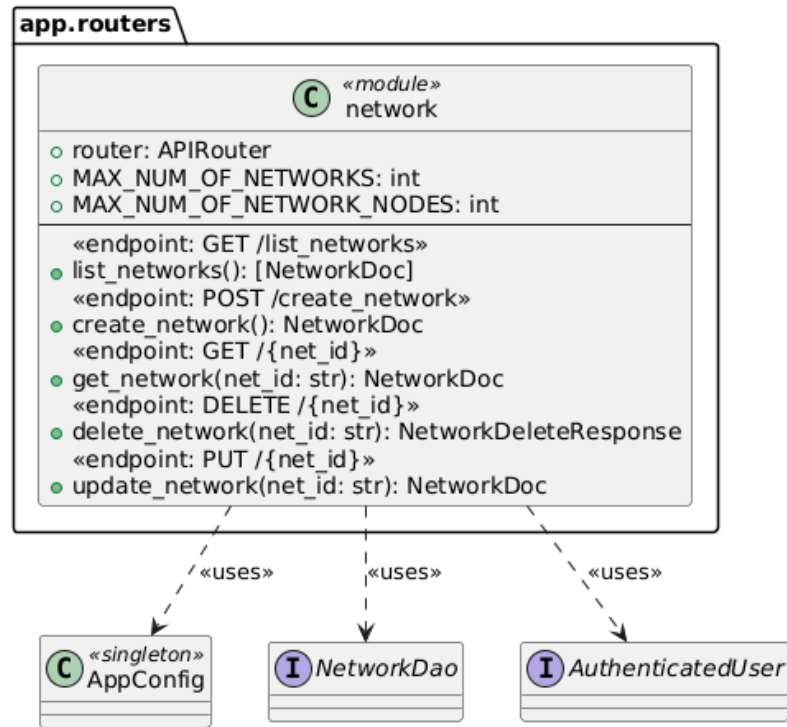


3.8. ábra. A User Router

Végpontok:

- GET /users/me: Bejelentkezett felhasználó adatai.
- POST /users/register: Új felhasználó regisztrációja.
- GET /users/username_available/{username}: Felhasználónév elérhetőség ellenőrzése.
- POST /users/login: Bejelentkezés.
- POST /users/refresh: Token frissítés refresh token segítségével.
- POST /users/logout: Kijelentkezés, munkamenet visszavonása.

Network Router (/networks) Neuronhálók CRUD műveletei.

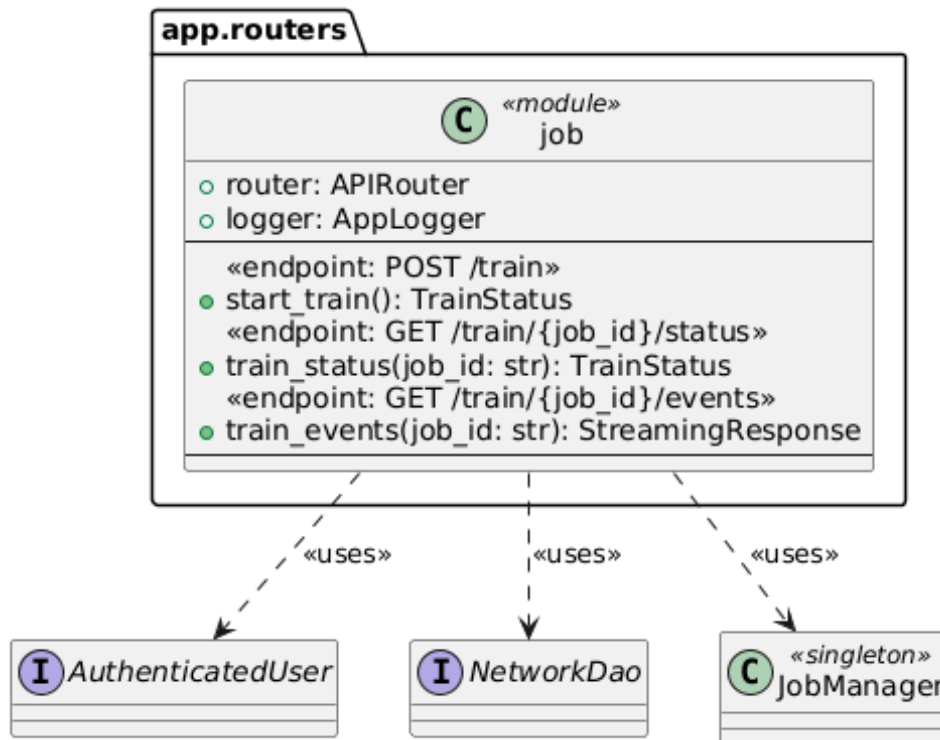


3.9. ábra. A Network Router

Végpontok:

- GET /networks/list_networks: Felhasználó neuronhálóinak listázása.
- POST /networks/create_network: Új neuronháló létrehozása.
- GET /networks/{net_id}: Egy neuronháló lekérdezések.
- PUT /networks/{net_id}: Neuronháló frissítése.
- DELETE /networks/{net_id}: Neuronháló törlése.

Job Router (/jobs) A tanítási folyamatok kezelése és követése.



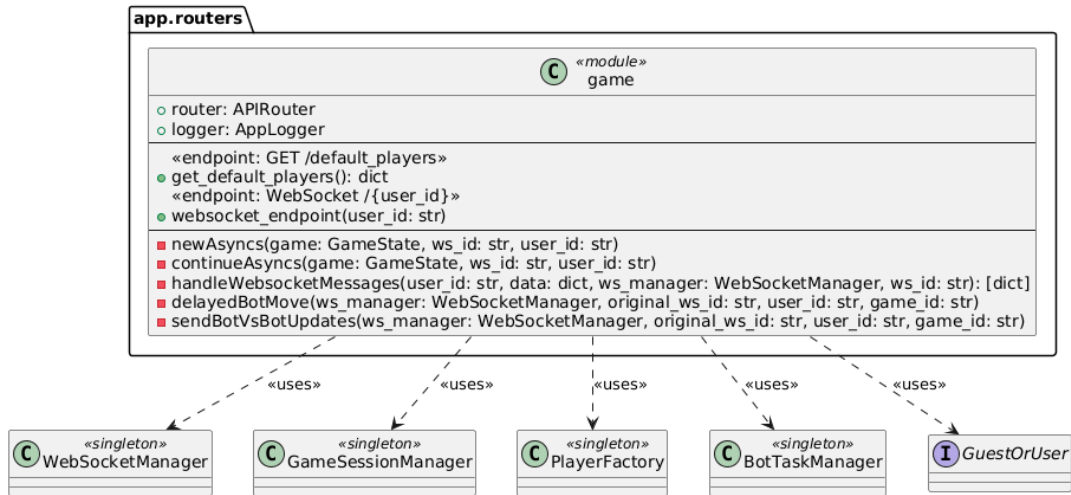
3.10. ábra. A Job Router

Végpontok:

- `POST /jobs/train`: Új tanítási folyamat indítása.
- `GET /jobs/train/{job_id}/status`: Tanítási folyamat állapotának lekérdezése.
- `GET /jobs/train/{job_id}/events`: Server-Sent Events stream a valós idejű állapotkövetéshez.

A tanítás indításához, követéséhez a **JobManager** osztály van felhasználva. A `train_events()` függvény létrehozza az SSE kapcsolatot a klienssel, és egy generátort [34] használ a **JobManager** queue-ából érkező események streamelésére.

Game Router (/game) Játékosok és WebSocket alapú valós idejű játék.



3.11. ábra. A Game Router

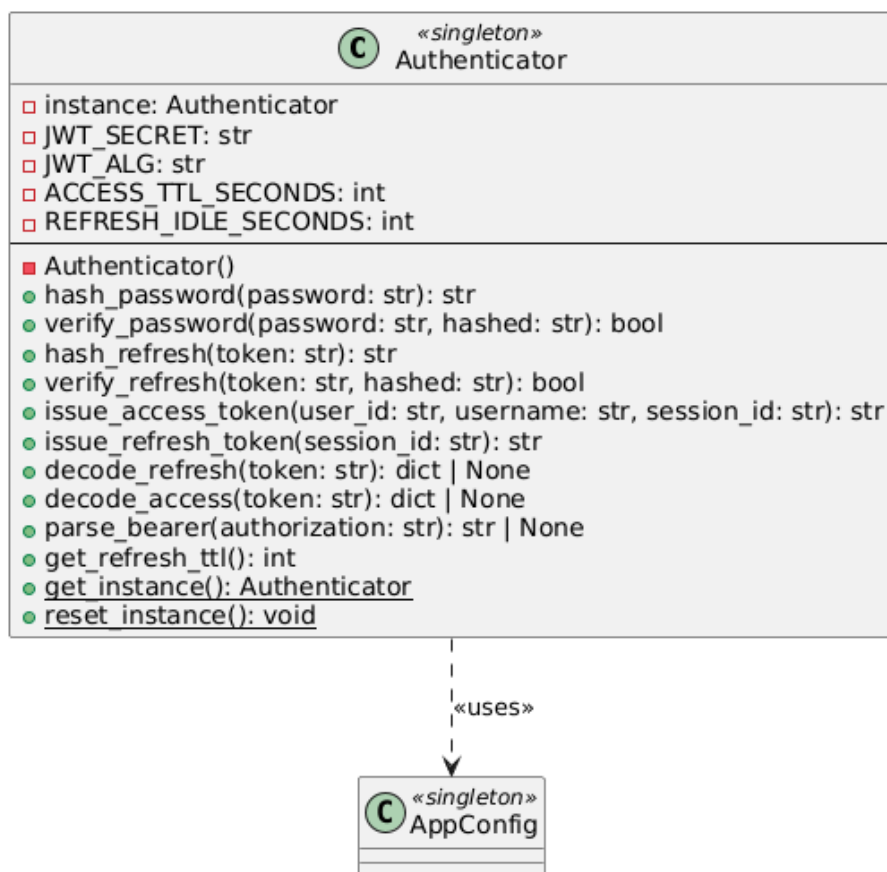
Végpontok:

- GET /game/default_players: Alapértelmezett ellenfelek.
- WS /game/{user_id}: WebSocket kapcsolat játékhoz (hitelesített felhasználónál token query paraméterrel, vendégnél token nélkül).

Szolgáltatások

A szolgáltatások az *app.services* csomagban találhatók.

Authenticator Az Authenticator egy singleton osztály a hitelesítési műveletek központosítására.



3.12. ábra. Az Authenticator osztály

Felelősségek:

- **Jelszó hashelés:** bcrypt algoritmussal, salt generálással.
- **Jelszó verifikálás:** bcrypt hash ellenőrzés.
- **Refresh token hashelés:** SHA-256 algoritmussal.
- **JWT access token generálás:** Felhasználó ID, username, session ID, lejárati idő, token típus tartalmával.
- **JWT refresh token generálás:** Session ID, kiállítás ideje, token típus tartalmával.
- **Token dekódolás és validálás:** HS256 algoritmussal, típus ellenőrzéssel.

GameSessionManager A GameSessionManager osztály a játékok kezeléséért felelős.



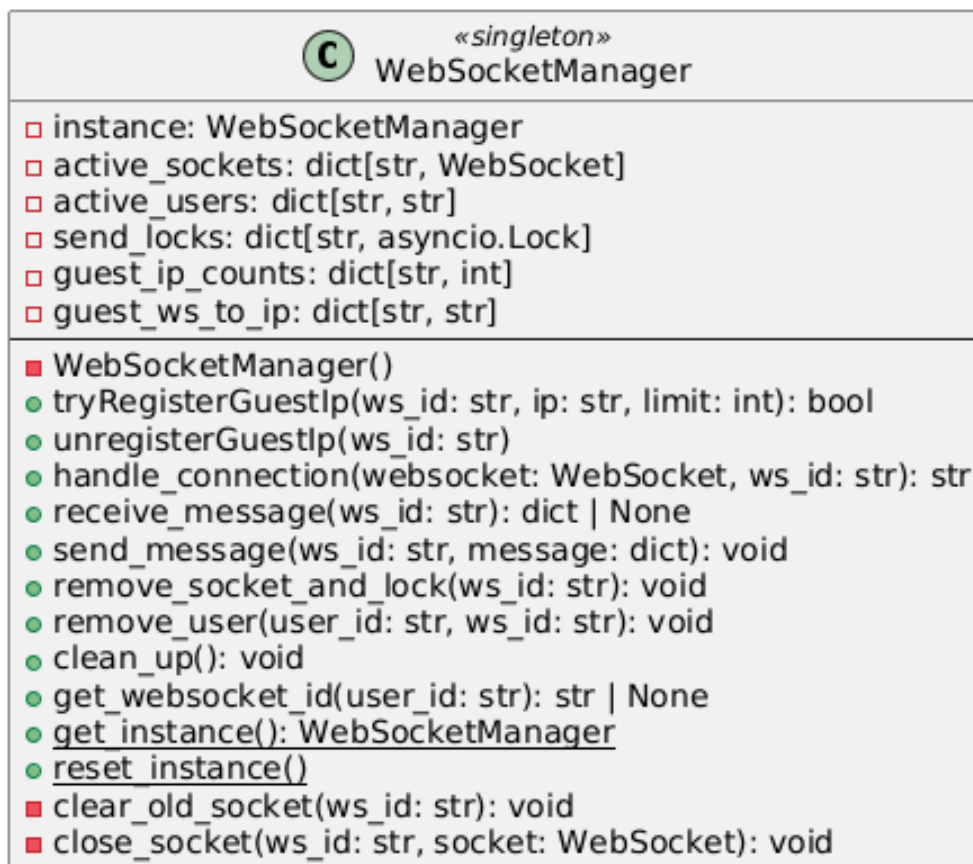
3.13. ábra. A GameSessionManager osztály

Felelősségek:

- **Játék létrehozás:** Két játékos inicializálása, játéktábla és beállítások tárolása.
- **Játék frissítés:** Játék beállítások módosítása.
- **Lépés validálás és végrehajtás:** Játékos lépéseinek ellenőrzése és alkalmazása.
- **Győzelem ellenőrzés:** Sorok, oszlopok, átlók vizsgálata.
- **Lépés generáltatás:** Megfelelő játékos léptetése.
- **Munkamenetek kezelése:** Játék mentése és visszatöltése azonosítóhoz kötve.

A `prune_inactive_sessions()` függvény törli az inaktív játékokat a szótárból.

WebSocketManager A WebSocketManager osztály a WebSocket kapcsolatok kezeléséért felelős. Felhasználónként egy aktív kapcsolat lehet.

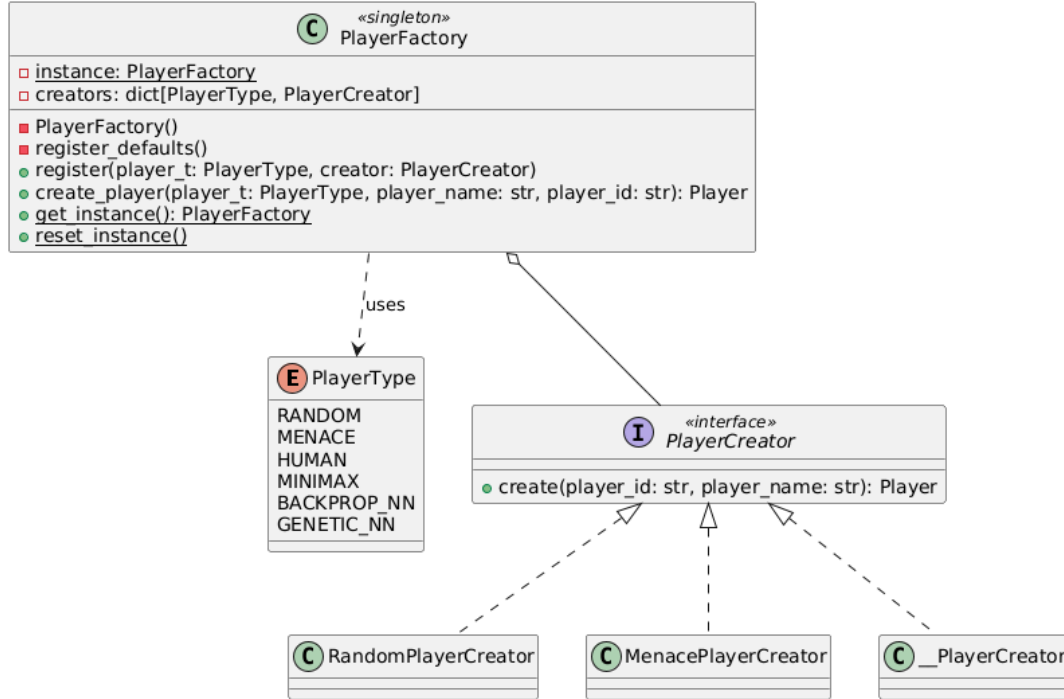


3.14. ábra. A WebSocketManager osztály

Felelősségek:

- **Kapcsolat engedélyezés:** Adott IP címről csak bizonyos számú vendég hozhat létre kapcsolatot.
- **Kapcsolat kezelés:** WebSocket fogadás, tárolás, lezárás.
- **Üzenet küldés/fogadás:** JSON formátumú üzenetek küldése és fogadása.
- **Előző kapcsolatok tisztítása:** Ugyanazon felhasználó új kapcsolatának létrehozásakor a régi lezárása.
- **Cleanup:** Kapcsolatok lezárása és az állapotok törlése.

PlayerFactory A PlayerFactory singleton osztály különböző típusú játékosok létrehozásáért felel. Ezt a feladatot egy factory-registry minta segítségével valósítja meg, ahol a játékos típusokhoz gyáarak vannak társítva.



3.15. ábra. A PlayerFactory osztály³

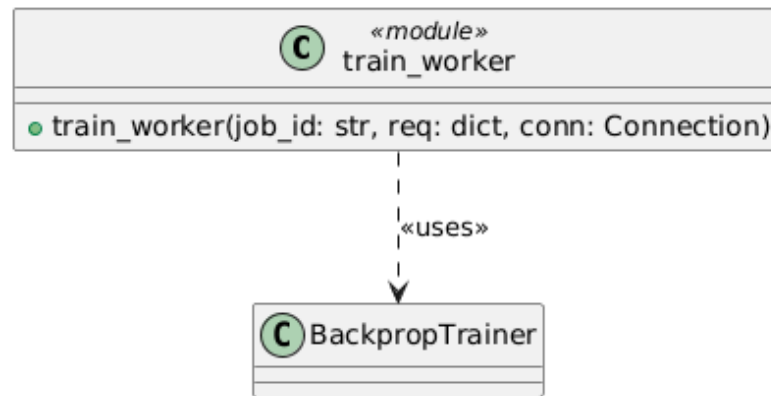
Felelősségek:

- **Játékos létrehozás az egyes factoryk segítségével:** A megadott típus és paraméterek alapján megfelelő játékos objektum létrehozása. Adott típusok esetén adatbázis lekérdezés szükséges:
 - *Random*: Azonnal létrehozható, nincs adatbázis függőség.
 - *Minimax*: Azonnal létrehozható, nincs adatbázis függőség.
 - *Human*: Azonnal létrehozható, nincs adatbázis függőség.
 - *MENACE*: MenaceDao lekérdezés ID alapján, gyufásdoboz adatok betöltése.
 - *Backprop NN*: NetworkDao lekérdezés ID alapján, NeuralNetwork objektum létrehozása a mentett konfigurációval.
 - *Genetic NN*: EvolutionDao lekérdezés ID alapján, NeuralNetwork objektum létrehozása a mentett konfigurációval.

³A _PlayerCreator a további játékosgyáarakat reprezentálja

A neurális hálózatok tanítása számításigényes művelet, ezért indokolt azt úgy végrehajtani, hogy a fő alkalmazást a legkevésbé terhelje. Mivel a tanítás során végzett számítások ebben a megvalósításban döntően Python nyelven írt, CPU-igényes ciklusokban történnek, a szálak (`threads`) [35] [36] használata a GIL [37] miatt nem biztosít megfelelő megoldást. Ezért a megvalósítás a `multiprocessing` [38] modult alkalmazza, amely külön folyamatok indításával végzi a tanítási feladatok futtatását. Az alábbi osztályok ezt a rendszert valósítják meg.

TrainerWorker A `train_worker()` a tanítási folyamatok végrehajtásában vesz részt.

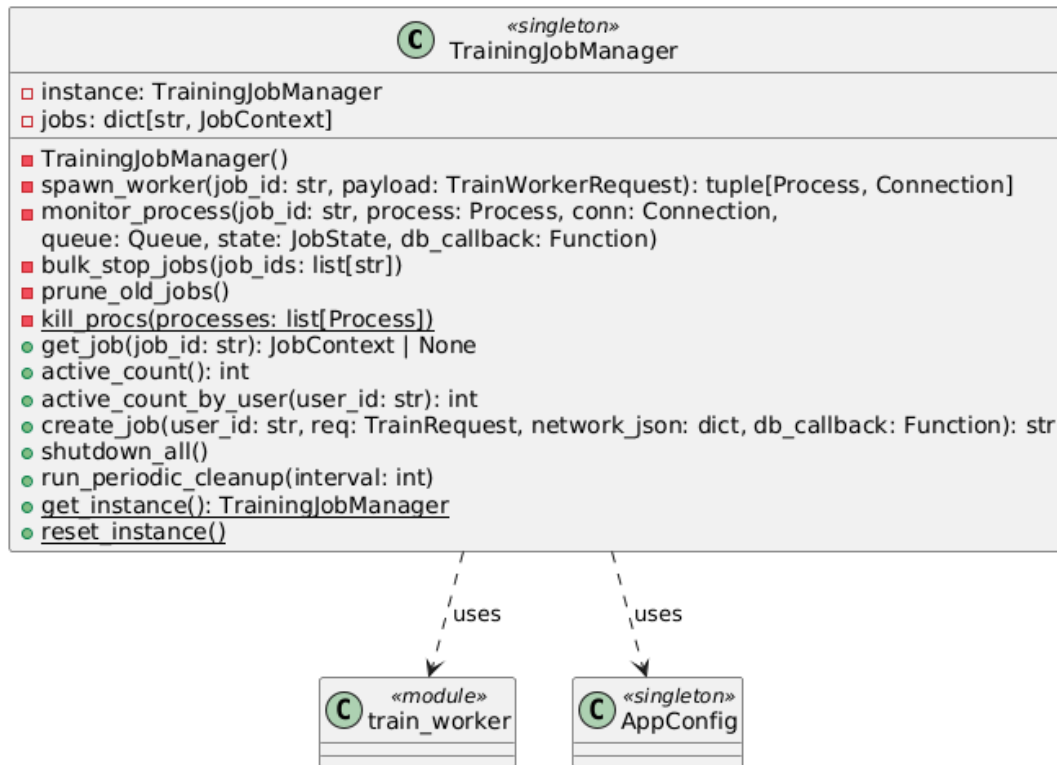


3.16. ábra. A TrainerWorker modul

Működés:

1. A függvény paraméterként megkapja egy `Pipe` [39] egyik végét és egyéb konfigurációkat.
2. A `BackpropTrainer`-nek átadja a szükséges paramétereket, ezzel elindítva a tanítási folyamatot.
3. A `BackpropTrainer` generátor-ként [34] működik, periodikusan visszaadja a tanítás állapotát.
4. A függvény a `Pipe`-on keresztül küldi vissza az állapotfrissítéseket.

TrainingJobManager A `TrainingJobManager` osztály a híd a tanítási folyamatok - `TrainerWorker` és a `Job Router` között.



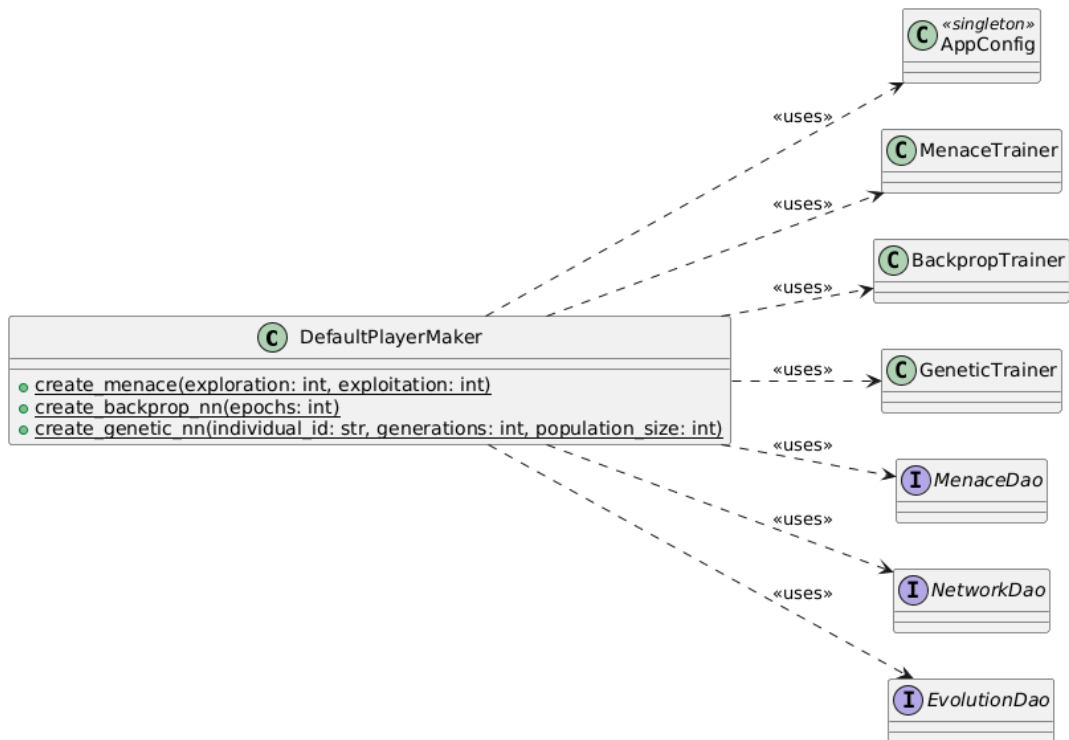
3.17. ábra. A TrainingJobManager osztály

Felelősségek:• **Job indítása:**

1. Megszorítások ellenőrzése (pl. egyidejű tanítási folyamatok száma).
2. ID generálás és kezdeti állapot létrehozása.
3. Egy új Process-ben a TrainerWorker indítása, Pipe használatával a kommunikációhoz.
4. Monitor task indítása - a Pipe-on érkező üzenetek Queue-ba [40] továbbítása.

- **Folyamatok kezelése:** Az elindított folyamatok és monitor task-ok nyilvántartása, leállítása

DefaultPlayerMaker A DefaultPlayerMaker osztály feladata az alapértelmezett ellenfelek létrehozása és tanítása.



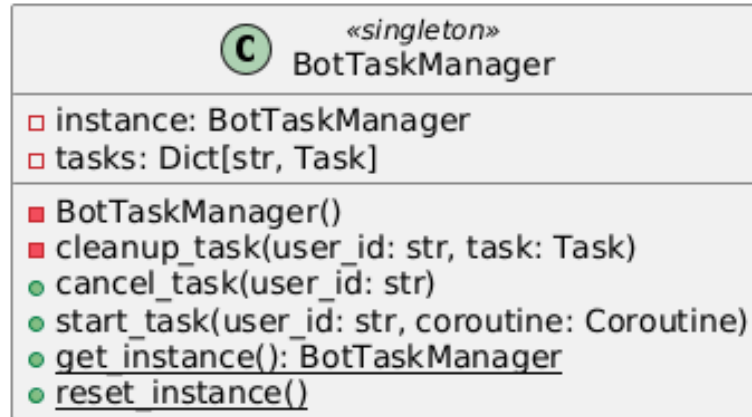
3.18. ábra. A DefaultPlayerMaker osztály

Felelősségek:

- `create_menace()`: MENACE inicializálás, tanítás random ellen, majd self-play, adatbázisba mentés.
- `create_backprop_nn()`: Neuronháló inicializálás, visszaterjesztéses tanítás, adatbázisba mentés.
- `create_genetic_nn()`: Genetikus algoritmussal neuronháló populáció evolválása, legjobb egyed mentése adatbázisba.

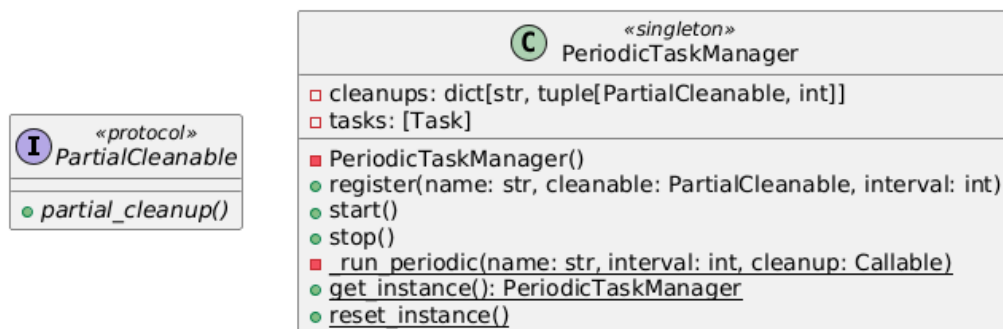
Feltételes létrehozás: Csak akkor hoz létre új játékost, ha a környezeti változóban szereplő ID-k még nem léteznek az adatbázisban.

BotTaskManager A BotTaskManager egy egyszerű osztály taskok kezelésére. A Game Router használja a botok lépéseihez kapcsolódó aszinkron taskok bonyolítására.



3.19. ábra. A BotTaskManager osztály

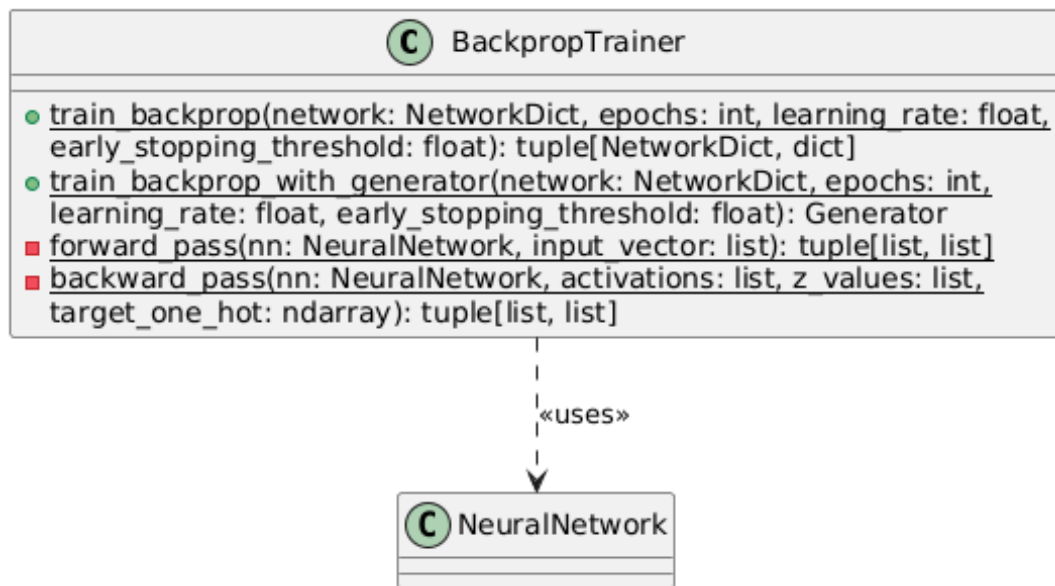
PeriodicTaskManager A PeriodicTaskManager biztosítja a karbantartó műveletek periodikus végrehajtását.



3.20. ábra. A PeriodicTaskManager osztály

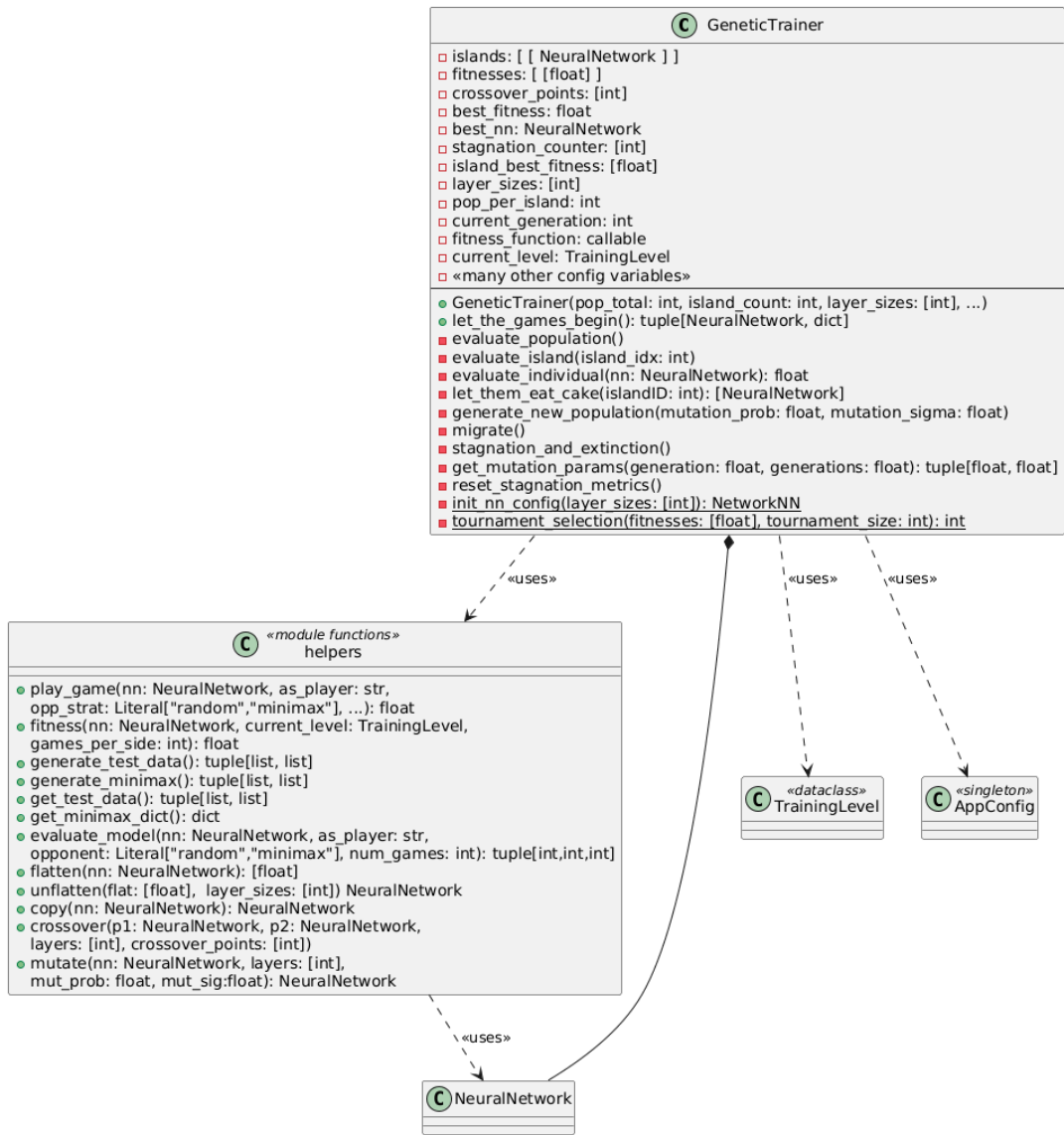
Trainers Az app.services.trainers csomag tartalmazza a különböző tanító osztályokat. A tanítási folyamatokhoz felhasznált algoritmusok az Algoritmusok részben vannak tárgyalva.

BackpropTrainer A BackpropTrainer osztály a visszaterjesztéses tanítást valósítja meg.



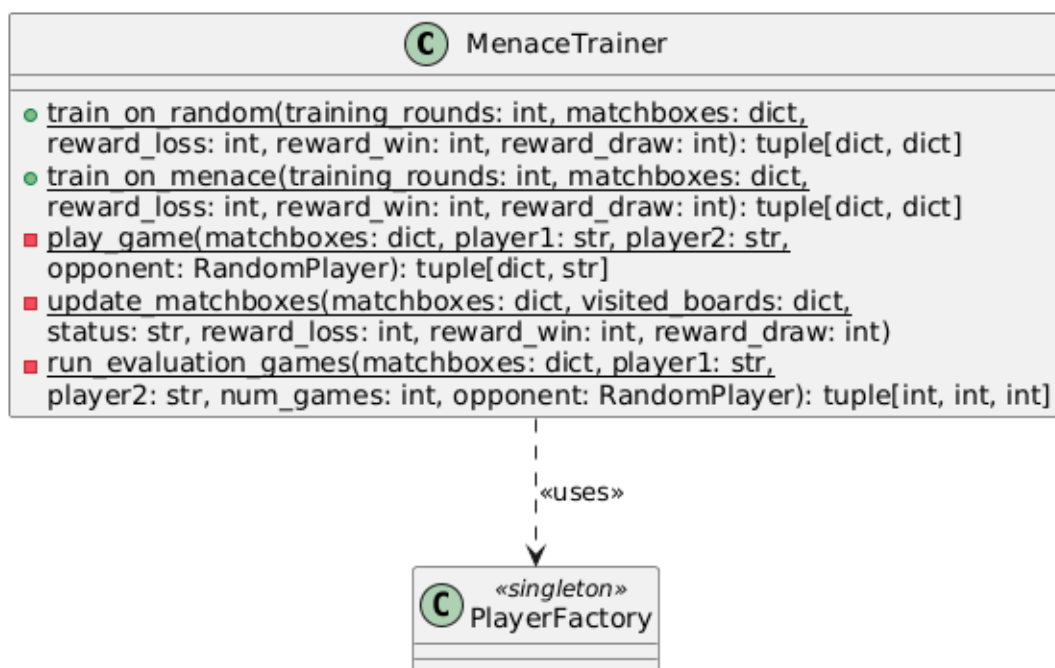
3.21. ábra. A BackpropTrainer osztály

GeneticTrainer A **GeneticTrainer** osztály a genetikus algoritmussal történő tanítást valósítja meg.



3.22. ábra. A GeneticTrainer modul

MenaceTrainer A `MenaceTrainer` osztály a MENACE játékos tanítását valósítja meg.



3.23. ábra. A MenaceTrainer osztály

Végpont függőségek, korlátozás

Dependency injection A Depends [41], a FastAPI beépített dependency injection mechanizmusa lehetővé teszi a szolgáltatások injektálását az API végpontokba. A Depends használatát az Annotated típusannotáció egyszerűbbé teszi [42]. Az alkalmazás a következő függőségeket injektálja:

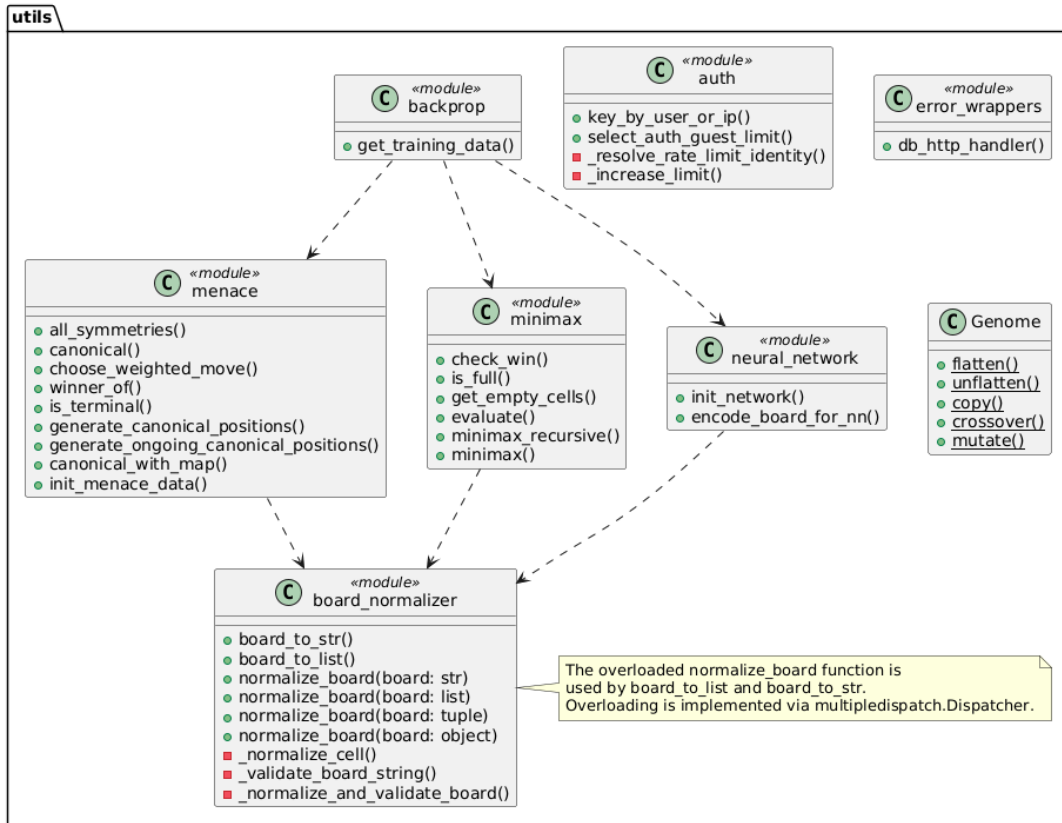
Függőség	Leírás
UserSessionDBDep	User és Session interfészt megvalósító adatbázis
NetworkDBDep	Network interfészt megvalósító adatbázis
ConnectionDBDep	Connection interfészt megvalósító adatbázis
AuthDep	Authenticator példány
PlayerFactoryDep	PlayerFactory példány
ConfDep	AppConfig példány
UserDep	Hitelesített felhasználó, access token alapján
GuestOrUserDep	Opcionális hitelesített felhasználó WebSocket kapcsolat esetén

3.2. táblázat. Injektált függőségek

Kérésszám korlátozás A végpontokra érkező kérekszám korlátozása a SlowApi [43] könyvtár segítségével valósul meg. A korlátozás ID vagy IP cím alapján történik.

Segédfüggvények

Az `app.utils` csomagban találhatóak a különböző segédfüggvények, amelyek a rendszer különböző részein használatosak.

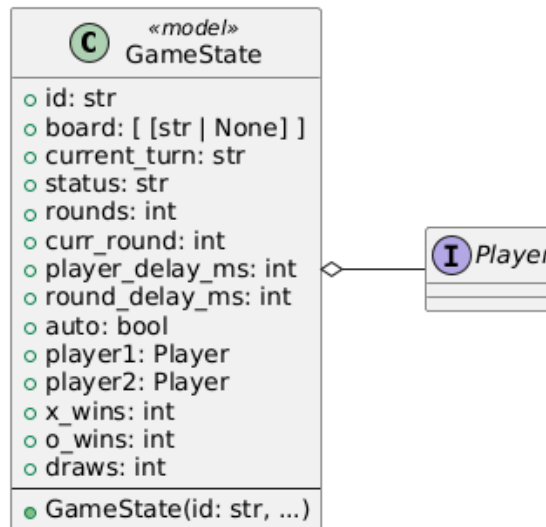


3.24. ábra. Az utils csomag

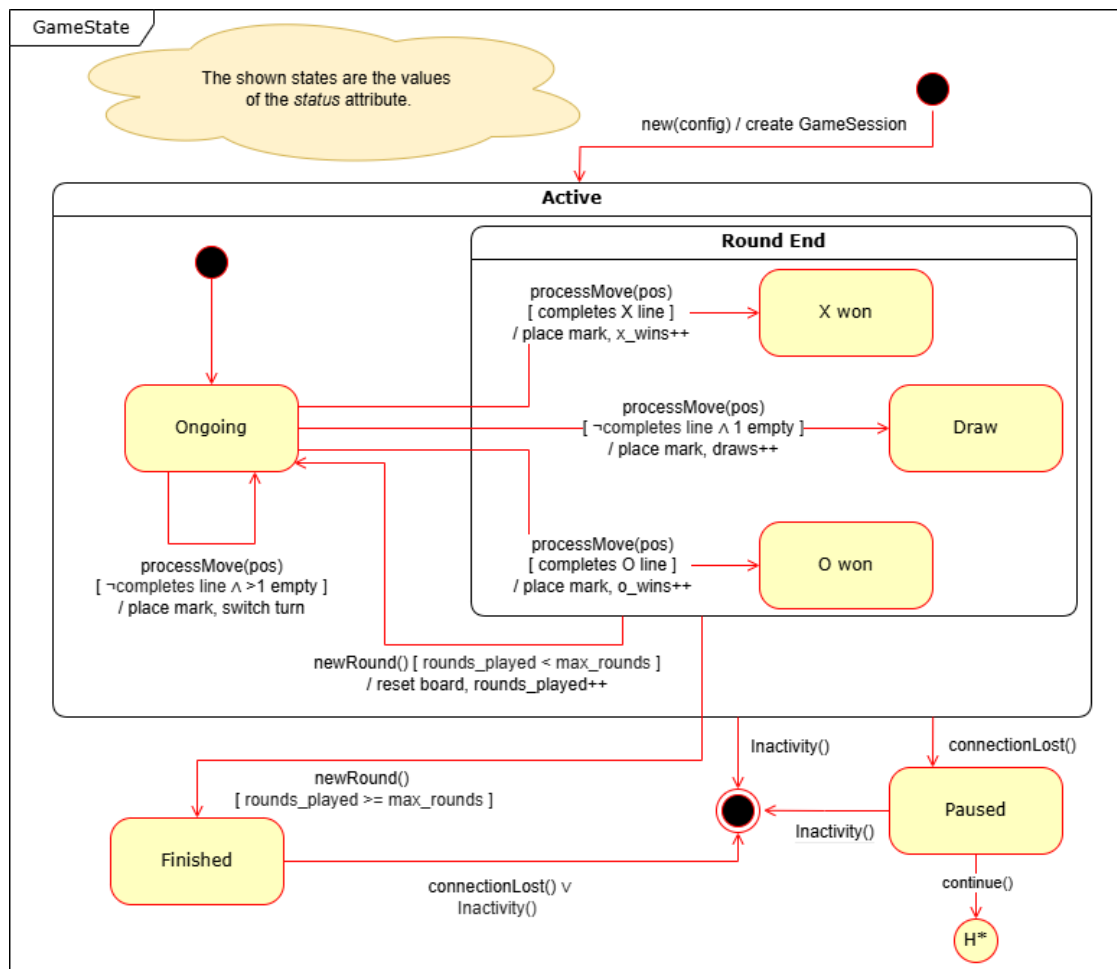
Játéklogika

Játékszabályok és állapot kezelés A játék a klasszikus Tic-Tac-Toe szabályait követi. A játéktábla egy 3×3 -as rács, ahol két játékos felváltva helyezi le a saját jelét (X vagy O). Az nyer, aki elsőként szerez három saját jelet egy sorban, oszlopban vagy átlóban. A játék döntetlennel ér véget, ha minden cella foglalt és nincs győztes.

A játék állapotát a `GameState` tároló osztály reprezentálja. A játék állapotát a `GameSessionManager` manipulálja.

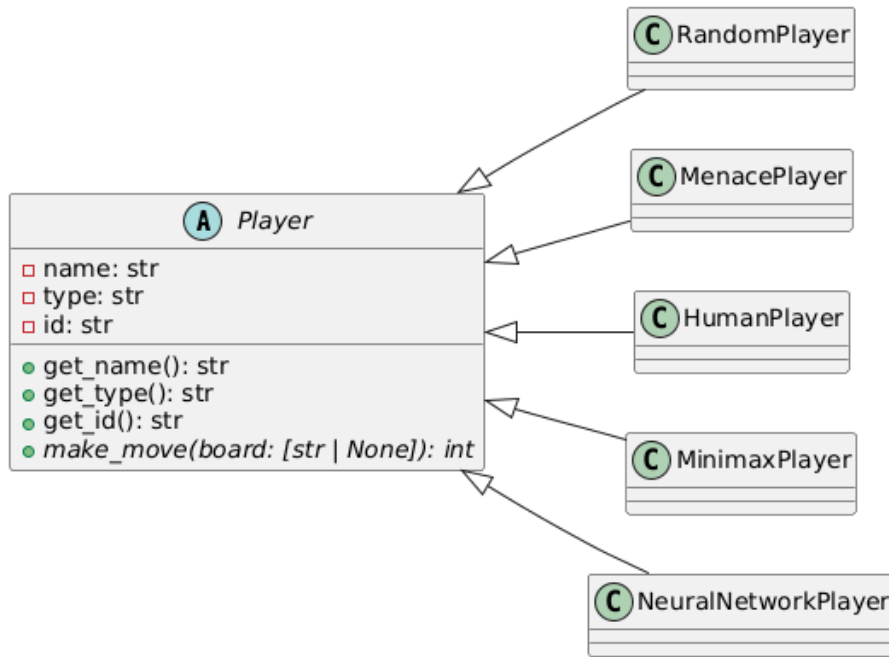


3.25. ábra. A GameState osztály



3.26. ábra. A játék állapotgépe

Játékosok Minden játékos típus a **Player** absztrakt osztálytól örököl.



3.27. ábra. A Player absztrakt osztály

Human Player A **HumanPlayer** csak adatokat tárol, a tényleges lépés a WebSocket üzenetből érkezik. A `get_move()` metódust nem hívja meg a rendszer.

Random Player A **RandomPlayer** véletlenszerűen választ a szabad cellák közül.

Minimax Player A **MinimaxPlayer** a Minimax algoritmust használja.

A teljesítményének növelése érdekében a `get_move()` metódusa külön folyamatot, illetve cache-t használ. A külön folyamat **ProcessPool**ban [44] futtatja a Minimax algoritmust.

MENACE Player A **MenacePlayer** a MENACE algoritmus eredményét használja. A `get_move()` metódus:

1. A táblát kanonikus formára alakítja.
2. Kiválasztja a megfelelő gyufásdobozt.
3. Súlyozott véletlenszerű választást végez.
4. A kanonikus lépést visszaalakítja az eredeti tábla koordinátáira.

Neural Network Player A `NeuralNetworkPlayer` egy tanított neuronhálót használ. A `get_move()` metódus:

1. A tábla állapotot numerikus vektorrá kódolja (18 hosszú).
2. A `NeuralNetwork` objektum `forward()` metódusát hívja.
3. Genetikusan tanított neuronháló esetén a kimeneti vektort szűri.
4. A legnagyobb valószínűségű lépést választja.

Algoritmusok

A fejezet a játékosok által, illetve tanításukhoz használt algoritmusokat mutatja be.

Megjegyzés. A bemutatott algoritmusok nem teljesen fedik le a megvalósítás részleteit, a főbb elvekre és lépésekre fókuszálnak.

MENACE (Matchbox Educable Noughts and Crosses Engine) A MENACE egy 1961-ben Donald Michie által kifejlesztett megerősítéses tanulási algoritmus, amely gyufásdobozok és színes gyöngyök segítségével tanul meg amőbázni. [45]

Gyufásdobozok (matchboxes) : Minden lehetséges játéálláshoz egy gyufásdoboz tartozik, amely a lehetséges lépésekhez rendelt gyöngyöket tárolja. A programban egy szótár reprezentálja a gyufásdobozokat, ahol a kulcs a tábla állapota kanonikus alakja, az érték pedig egy másik szótár, ahol a kulcsok a lehetséges lépések (0-8 index), az értékek pedig a gyöngyök száma.

Kanonikus forma : A Tic-Tac-Toe szimmetriáinak kihasználásával (4 forgatás és 2 tükrözés, azaz 8 transzformáció) minden állapot egy kanonikus reprezentációra van leképezve, így jelentősen csökken a tárolandó állapotok száma. Összesen 627 kanonikus, nem terminális állapot van: 338, ahol *X*, és 289, ahol *O* következik.

Döntéshozatal A MENACE súlyozott véletlenszerű választást alkalmaz:

1. A tábla kanonikus formára alakítása.
2. A megfelelő gyufásdoboz kiválasztása.

3. Súlyozott véletlen választás a gyöngyök száma alapján: $P(\text{lépés}_i) = \frac{w_i}{\sum_j w_j}$.
4. A kanonikus lépés visszaképzése az eredeti táblára.

Tanulási folyamat A MENACE játékokat játszik és az eredmény alapján büntetést vagy jutalmat kap:

- Győzelem: +3 gyöngy a megtett lépésekhez.
- Döntetlen: +1 gyöngy a megtett lépésekhez.
- Vereség: -1 gyöngy a megtett lépésekhez.

1. algoritmus MENACE tanító algoritmus

Függvény TrainMENACE(*trainingRounds*)

```

1: Inicializáljuk a gyufásdobozokat  $\mathcal{M}$  minden kanonikus játékálláshoz
2:  $\triangleright$  Minden dobozban gyöngyök az érvényes lépésekhez, kezdetben egyenlő számú
   gyöngy
3: for round = 1 to trainingRounds do
4:   Inicializáljuk az üres játék táblát  $B$ 
5:    $V \leftarrow \emptyset$   $\triangleright$  A játék során MENACE által látogatott állások és lépések
6:   currentPlayer  $\leftarrow X$ 
7:   while a játék nem ért véget do
8:     if currentPlayer = MENACE then
9:        $B_{canonical}, mapping \leftarrow \text{KanonikusForma}(B)$ 
10:       $matchbox \leftarrow \mathcal{M}[B_{canonical}]$ 
11:      canonicalMove  $\leftarrow$  Véletlenszerű gyöngy kiválasztás (matchbox)
12:      move  $\leftarrow mapping[canonicalMove]$   $\triangleright$  Lépés az eredeti táblán
13:       $V \leftarrow V \cup \{(B_{canonical}, canonicalMove)\}$ 
14:     else
15:       move  $\leftarrow$  Ellenfél választása ( $B$ )
16:     end if
17:      $B[move] \leftarrow currentPlayer$   $\triangleright$  Jelölés a táblán
18:     currentPlayer  $\leftarrow$  következő játékos
19:   end while
20:    $\triangleright$  Megerősítés: frissítjük a gyufásdobozokat a játék kimenetele alapján
21:   reward  $\leftarrow 0$ 
22:   if MENACE nyert then
23:     reward  $\leftarrow 3$ 
24:   else if Döntetlen then
25:     reward  $\leftarrow 1$ 
26:   else
27:     reward  $\leftarrow -1$ 
28:   end if
29:   for minden (board, move) a  $V$ -ben do
30:      $\mathcal{M}[board][move] \leftarrow \mathcal{M}[board][move] + reward$   $\triangleright$  A megtett lépést
       ábrázoló gyöngyök számának frissítése
31:      $\mathcal{M}[board][move] \leftarrow \max(\mathcal{M}[board][move], 1)$   $\triangleright$  Minimum 1 gyöngy
       marad a gyufásdobozban minden lépéshez
32:   end for
33: end for
34: return  $\mathcal{M}$   $\triangleright$  Tanítás utáni gyufásdobozok

```

A MenaceTrainer két fázisban tanít:

1. **Exploration fázis:** Véletlenszerű ellenfél ellen - alapvető stratégiák felismerése.
2. **Exploitation fázis:** Self-play két MENACE példány között - finomhangolás.

Minimax algoritmus A Minimax egy döntési szabály, miszerint azt a lépést kell választani, amely *minimalizálja* a *maximális* veszteséget. A Tic-Tac-Toe játék egy kétszemélyes, nulla összegű játék, ahol véges sok lépés létezik, így a Minimax algoritmus jól alkalmazható.

Alapelv A Minimax felépít egy játékfát, ahol minden belső csúcs egy nem terminális játékalapotot reprezentál, a levelek pedig a terminális állapotokat tartalmazzák. A csúcsok gyerekei a lehetséges lépéseket és azokkal létrejövő új állapotokat jelölik. A játékfát rekurzívan bejárja, feltételezve hogy mindkét játékos optimálisan játszik:

- **Maximalizáló szint:** A saját játékos a legnagyobb értékű lépést választja.
- **Minimalizáló szint:** Az ellenfél a legkisebb értékű lépést választja.

Értékelő függvény A terminális állapotokhoz a kimeneteltől függő (a gyökérben lévő játékos szemszögéből nézve) értéket rendel. Tic-Tac-Toe esetén az értékelés (amennyiben a mélység is számít) a következő:

- Győzelem: $10 - \text{mélység}$ (korábbi győzelem jobb)
- Vereség: $\text{mélység} - 10$ (későbbi vereség jobb)
- Döntetlen: 0

2. algoritmus Minimax rekurzív függvény

Függvény MinimaxRekurzív(*board*, *depth*, *maximizingPlayer*, *playerMarker*, *opponentMarker*)

```
1: score  $\leftarrow$  Értékelés(board, playerMarker, opponentMarker, depth)
2: if score  $\neq$  Null then                                 $\triangleright$  A játék véget ért, van eredmény
3:   return score
4: end if
5: if maximizingPlayer then
6:   bestScore  $\leftarrow -\infty$ 
7:   for minden lehetséges lépés move do
8:     board[move]  $\leftarrow$  playerMarker
9:     score  $\leftarrow$  MinimaxRekurzív(board, depth + 1, False, playerMarker,
      opponentMarker)
10:    board[move]  $\leftarrow$  üres
11:    bestScore  $\leftarrow$  max(bestScore, score)
12:   end for
13:   return bestScore
14: else
15:   bestScore  $\leftarrow \infty$ 
16:   for minden lehetséges lépés move do
17:     board[move]  $\leftarrow$  opponentMarker
18:     score  $\leftarrow$  MinimaxRekurzív(board, depth + 1, True, playerMarker,
      opponentMarker)
19:     board[move]  $\leftarrow$  üres
20:     bestScore  $\leftarrow$  min(bestScore, score)
21:   end for
22:   return bestScore
23: end if
```

Neuronhálók

Neuronháló architektúra Az alkalmazásban használt mesterséges neuronhálók teljesen összekötött, előrecsatolt MLP (Multi-Layer Perceptron) [46] architektúrán alapulnak. A hálók topológiája a következőképpen épül fel:

- **Bemeneti réteg:**
 - 18 neuron (1 táblamezőt 2 bemeneti neuron kódol: X jelenléte - O jelenléte).
- **Rejtett rétegek:** változó méretű teljesen összekötött rétegek, példák: [24, 11], [36, 18].
- **Kimeneti réteg:** 9 neuron, amelyek mindegyike egy lehetséges lépéshez tartozó értéket ad vissza.

Aktivációs függvények:

- **ReLU** (Rectified Linear Unit): $f(x) = \max(0, x)$ - rejtett rétegekben
- **Softmax**: $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ - kimeneti rétegben, valószínűségi eloszlást ad

Előrecsatolás

1. **Bemenet kódolás**: Tábla állapot $\rightarrow$ numerikus vektor
2. **Rétegenkénti számítás**: $z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)}$, ahol
 - $z^{(l)}$: l -edik réteg pre-aktivációs értékei
 - $W^{(l)}$: súlymátrix
 - $a^{(l-1)}$: előző réteg aktivációi
 - $b^{(l)}$: bias vektor
3. **Aktiváció**: $a^{(l)} = f(z^{(l)})$, ahol f a választott aktivációs függvény
4. **Kimenet**: A lépésekhez tartozó valószínűségi értékek.

Visszaterjesztés (Backpropagation) A visszaterjesztés egy tanítási algoritmus, amely gradienstörlesztés segítségével optimalizálja a súlyokat és torzításokat. [47]

3. algoritmus Viisszaterjesztéses tanítás algoritmus

Függvény $\text{TrainNetwork}(\text{NeuralNetwork}, \text{trainingData}, \text{epochs}, \eta)$

```

1:  $L \leftarrow$  rétegek száma( $\text{NeuralNetwork}$ )
2:  $W \leftarrow$  súlyok( $\text{NeuralNetwork}$ )
3:  $b \leftarrow$  torzítások( $\text{NeuralNetwork}$ )
4: for  $\text{epoch} = 1$  to  $\text{epochs}$  do
5:   for minden  $(x, y)$  tanítópár do
6:                                     ▷ Előreccsatolás
7:      $a^{(0)} \leftarrow x$ 
8:     for  $l = 1$  to  $L$  do
9:        $z^{(l)} \leftarrow W^{(l)}a^{(l-1)} + b^{(l)}$ 
10:       $a^{(l)} \leftarrow f(z^{(l)})$ 
11:    end for
12:                                     ▷ Viisszaterjesztés, rétegenkénti hibák
13:     $\delta^{(L)} \leftarrow \text{OutputError}(a^{(L)}, y)$ 
14:    for  $l = L - 1$  downto  $1$  do
15:       $\delta^{(l)} \leftarrow (W^{(l+1)})^T \delta^{(l+1)} \odot f'(z^{(l)})$ 
16:    end for
17:                                     ▷ Súly és torzítás gradiensek számítása
18:    for  $l = 1$  to  $L$  do
19:       $dW^{(l)} \leftarrow \delta^{(l)}(a^{(l-1)})^T$ 
20:       $db^{(l)} \leftarrow \delta^{(l)}$ 
21:    end for
22:                                     ▷ Súlyok és torzítások frissítése
23:    for  $l = 1$  to  $L$  do
24:       $W^{(l)} \leftarrow W^{(l)} - \eta dW^{(l)}$ 
25:       $b^{(l)} \leftarrow b^{(l)} - \eta db^{(l)}$ 
26:    end for
27:  end for
28: end for
29: return háló paraméterei  $(W, b)$ 

```

Evolúciós algoritmus és Genetikus algoritmus Az evolúciós algoritmus egyik típusa a Genetikus algoritmus, amely a lehetséges megoldásokat egyedekként kezeli, és az evolúciót szimulálva evolválja őket szelekció, keresztezés és mutáció révén. [48]

4. algoritmus Genetikus algoritmus

Függvény GeneticAlgorithm(*populationSize*, *generations*, *mutationRate*, *elitismCount*)

```
1:  $P \leftarrow \text{PopulációLétrehozása}(\text{populationSize})$  ▷ Inicializáció
2: for  $g = 1$  to  $\text{generations}$  do
3:   for minden egyed  $i$  a  $P$  populációban do
4:      $\text{fitness}(i) \leftarrow \text{Értékelés}(i)$ 
5:   end for
6:    $\text{Elites} \leftarrow \text{LegjobbEgyedek}(P, \text{elitismCount})$  ▷ Legjobb egyedek megőrzése
7:    $P_{\text{new}} \leftarrow \text{Elites}$ 
8:   while  $|P_{\text{new}}| < \text{populationSize}$  do ▷ Új populáció feltöltése
9:      $\text{parent1}, \text{parent2} \leftarrow \text{Szelekció}(P)$  ▷ Szülők kiválasztása
10:     $\text{child1}, \text{child2} \leftarrow \text{Keresztezés}(\text{parent1}, \text{parent2})$ 
11:     $\text{child1} \leftarrow \text{Mutáció}(\text{child1}, \text{mutationRate})$ 
12:     $\text{child2} \leftarrow \text{Mutáció}(\text{child2}, \text{mutationRate})$ 
13:     $P_{\text{new}} \leftarrow P_{\text{new}} \cup \{\text{child1}, \text{child2}\}$ 
14:   end while
15:    $P \leftarrow P_{\text{new}}$ 
16: end for
17: return LegjobbEgyed( $P$ )
```

Megjegyzés. A fenti algoritmus sokféle képpen alakítható, például szigetek bevezetésével, migrációval, kihalással, adaptív mutációs rátával, vagy korai terminálással.

3.2.3. Adatelérési réteg

Kezelő osztályok és interfészek

Config interfész és implementáció A konfigurációs interfész (`DatabaseConfig`) definiálja az adatbázis-kapcsolat létrehozásához szükséges alapvető paramétereket. Ennek megvalósítása a `PostgreSQLConfig`, amely a környezeti változókból (a `.env` fájlból) tölti be a kapcsolat adatait és paramétereit.

DAO interfészek Az üzleti logikához kapcsolódó adatműveletek entitásonként külön Data Access Object (DAO) interfészekbe vannak szervezve. Ezek az interfészek (`EvolutionDao`, `NetworkDao`, `SessionDao`, `MenaceDao`, `UserDao`) CRUD műveleteket definiálnak.

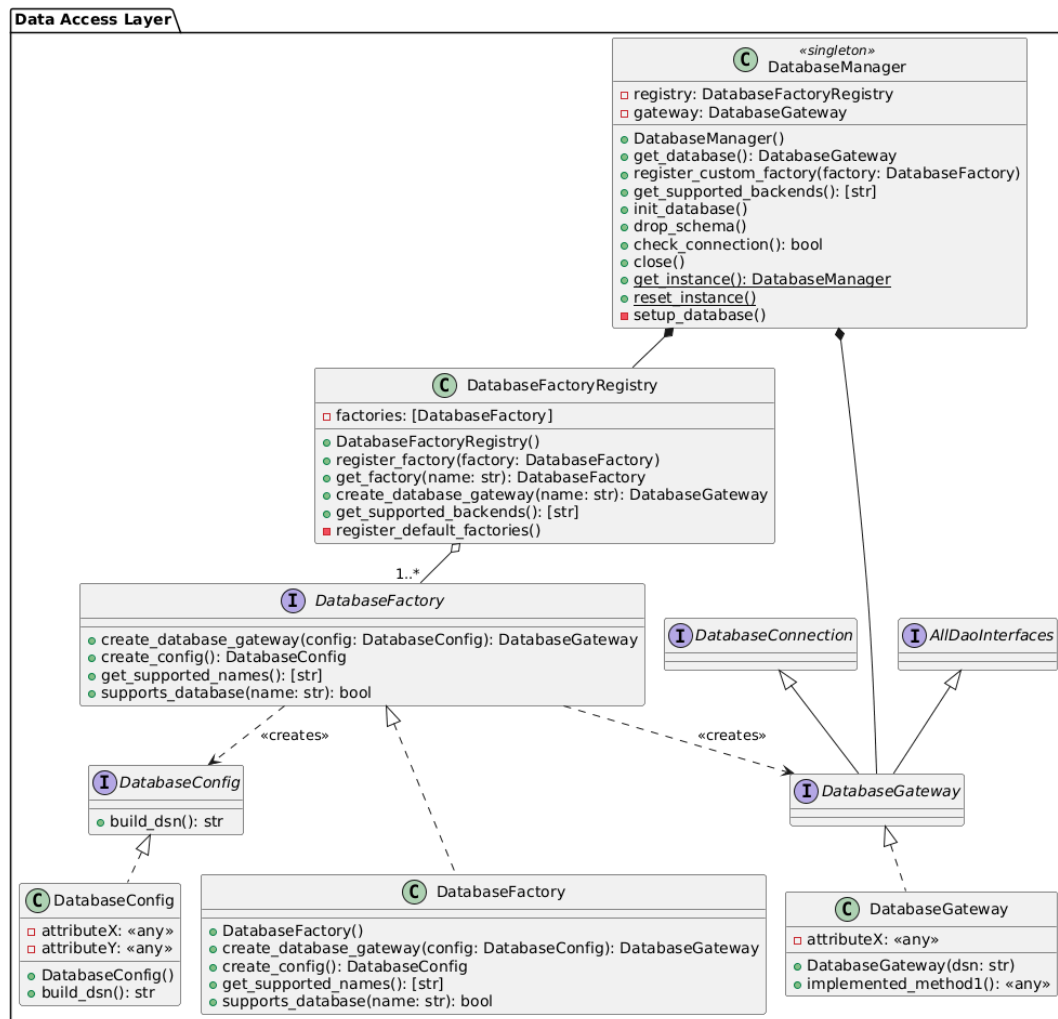
Gateway interfész A `DatabaseGateway` egy összetett interfész, amely egyesíti a DAO-kat és az adatbázis életciklusát kezelő `DatabaseConnection` interfészt. Ez

az interfész szolgál az adatelérési réteg szerződéseként, amelyet az alkalmazás üzleti logikája használ.

Factory és Registry A **Factory** felelős a konfiguráció betöltéséért és a gateway példányosításáért. A gyárat a **DatabaseFactoryRegistry** tartja nyilván, lehetővé téve a különböző adatbázis-backendek közötti egyszerű váltást.

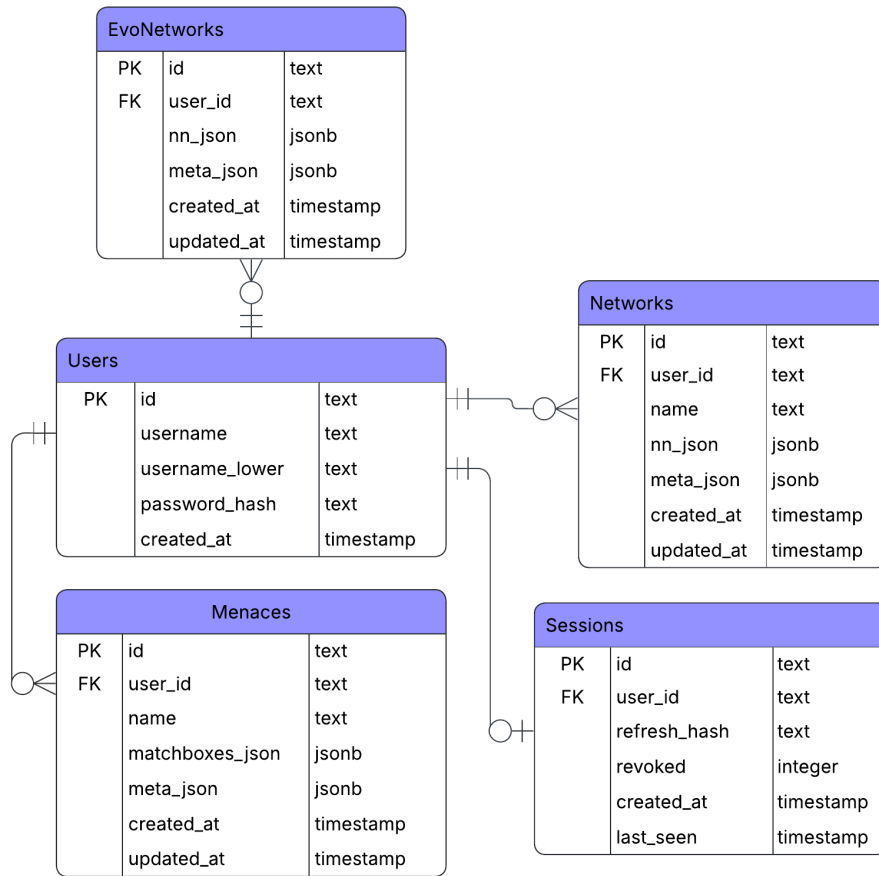
Database manager Ez a singleton osztály adattagként tartalmazza a **DatabaseFactoryRegistry**-t. Feladata az adatelérési réteg fő belépési pontjaként szolgálni. Az alkalmazás indulásakor létrehozza a **DatabaseGateway** implementációját, és gondoskodik annak eléréséről a működés során.

Megjegyzés. Új adatbázis-kezelő rendszer esetén, új config, gateway, és factory implementációk létrehozása, valamint a factory regisztrálása szükséges. Ezután a megfelelő környezeti változó állításával az alkalmazás az új adatbázis-kezelő rendszert használja.



3.28. ábra. Adatelérést kezelő osztályok

Adatbázis



3.29. ábra. ER diagram

Adatbázis táblák Az alkalmazás öt táblát használ az adatok perzisztens tárolására.

- **Users:** Regisztrált felhasználók adatai.
- **Sessions:** Felhasználói munkamenetek adatai.
- **Menaces:** MENACE játékosok adatai.
- **Networks:** Visszaterjesztéssel tanított neurális hálózatok adatai.
- **EvoNetworks:** Genetikus algoritmussal tanított neurális hálózatok adatai.

Az alkalmazás PostgreSQL adatbázist használ, másik adatbázis-kezelő rendszer használata esetén az oszlopok típusa eltérhet, ilyen esetben is garantálni kell, hogy a Gateway felől az üzleti logika által elvárt típusok érkezzenek.

Adatmezők	Elvárt Python típus
User	
id, username, password_hash, username_lower	str
created_at	datetime
Network	
id, user_id, name	str
nn_json	dict
meta_json	dict None
created_at, updated_at	datetime
Session	
id, user_id, refresh_hash	str
created_at, last_seen	datetime
Menace	
id, user_id, name	str
matchboxes_json	dict
meta_json	dict None
created_at, updated_at	datetime
EvoNetwork	
id, user_id	str
nn_json	dict
meta_json	dict None
created_at, updated_at	datetime

3.3. táblázat. Az adatbázis entitások mezőinek elvárt Python típusai

Indexek Az egyedi megszorítással és elsődleges kulccsal (PK) ellátott oszlopok esetén a használt adatbázis-kezelő rendszer automatikusan létrehozza a szükséges indexeket. A lekérdezések teljesítményének javítása érdekében a következő oszlopokra manuálisan lettek indexek létrehozva: Sessions.user_id, Menaces.user_id, Networks.user_id, EvoNetworks.user_id.

3.3. Tesztelés

3.3.1. Automatikus

Egység- és integrációstesztetek

Az alkalmazás kisebb részeinek, moduljainak, illetve ezek integrációjának tesztelése a `vitest` [49] és a `pytest` [50] eszközök segítségével történik.

208 passed 272 passed

(a) Frontend
tesztetek

(b) Backend
tesztetek

3.30. ábra. Automatikus tesztek

Funkcionális rendszertesztek

Az alábbi tesztek a determinisztikusan tesztelhető felhasználói történeteket fedik le. A tesztelés a teljes rendszer működése közben, a `Playwright` [51] E2E tesztelő eszköz segítségével valósult meg.

ID	Eset	Eredmény		
		Chromium	Firefox	WebKit
1.1	Regisztráció helyesen	✓	✓	✓
1.2	Regisztráció helytelenül - eltérő jelszavak	✓	✓	✓
1.2	Regisztráció helytelenül - rövid jelszó	✓	✓	✓
1.2	Regisztráció helytelenül - nagybetű nélküli jelszó	✓	✓	✓
1.2	Regisztráció helytelenül - kisbetű nélküli jelszó	✓	✓	✓
1.2	Regisztráció helytelenül - szám nélküli jelszó	✓	✓	✓
1.2	Regisztráció helytelenül - túl hosszú jelszó	✓	✓	✓
1.2	Regisztráció helytelenül - nem egyedi felhasználónév	✓	✓	✓
1.2	Regisztráció helytelenül - túl rövid felhasználónév	✓	✓	✓

ID	Eset	Eredmény		
		Chromium	Firefox	WebKit
1.2	Regisztráció helytelenül - túl hosszú felhasználónév	✓	✓	✓
1.2	Regisztráció helytelenül - érvénytelen karakter (*) felhasználónévben	✓	✓	✓
2.1	Bejelentkezés helyesen	✓	✓	✓
2.2	Bejelentkezés helytelenül - helytelen jelszó	✓	✓	✓
2.2	Bejelentkezés helytelenül - nem létező felhasználónév	✓	✓	✓
3	Kijelentkezés	✓	✓	✓
4.1.1	Ellenfelek kiválasztása	✓	✓	✓
4.1.2	Lépési várakozás megadása - túl nagy	✓	✓	✓
4.1.2	Lépési várakozás megadása - helyesen	✓	✓	✓
4.1.3	Körök számának megadása - túl nagy	✓	✓	✓
4.1.3	Körök számának megadása - helyesen	✓	✓	✓
4.1.4	Kör közti várakozás megadása - túl nagy	✓	✓	✓
4.1.4	Kör közti várakozás megadása - helyesen	✓	✓	✓
4.2	Játék indítása	✓	✓	✓
4.3.1	Lépés a játékban, helyesen	✓	✓	✓
4.3.2	Lépés a játékban, nem következik	✓	✓	✓
5.1	Algoritmus működésének megtekintése - Minimax ²	✓	✓	✓
5.1	Algoritmus működésének megtekintése - Menace ²	✓	✓	✓
5.1	Algoritmus működésének megtekintése - Random ²	✓	✓	✓
5.1	Algoritmus működésének megtekintése - Neuronháló ²	✓	✓	✓
5.1	Algoritmus működésének megtekintése - Genetikus algoritmus ²	✓	✓	✓

ID	Eset	Eredmény		
		Chromium	Firefox	WebKit
5.1	Algoritmus működésének megtekintése - Visszaterjesztés ²	✓	✓	✓
5.2	Algoritmus animáció megtekintése - Random ³	✓	✓	✓
5.2	Algoritmus animáció megtekintése - Menace ³	✓	✓	✓
5.2	Algoritmus animáció megtekintése - Minimax ³	✓	✓	✓
5.2	Algoritmus animáció megtekintése - Visszaterjesztés ³	✓	✓	✓
6.1	Saját neuronháló létrehozása	✓	✓	✓
6.2.1	Saját neuronháló szerkesztése	✓	✓	✓
6.2.2	Saját neuronháló szerk. - új réteg hozzáadása	✓	✓	✓
6.2.2	Saját neuronháló szerk. - réteg törlése	✓	✓	✓
6.2.3	Saját neuronháló szerk. - neuron paraméterek	✓	✓	✓
6.2.4	Saját neuronháló szerk. - név állítása	✓	✓	✓
6.3	Saját neuronháló mentése	✓	✓	✓
6.4	Saját neuronháló megtekintése	✓	✓	✓
6.5	Saját neuronháló törlése	✓	✓	✓
6.6.1	Saját neuronháló tanítása, paraméterek megadása	✓	✓	✓
6.6.2	Saját neuronháló tanítása, tanítás indítása	✓	✓	✓
6.6.3	Saját neuronháló tanításakor progresszió megjelenítése	✓	✓	✓

3.4. táblázat. Automatizált funkcionális tesztek

²A téma címe megjelenik³Az animáció egy adott eleme megjelenik

Nem funkcionális rendszertesztek

A program válaszidejét és stabilitását nagyobb terhelés, illetve hosszabb időtartam alatt is teszteltem. Ehhez a `k6` [52] eszközt használtam.

Terhelési teszt Egy 5 perces terhelési teszt során intenzív terhelést szimuláltam, melynek részei:

- felhasználói bejelentkezések
- CRUD műveletek
- Neurális háló tanítás
- Tic-Tac-Toe játék

```
db_failures.....: 0      0/s
db_requests_count.....: 24552 81.370268/s
login_failures.....: 0      0/s
login_successes.....: 1525  5.054157/s
train_attempts.....: 36     0.119311/s
train_started.....: 36     0.119311/s
train_status_done.....: 36     0.119311/s
```

(a) HTTP-kérések száma

```
http_req_duration.....: avg=188.39ms min=4ms    med=178.07ms max=734.27ms p(90)=281.58ms p(95)=397.68ms
{ expected_response:true }...: avg=188.39ms min=4ms    med=178.07ms max=734.27ms p(90)=281.58ms p(95)=397.68ms
{ type:ai_jobs }.....: avg=170.67ms min=4ms    med=176.71ms max=441.93ms p(90)=258.29ms p(95)=281.72ms
{ type:auth }.....: avg=451.91ms min=217.74ms med=463.49ms max=734.27ms p(90)=543.87ms p(95)=565.2ms
{ type:database }.....: avg=172.23ms min=4ms    med=172.82ms max=531.75ms p(90)=254.74ms p(95)=281.37ms
http_req_failed.....: 0.00% 0 out of 26370
http_reqs.....: 26370 87.395486/s
```

(b) HTTP-kérések válaszideje

```
ws_closed_by_game_over.....: 7310 24.226811/s
ws_connecting.....: avg=114.95ms min=1.5ms    med=116.68ms max=386.7ms p(90)=161.75ms p(95)=183.74ms
ws_game_duration_ms.....: avg=1.53s min=1.16s    med=1.52s max=2.2s p(90)=1.67s p(95)=1.74s
ws_games_done.....: 7310 24.226811/s
ws_games_started.....: 7310 24.226811/s
ws_msgs_received.....: 652427 2162.274356/s
ws_msgs_sent.....: 7310 24.226811/s
ws_session_duration.....: avg=1.65s min=1.23s    med=1.64s max=2.37s p(90)=1.79s p(95)=1.87s
ws_sessions.....: 7310 24.226811/s
```

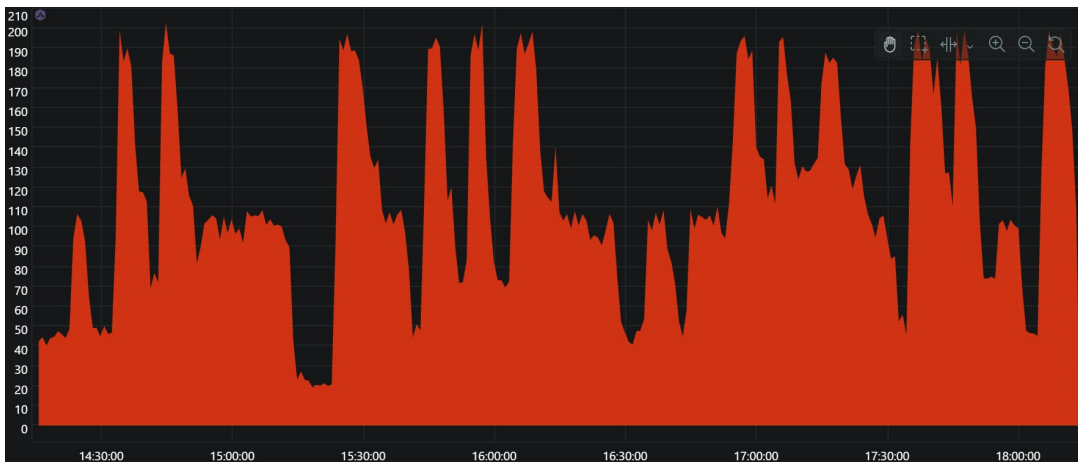
(c) WebSocket statisztikák

3.31. ábra. Terhelési teszt eredményei

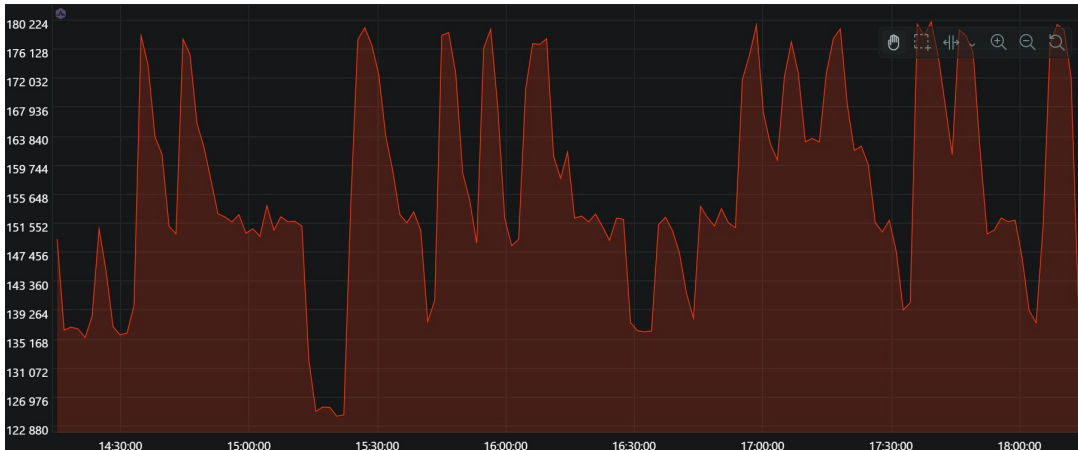
Soak teszt Ezen teszt során a program memóriahasználatát és hosszabb távú stabilitását vizsgáltam.

- 4 órás teszt

- 200 000 HTTP-kérés
- 1600 tanítási folyamat (párhuzamosan legfeljebb 2)
- 19 000 WebSocket alapú Tic-Tac-Toe játék
 - átlagosan 21 játék futott egyszerre
 - 3700 befejezett játék - automatikusan törlődik a memóriából
 - 15 300 megszakított játék - periodikus tisztítás esetén törlődik a memóriából



(a) CPU-használat



(b) Memória-használat

3.32. ábra. Soak teszt eredményei

Az eredmények alapján a program teljesítménye és erőforrásigénye nem növekszik az idő függvényében, nem tapasztalható memóriaszivárgás.

3.3.2. Manuális

A tanítás eredményességét manuális teszttel ellenőriztem, ehhez neuronhálót hoztam létre és tanítottam. Végül különböző ellenfelek ellen játszottam, illetve megnéztem az esetek hány százalékban találja meg a tökéletes lépéseket. Az alábbi táblázat a nem automatizált (vagy további tesztelést igénylő) felhasználói történeteket tartalmazza, melyek manuális tesztelése Google Chrome-ban történt.

ID	Eset	Eredmény
5.1	Algoritmus működésének megtekintése	✓
5.2	Algoritmus animáció megtekintése	✓

3.5. táblázat. Manuális funkcionális tesztek

3.4. Futtatás lokális környezetben

3.4.1. Forráskód letöltése

A forráskódot a <https://github.com/mysteryMrE/szakdolgozat> helyen található tárolóból lehet letölteni. Ez az alábbi módokon történhet:

- Git használata esetén:

```
1 git clone https://github.com/mysteryMrE/szakdolgozat
```

- Zip fájl letöltése esetén:

```
1 powershell -command "Expand-Archive -Path 'C:\Path\To\Downloads\  
downloaded.zip' -DestinationPath 'C:\Path\To\Extracted'"
```

3.4.2. Alkalmazás futtatása

A forráskód rendelkezésre állása esetén az alkalmazás futtatása két módon történhet: Docker-rel vagy manuális telepítéssel. Az alábbiakban Windows operációs rendszerre vonatkozó lépések találhatók, más operációs rendszerek esetén a parancsok eltérhetnek.

Helyi futtatás Docker-rel

Előfeltételek:

- Docker Desktop 4.43.1+

Lépések:

1. Környezeti változók beállítása:

```
1 cd szakdolgozat
2 copy .env.docker.example .env
```

A .env fájl a kommentek alapján testreszabható.

2. Docker konténerek indítása:

```
1 docker compose up -d
```

Ez elindítja a három fő szolgáltatást:

- PostgreSQL adatbázis (alapértelmezetten a 5433-as porton)
- Backend (alapértelmezetten a 8000-es porton)
- Frontend (alapértelmezetten a 5173-as porton)

3. Alkalmazás elérése:

- A frontend a böngészőben a `http://localhost:5173` címen érhető el.
- API dokumentáció a `http://localhost:8000/docs` címen érhető el.

4. Konténerek leállítása:

```
1 docker compose down
```

Első futtatáskor a Dockernek hosszabb időre is szüksége lehet, mivel le kell töltenie, és fel kell építenie a szükséges képeket és létre kell hoznia a konténereket.

Helyi futtatás manuálisan

Előfeltételek:

- Python 3.12+ (tesztelve a 3.12.6 verzióval)
- Node.js 22+
- PostgreSQL 16+

Lépések:

1. Környezeti változók beállítása:

```
1 cd szakdolgozat/backend & copy .env.example .env
2 cd szakdolgozat/frontend & copy .env.example .env
```

A .env fájlok a kommentek alapján testreszabhatók.

2. Backend telepítése:

```
1 cd szakdolgozat/backend
2 py -3.12 -m venv venv
3 venv\Scripts\activate
4 pip install -r requirements.txt
```

3. Frontend telepítése:

```
1 cd szakdolgozat/frontend
2 npm ci
```

4. Adatbázis beállítása:

(a) PostgreSQL mappa megnyitása:

```
1 cd "C:\Program Files\PostgreSQL\16\bin"
```

A számnak a telepített PostgreSQL verziószámát kell jelölnie.

(b) Bejelentkezés superfelhasználóként:

```
1 psql -U postgres
```

A fenti parancs után a telepítéskor megadott jelszót kell megadni.

(c) Felhasználó létrehozása:

```
1 CREATE USER myuser WITH PASSWORD 'abc123';
```

(d) Adatbázis létrehozása:

```
1 CREATE DATABASE mydatabase OWNER myuser;
```

(e) Kilépés a PostgreSQL parancssorból:

```
1 \q
```

5. Szolgáltatások indítása:

- **Backend indítása:**

```
1 cd szakdolgozat/backend
2 venv\Scripts\activate
3 python -m app.main
```

- **Frontend indítása:**

```
1 cd szakdolgozat/frontend
2 npm run dev (vagy) npm run build & npm run preview
```

6. Alkalmazás elérése:

- A frontend a böngészőben a `http://localhost:5173` (vagy) a `http://localhost:4173` címen érhető el.
- API dokumentáció a `http://localhost:8000/docs` címen érhető el.

Első futtatás

Az alkalmazás első indításakor a következő lépések automatikusan végrehajtásra kerülnek:

1. **Adatbázis inicializálása:** Az adatbázis tábláinak létrehozása.
2. **Admin felhasználó:** Egy adminisztrátori fiók mentése.
3. **Alapértelmezett ellenfelek:** Az alapértelmezett ellenfelek tanítása és mentése.

4. fejezet

Összegzés

A létrehozott program teljesíti a kitűzött célokat, a megvalósított funkciók megfelelnek a követelményeknek. A tesztek eredményei alapján a program nem csupán egyfelhasználós, lokális környezetben működik megbízhatóan, hanem több felhasználó egyidejű használata mellett is. A felhasználói élményt az alacsony válaszidő mellett a reszponzív és modern felhasználói felület is növeli.

A fejlesztés során a legtöbb új ismeretet a webes technológiák területén szereztem, ugyanakkor a tananyag elkészítéséhez a gépi tanulás területén is elmélyültem. A megfelelő teljesítmény biztosításához elengedhetetlen volt a konkurens és aszinkron végrehajtási modellek megértése és helyes alkalmazása. A neurális hálózat tanításának megvalósítása különösen nagy kihívást jelentett a számításigényes műveletek és a Python GIL[37] kombinációja miatt. Biztonsági szempontból a JWT tokenekről, a tokenforgatásról, a hashelésről, a middleware-ekről és a forgalomkorlátozásról szereztem új ismereteket. Számos kódrész többszöri átdolgozáson esett át az átláthatóság és a karbantarthatóság javítása érdekében. Ez hasznos tapasztalatokat és új szemléletmódot adott.

4.1. További fejlesztési lehetőségek

A backend jelenlegi formájában single-worker módban működik, ami korlátozza az egyidejűleg kiszolgálható kérések számát. A jövőben érdemes megvizsgálni a multi-worker működés lehetőségét. Ehhez például a *manager* osztályokban alkalmazott, állapotkezelésre használt szótárak helyett mindenképpen egy gyors, külső adattároló alkalmazása szükséges. Ilyen lehet például a Redis[54].

Indokolt egy adminisztrátori felület kialakítása is, amely lehetővé teszi a tanítási folyamatok, a tanított ellenfelek és a felhasználók hatékony kezelését. A tanítási folyamat kiegészíthető megszakítási lehetőséggel, amelyhez mind a felhasználó, mind az adminisztrátor hozzáférhet.

További kapcsolódó témák feldolgozása, valamint a meglévő tartalmak részletesebb bemutatása szintén hasznos fejlesztési irányt jelenthet a jövőben.

Irodalomjegyzék

- [1] *Az alkalmazás webes felülete.* URL: <https://tic-tac-toe-algo-client-42f52d27345e.herokuapp.com> (elérés dátuma 2026. 05. 01.).
- [2] *MENACE - Wikipedia.* URL: https://en.wikipedia.org/wiki/Matchbox_Educable_Noughts_and_Crosses_Engine (elérés dátuma 2026. 03. 20.).
- [3] *Minimax Algorithm - GeeksforGeeks.* URL: <https://www.geeksforgeeks.org/artificial-intelligence/mini-max-algorithm-in-artificial-intelligence/> (elérés dátuma 2026. 03. 20.).
- [4] *Multilayer Perceptron - GeeksforGeeks.* URL: <https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/> (elérés dátuma 2026. 03. 20.).
- [5] *Genetic Algorithm - GeeksforGeeks.* URL: <https://www.geeksforgeeks.org/dsa/genetic-algorithms/> (elérés dátuma 2026. 03. 20.).
- [6] *Multilayer Perceptron and Backpropagation.* URL: <https://medium.com/data-science/multilayer-perceptron-explained-a-visual-guide-with-mini-2d-dataset-0ae8100c5d1c> (elérés dátuma 2026. 03. 20.).
- [7] *Backpropagation - mathematical details.* URL: <https://www.youtube.com/watch?v=znqbtL0fRA0> (elérés dátuma 2026. 03. 20.).
- [8] *Visual Studio Code.* URL: <https://code.visualstudio.com/> (elérés dátuma 2025. 11. 10.).
- [9] *pgAdmin 4.* URL: <https://www.pgadmin.org/> (elérés dátuma 2025. 11. 10.).
- [10] *Git.* URL: <https://git-scm.com/> (elérés dátuma 2025. 11. 10.).
- [11] *Docker.* URL: <https://www.docker.com/> (elérés dátuma 2025. 11. 10.).
- [12] *React.* URL: <https://react.dev/> (elérés dátuma 2025. 11. 10.).

- [13] *TypeScript*. URL: <https://www.typescriptlang.org/> (elérés dátuma 2025. 11. 10.).
- [14] *Tailwind CSS*. URL: <https://tailwindcss.com/> (elérés dátuma 2025. 11. 10.).
- [15] *React Router*. URL: <https://reactrouter.com/start/declarative/installation/> (elérés dátuma 2025. 11. 10.).
- [16] *JSON Web Tokens*. URL: <https://jwt.io/> (elérés dátuma 2025. 11. 15.).
- [17] *REST API Design*. URL: <https://restfulapi.net/> (elérés dátuma 2025. 11. 20.).
- [18] *Axios*. URL: <https://axios-http.com/docs/intro/> (elérés dátuma 2025. 11. 10.).
- [19] *WebSocket in Web*. URL: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket> (elérés dátuma 2025. 11. 14.).
- [20] *Server-Sent Events in Web*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events (elérés dátuma 2025. 11. 15.).
- [21] *MathJax*. URL: <https://www.mathjax.org/> (elérés dátuma 2025. 11. 13.).
- [22] *better-react-mathjax*. URL: <https://www.npmjs.com/package/better-react-mathjax> (elérés dátuma 2025. 11. 13.).
- [23] *Canvas API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API (elérés dátuma 2025. 11. 13.).
- [24] *Motion (előzőleg Framer Motion)*. URL: <https://motion.dev/react/> (elérés dátuma 2025. 11. 14.).
- [25] *FastAPI*. URL: <https://fastapi.tiangolo.com/> (elérés dátuma 2025. 11. 14.).
- [26] *Uvicorn*. URL: <https://www.uvicorn.org/> (elérés dátuma 2025. 11. 14.).
- [27] *WebSocket in FastAPI*. URL: <https://fastapi.tiangolo.com/advanced/websockets/> (elérés dátuma 2025. 11. 20.).
- [28] *Server-Sent Events with StreamingResponse in FastAPI*. URL: <https://fastapi.tiangolo.com/advanced/custom-response/#streamingresponse> (elérés dátuma 2025. 11. 20.).

- [29] *Trusted Host Middleware in FastAPI*. URL: <https://fastapi.tiangolo.com/advanced/middleware/#trustedhostmiddleware> (elérés dátuma 2026. 03. 05.).
- [30] *Proxy Headers Middleware in FastAPI*. URL: <https://starlette.dev/middleware/#proxyheadersmiddleware> (elérés dátuma 2026. 03. 05.).
- [31] *CORS (Cross-Origin Resource Sharing) in FastAPI*. URL: <https://fastapi.tiangolo.com/tutorial/cors/> (elérés dátuma 2025. 11. 20.).
- [32] *Path Operation Configuration with APIRouter in FastAPI*. URL: <https://fastapi.tiangolo.com/reference/apirouter/> (elérés dátuma 2025. 11. 20.).
- [33] *Server-Sent Events in FastAPI*. URL: <https://fastapi.tiangolo.com/tutorial/server-sent-events/> (elérés dátuma 2025. 11. 20.).
- [34] *Generators in Python*. URL: <https://docs.python.org/3.12/howto/functional.html#generators> (elérés dátuma 2025. 11. 25.).
- [35] *Thread-based parallelism in Python*. URL: <https://docs.python.org/3.12/library/threading.html> (elérés dátuma 2025. 11. 25.).
- [36] *Asyncio and Threads in Python*. URL: <https://docs.python.org/3.12/library/asyncio-task.html#running-in-threads> (elérés dátuma 2025. 11. 25.).
- [37] *Global Interpreter Lock in Python*. URL: <https://docs.python.org/3.12/glossary.html#term-global-interpreter-lock> (elérés dátuma 2025. 11. 25.).
- [38] *multiprocessing module in Python*. URL: <https://docs.python.org/3.12/library/multiprocessing.html> (elérés dátuma 2026. 03. 06.).
- [39] *multiprocessing.Pipe in Python*. URL: <https://docs.python.org/3.12/library/multiprocessing.html#multiprocessing.Pipe> (elérés dátuma 2025. 11. 25.).
- [40] *asyncio.Queue in Python*. URL: <https://docs.python.org/3.12/library/asyncio-queue.html#asyncio.Queue> (elérés dátuma 2026. 03. 06.).
- [41] *Dependency Injection with Depends in FastAPI*. URL: <https://fastapi.tiangolo.com/tutorial/dependencies/> (elérés dátuma 2025. 11. 20.).

- [42] *Using Annotated for Dependency Injection in FastAPI*. URL: <https://fastapi.tiangolo.com/tutorial/dependencies/#share-annotated-dependencies> (elérés dátuma 2026. 03. 07.).
- [43] *SlowApi - Rate Limiting*. (Elérés dátuma 2026. 03. 07.).
- [44] *concurrent.futures.ProcessPoolExecutor in Python*. URL: <https://docs.python.org/3.12/library/concurrent.futures.html> (elérés dátuma 2025. 11. 25.).
- [45] *MENACE - Matchbox Educable Noughts and Crosses Engine*. URL: <https://en.wikipedia.org/wiki/MENACE> (elérés dátuma 2025. 12. 01.).
- [46] *Multilayer perceptron*. URL: https://en.wikipedia.org/wiki/Multilayer_perceptron (elérés dátuma 2025. 12. 01.).
- [47] *Backpropagation*. URL: <https://en.wikipedia.org/wiki/Backpropagation> (elérés dátuma 2025. 12. 01.).
- [48] *Evolutionary algorithm*. URL: https://en.wikipedia.org/wiki/Evolutionary_algorithm (elérés dátuma 2025. 12. 01.).
- [49] *Vitest*. URL: <https://vitest.dev/> (elérés dátuma 2025. 12. 05.).
- [50] *pytest*. URL: <https://docs.pytest.org/en/stable/> (elérés dátuma 2025. 12. 05.).
- [51] *Playwright*. URL: <https://playwright.dev/docs/intro#introduction> (elérés dátuma 2026. 03. 15.).
- [52] *k6 - Load Testing Tool*. URL: <https://grafana.com/docs/k6/latest/javascript-api/> (elérés dátuma 2026. 03. 15.).
- [53] *Alkalmazás depó*. URL: <https://github.com/mysteryMrE/szakedolgozat> (elérés dátuma 2026. 05. 01.).
- [54] *Redis*. URL: <https://redis.io/docs/latest/develop/clients/redis-py/> (elérés dátuma 2026. 03. 15.).

Ábrák jegyzéke

2.1. Navigációs menü vendég felhasználó számára	7
2.2. Navigációs menü bejelentkezett felhasználó számára	7
2.3. Hamburger menü	7
2.4. Navigációs gomb regisztrációhoz	8
2.5. Legördülő menü	8
2.6. Legördülő menü kinyitva	8
2.7. Az alkalmazás lapjai és navigációs útvonalai	9
2.8. Az alkalmazás kezdőlapja	10
2.9. „Csatlakozva” státusz ikon	11
2.10. „Hiba történt” státusz ikon	11
2.11. A játékbeállítások panelje	12
2.12. „Játék létrehozása” gomb	12
2.13. „Új játék” gomb	13
2.14. „Beállítások frissítése” gomb	14
2.15. Játékálláspanel	14
2.16. Játéktábla	15
2.17. „Következő kör” gomb	15
2.18. „Játék folytatása” gomb	16
2.19. Tanulás lap, Random témakör	16
2.20. Random algoritmus animáció	17
2.21. MENACE tanulási folyamat animációs panel	18
2.22. MENACE játékszimuláció-panel	19
2.23. Minimax játéktábla, kezdőállapotban	20
2.24. Minimax játékfa gyökérrel és a gyökér két gyerekével	21
2.25. Neuronvizualizáció és paraméterek beállításai	22
2.26. Neuronháló kiválasztó gombok	22
2.27. Neuronháló-vizualizáció és paraméterek beállításai	23

2.28. Genetikus algoritmus, kiválasztás operátorok	24
2.29. Genetikus algoritmus, keresztezés operátorok	25
2.30. Genetikus algoritmus, mutáció operátor	26
2.31. Aktivációs függvények	27
2.32. Veszteségfüggvények	28
2.33. Visszaterjesztés egy lépése	29
2.34. Regisztrációs lap	30
2.35. Bejelentkezési lap	31
2.36. Játékosválasztás bejelentkezett felhasználó számára	32
2.37. Barkácsolás lap	33
2.38. Neurális hálózat létrehozása űrlap	33
2.39. Neurális hálózat táblázat	34
2.40. Neurális hálózat szerkesztő főmenü	35
2.41. Neurális hálózat megjelenítőpanel	36
2.42. Neurális hálózat tanítópanel	37
2.43. Neurális hálózat tanításának státusza	37
2.44. Az értesítés megjelenése	40
3.1. A látogató használatieset-diagramja	43
3.2. A bejelentkezett felhasználó használatieset-diagramja	44
3.3. A prezentációs réteg fő komponensei és kapcsolataik	53
3.4. Az AuthContext által az api példányra regisztrált Axios interceptor- ok működése	58
3.5. Az üzletilogika-réteg komponens diagramja	64
3.6. A FastAPIApp osztály	66
3.7. A System Router	68
3.8. A User Router	69
3.9. A Network Router	70
3.10. A Job Router	71
3.11. A Game Router	72
3.12. Az Authenticator osztály	73
3.13. A GameSessionManager osztály	74
3.14. A WebSocketManager osztály	75
3.15. A PlayerFactory osztály	76

3.16. A TrainerWorker modul	77
3.17. A TrainingJobManager osztály	78
3.18. A DefaultPlayerMaker osztály	79
3.19. A BotTaskManager osztály	80
3.20. A PeriodicTaskManager osztály	80
3.21. A BackpropTrainer osztály	81
3.22. A GeneticTrainer modul	82
3.23. A MenaceTrainer osztály	83
3.24. Az utils csomag	84
3.25. A GameState osztály	85
3.26. A játék állapotgépe	85
3.27. A Player absztrakt osztály	86
3.28. Adatelérést kezelő osztályok	96
3.29. ER diagram	97
3.30. Automatikus tesztek	99
3.31. Terhelési teszt eredményei	102
3.32. Soak teszt eredményei	103

Táblázatok jegyzéke

2.1. Navigációs menü áttekintése	7
2.2. Csatlakozási státuszok	10
2.3. Játékbeállítások megszorításai	12
2.4. Játékbeállítások módosítása játék közben	13
2.5. MENACE animáció lépései	18
2.6. MENACE szimuláció műveletei	19
2.7. Regisztrációs mezők megszorításai	30
2.8. Neurális hálózat létrehozásának megszorításai	33
2.9. Neurális hálózat szerkesztése és megszorításai	35
2.10. Neurális hálózat tanítópanel értékei és megszorításai	36
2.11. Bemeneti mezők megszorításai	39
3.1. Felhasználói történetek	50
3.2. Injektált függőségek	83
3.3. Az adatbázis entitások mezőinek elvárt Python típusai	98
3.4. Automatizált funkcionális tesztek	101
3.5. Manuális funkcionális tesztek	104

Algoritmusjegyzék

1.	MENACE tanító algoritmus - Mesterséges intelligencia felhasználásával	89
2.	Minimax rekurzív függvény - Mesterséges intelligencia felhasználásával	91
3.	Visszaterjesztéses tanítás algoritmus - Mesterséges intelligencia felhasználásával	93
4.	Genetikus algoritmus - Mesterséges intelligencia felhasználásával . . .	94